
AI & CHATBOT

Revisão Chatbot e Introdução ao Aprendizado de Máquina

Prof. Henrique Ferreira • Prof. Miguel Bozer
Prof. Guilherme Aldeia • Prof. Michel Fornaciali
Prof. Daniel Gomes • Prof. Daniel Petrini
Prof. Felipe Amaral • Prof. Vinicius Holanda

GRADUAÇÃO FIAP

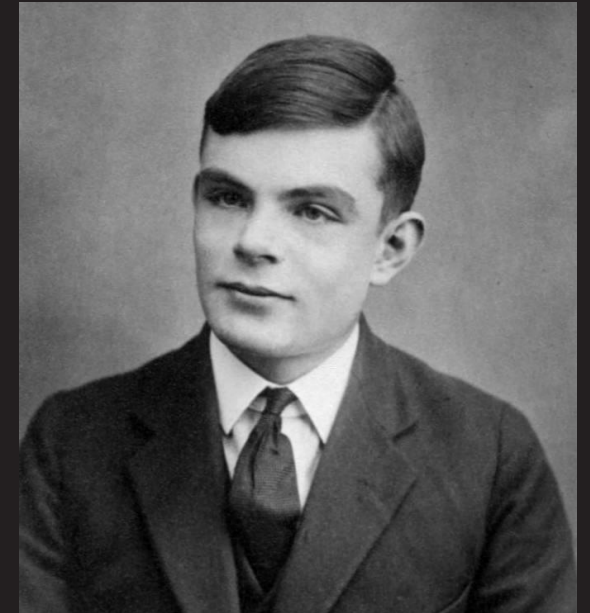


Chatbots

Uma revisão geral daquilo que vimos até agora

Chatbots - História

A ideia de computadores imitando humanos é proposta formalmente como uma maneira de medir inteligência por Turing em 1950, e já em 1964-1966 temos o primeiro chatbot (DOCTOR/ELIZA) criado por Weizenbaum.



Alan Turing (1912-1954)

Teste de Turing



Joseph Weizenbaum
(1923-2008)

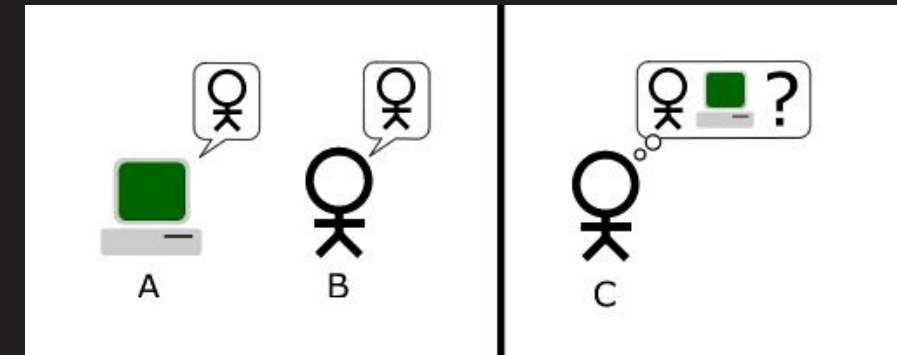
```
Welcome to
          EEEEE LL   IIII ZZZZZZ AAAAA
          EE    LL   II   ZZ   AA  AA
          EEEEE LL   II   ZZ   AAAAAA
          EE    LL   II   ZZ   AA  AA
          EEEEE LLLLL IIII ZZZZZZ AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

<https://web.njit.edu/~ronkowit/eliza.html>

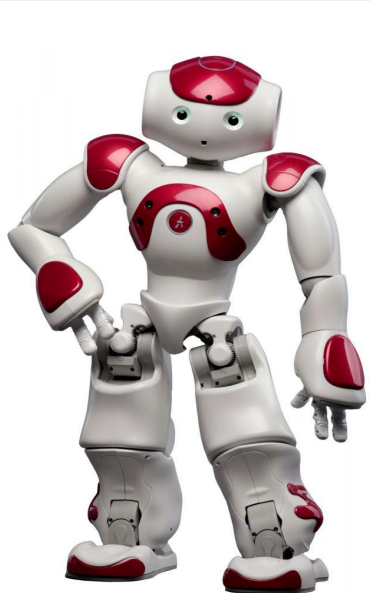
<http://psych.fullerton.edu/mbirnbaum/psych101/Eliza.htm>



Chatbots - Aplicações

- Chatbots podem ser embarcados em robôs, em dispositivos celulares, em carros e em diferentes dispositivos de IoT. Eles estão presentes em aplicativos de mensagens e em redes sociais

Robôs de Companhia



NAO, 2006



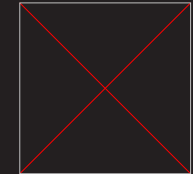
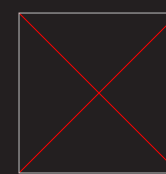
Pepper, 2014



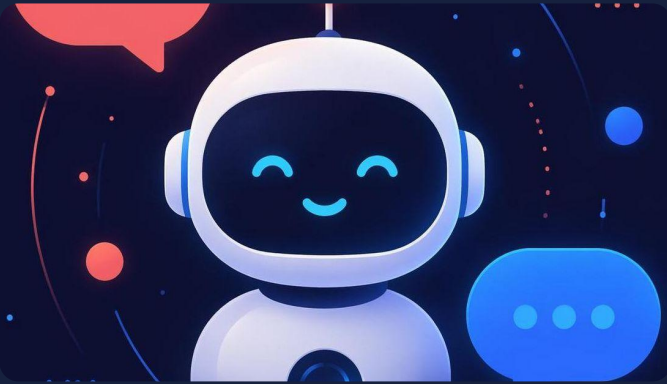
Alexa, 2014



Sophia, 2015



Chatbots – Definição



Agente Conversacional

Software que interage com humanos via conversa (texto ou fala) em um ambiente de interação humano-computador (HCI)



Interfaces Diversas

Podem ser integrados a robôs, dispositivos IoT, drones e até carros como interface humano-sistema



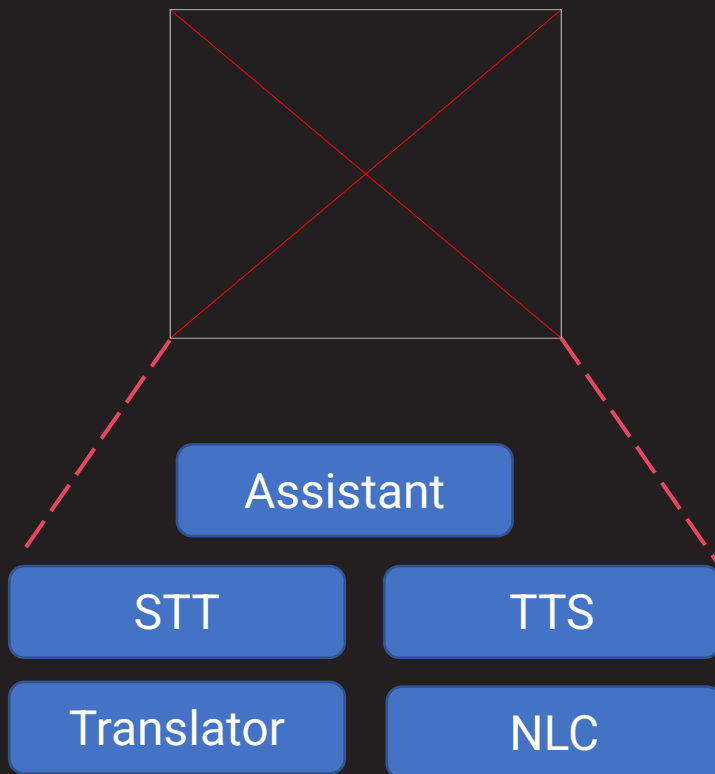
Texto e Fala

A conversa pode ser feita via texto (chat) ou fala (assistentes virtuais), usando NLU para compreensão e NLG para geração

Chatbots – Como fazer?

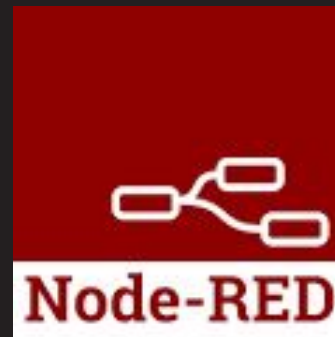
- Nós vimos como criar bots para o Telegram usando Node-RED e a plataforma Watson da IBM:

Serviços de IA



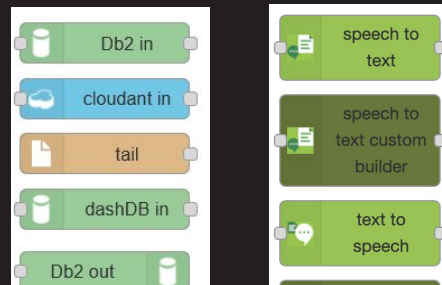
+

Desenvolvimento
da Aplicação



+

Interface com o
Usuário



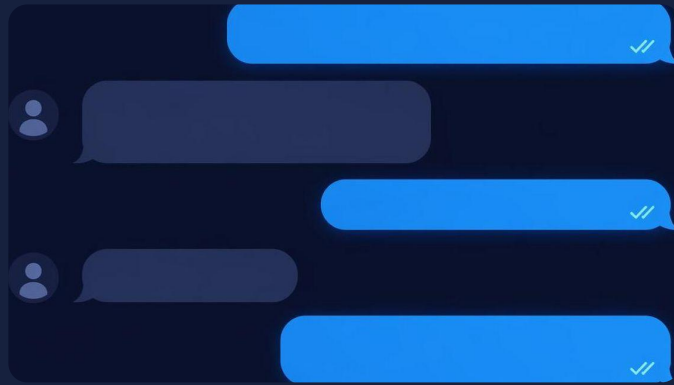
Node-RED Library

Chatbots – Arquitetura



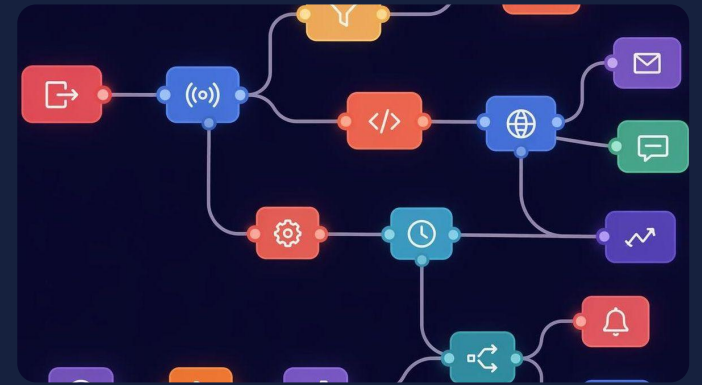
Nuvem da IBM

Soluções de IA (Watson Assistant) rodam na infraestrutura cloud da IBM, processando NLU e gerando respostas



Interface (Telegram)

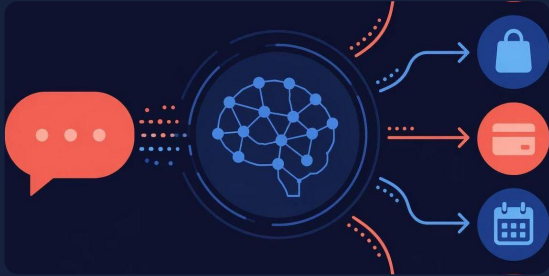
O frontend do chatbot roda na nuvem do Telegram, conectando o usuário à inteligência do backend via API



Node-RED

Orquestrador de fluxos que conecta os serviços. Pode rodar localmente ou na nuvem, integrando Watson, Telegram e outros

Watson Assistant – Componentes



Intenções

Frases que capturam a intenção geral do usuário. Requer quantidade e variedade de exemplos.



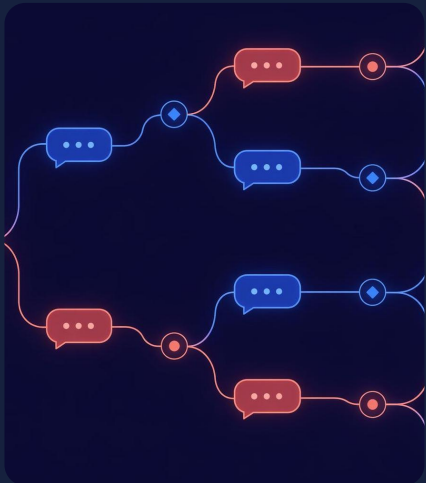
Entidades

Palavras ou padrões que capturam variáveis do diálogo. Inclui sinônimos e expressões regulares.



Variáveis de Contexto

Armazenam informações durante a conversa para personalizar as respostas.



Árvore de Diálogo

Fluxo de conversação que determina caminhos de resposta baseados em intenções e entidades detectadas.



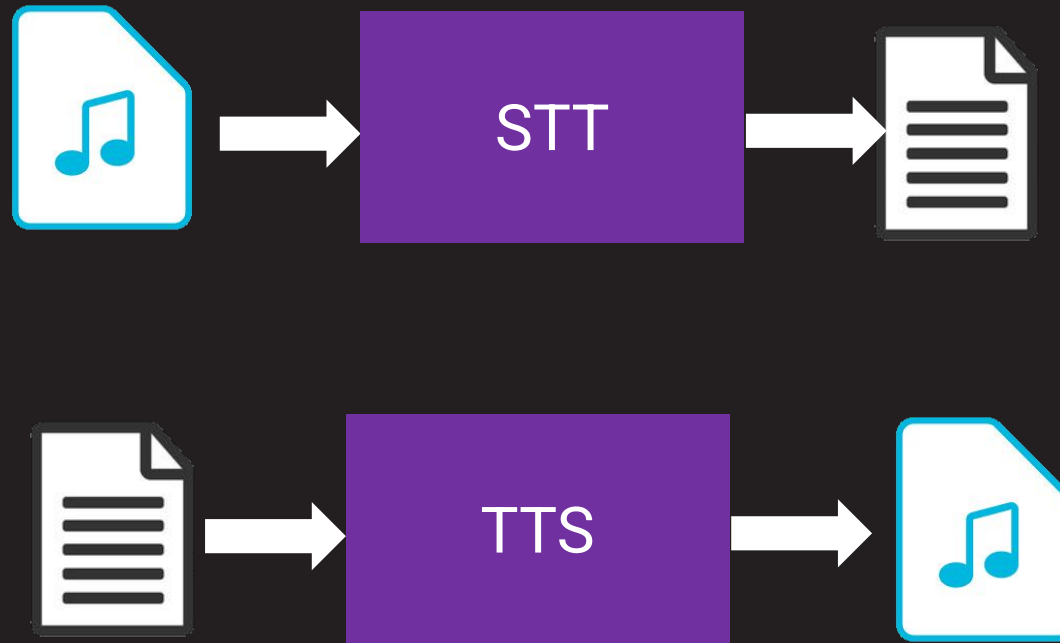
Curadoria

Análise e melhoria contínua das conversas para otimizar a qualidade das respostas.

Chatbots – STT e TTS



- Serviços para reconhecer fala (STT) e para sintetizar fala (TTS);



LLMs – O Panorama Atual (2025/2026)

Principais Modelos

- **GPT-5** (OpenAI, ago/2025) – modelo unificado com raciocínio integrado e roteamento automático
- **Claude 4** (Anthropic, mai/2025) – líder em codificação e agentes autônomos (SWE-bench: 72,5%)
- **Gemini 3** (Google, nov/2025) – multimodal nativo com 650M+ usuários/mês
- **LLaMA 4** (Meta, 2026) – open-source com 10M tokens de contexto e 128 experts (MoE)
- **DeepSeek R1/V3** (China, 2025) – open-source de alto desempenho, revolucionando o acesso

Modelos de Raciocínio

- Nova categoria: modelos que “pensam antes de responder” (chain-of-thought) – OpenAI o3/o4-mini, Claude com extended thinking

Tendências-Chave

- **IA Agêntica:** agentes autônomos que executam tarefas complexas de forma contínua. Gartner prevê 40% dos apps corporativos com agentes até 2026
- **Multimodalidade:** texto, imagem, áudio e vídeo integrados nativamente em um único modelo
- **Open-Source:** modelos abertos (LLaMA 4, DeepSeek) competem com modelos proprietários, democratizando o acesso
- **Contexto longo:** janelas de até 10 milhões de tokens permitem processar livros inteiros e repositórios de código

Do GPT-3.5 ao GPT-5

- Em 2,5 anos, saímos do ChatGPT original (nov/2022) para modelos com raciocínio avançado, uso de ferramentas, geração de imagens e capacidade de agir autonomamente no mundo real

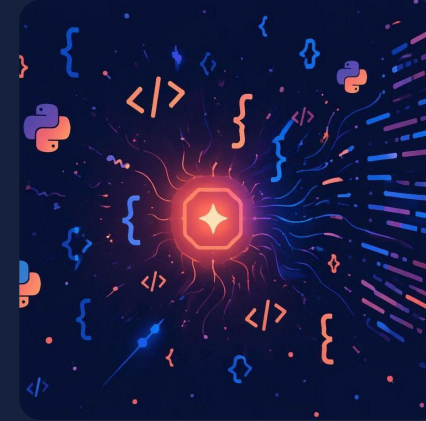
Atualizado em maio/2026

O que os LLMs podem fazer?



Conversação Natural

Diálogos complexos e contextuais, comparáveis a interações humanas



Geração de Código

Criação de scripts em Python e outras linguagens a partir de instruções em texto



Análise Crítica

Capacidade de avaliar, comparar e explicar conceitos técnicos e científicos



Consciência de Viés

Discussão sobre limitações, vieses nos dados e ética em IA

Reflexões sobre LLMs

Pontos importantes sobre modelos de linguagem

Viés nos Dados

LLMs podem reproduzir preconceitos presentes nos dados de treinamento. É essencial avaliar criticamente as respostas geradas.

Neutralidade Aparente

Modelos afirmam ser neutros, mas suas respostas são moldadas pelos dados e objetivos de treinamento. Neutralidade total não existe.

Ferramenta, não Oráculo

LLMs são ferramentas poderosas para produtividade, mas não substituem o pensamento crítico. Sempre valide as informações geradas.

Evolução Constante

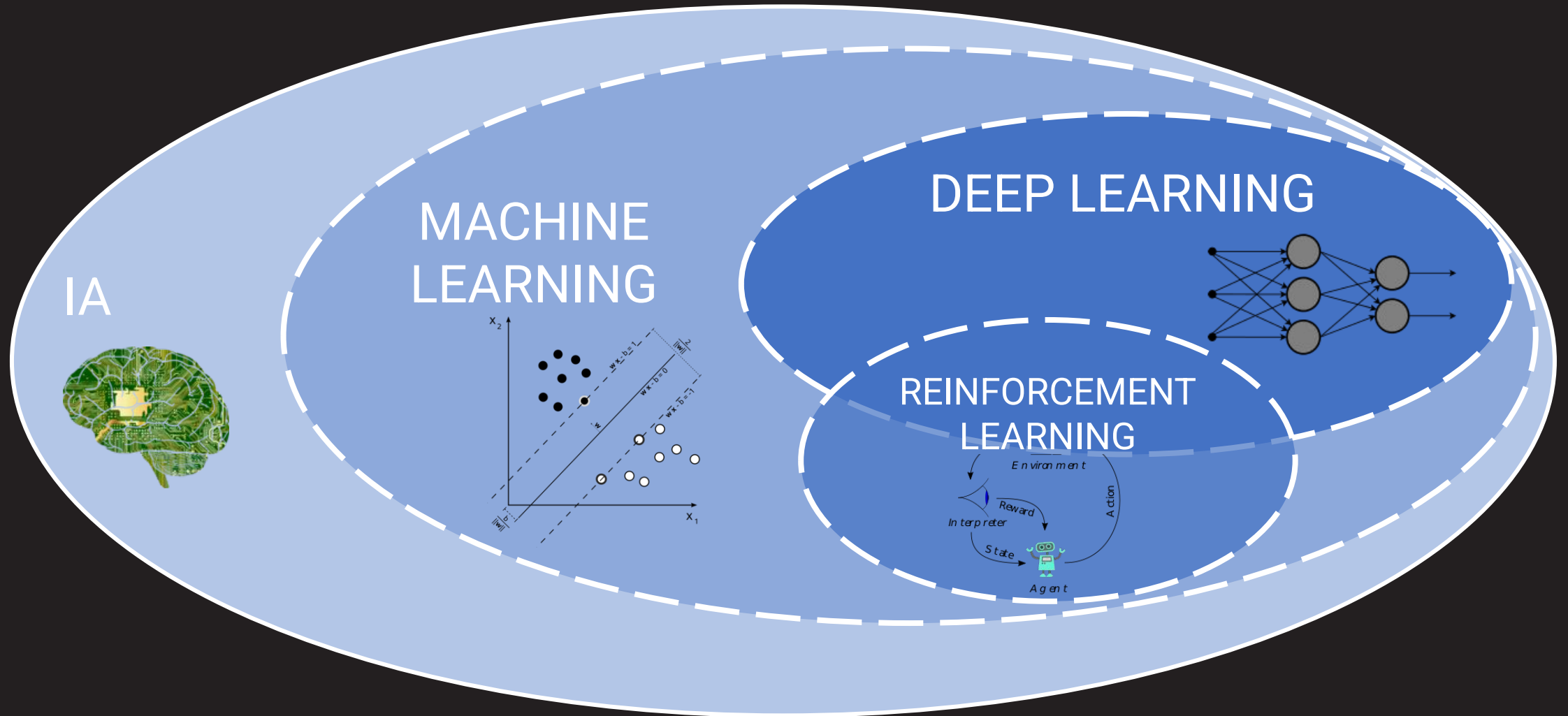
De GPT-3 a modelos multimodais: a área evolui rapidamente. Acompanhar essas mudanças é parte essencial da formação em IA.



Ensinando um computador

Introdução ao Aprendizado de Máquina

Inteligência Artificial



Aprendizado de Máquina (Machine Learning - ML)

- Alan Turing, abordagem cognitiva:

"As máquinas são capazes de pensar?"

- Tom Mitchell, abordagem operacional:

"As máquinas são capazes de fazer o que (nós como entidades pensantes) podemos fazer?"

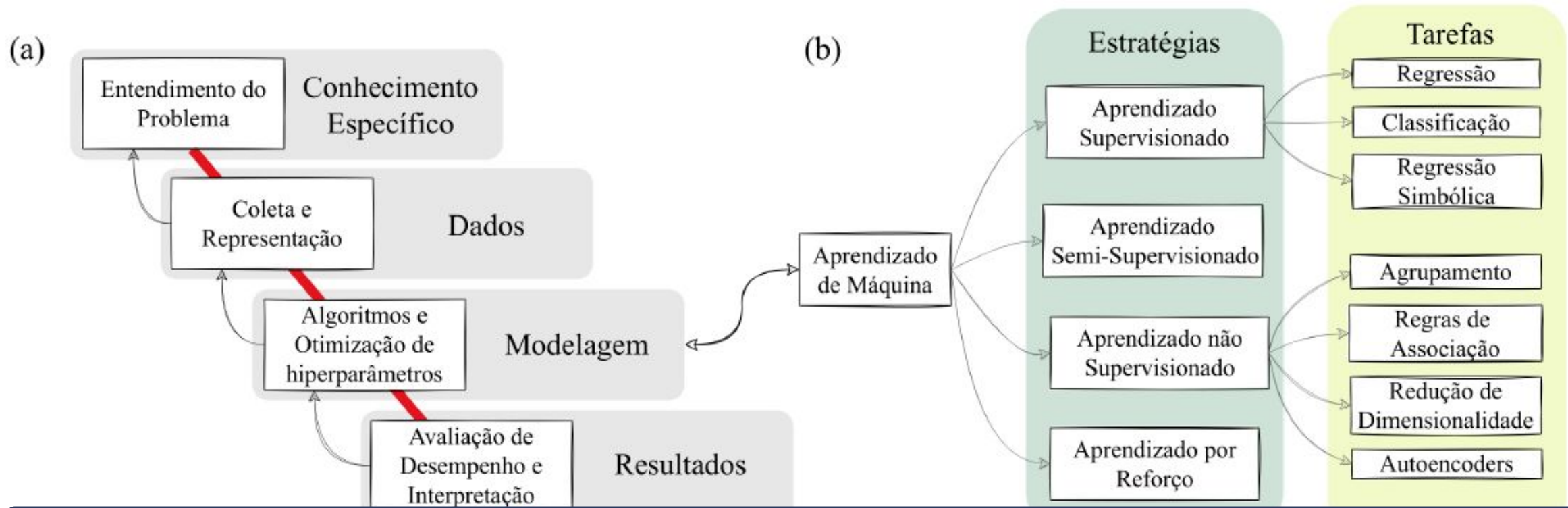
- Arthur Samuel, definição de Aprendizado de Máquina:

"Campo de estudo que dá aos computadores a habilidade de aprender sem serem explicitamente programados".

Aprendizado de Máquina

H. Ferreira, E. F. Almeida, W. Espinosa-Garcia, E. Novais, J. N. B. Rodrigues, G. M. Dalpian

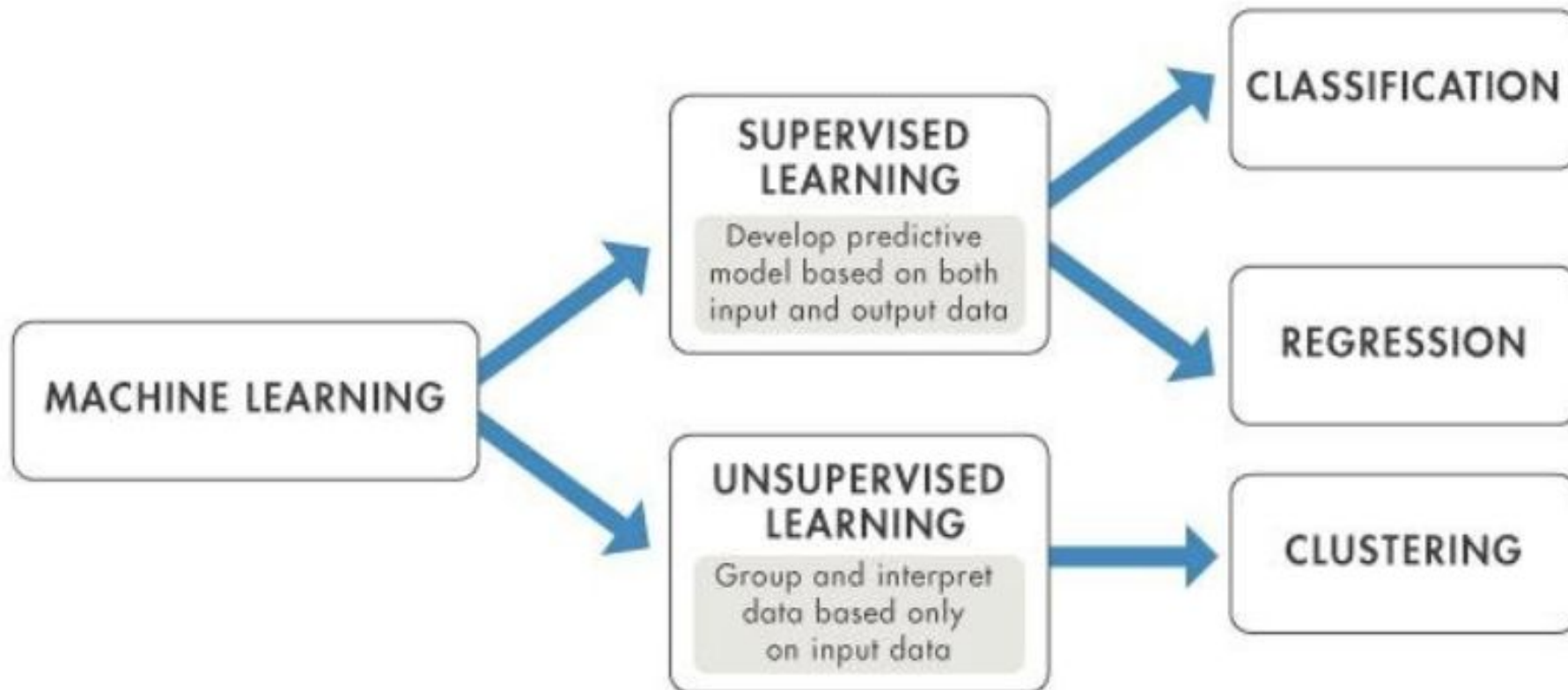
XXXX-5



Taxonomia do Aprendizado de Máquina

Diagrama ilustrando as principais categorias e subcategorias do aprendizado de máquina

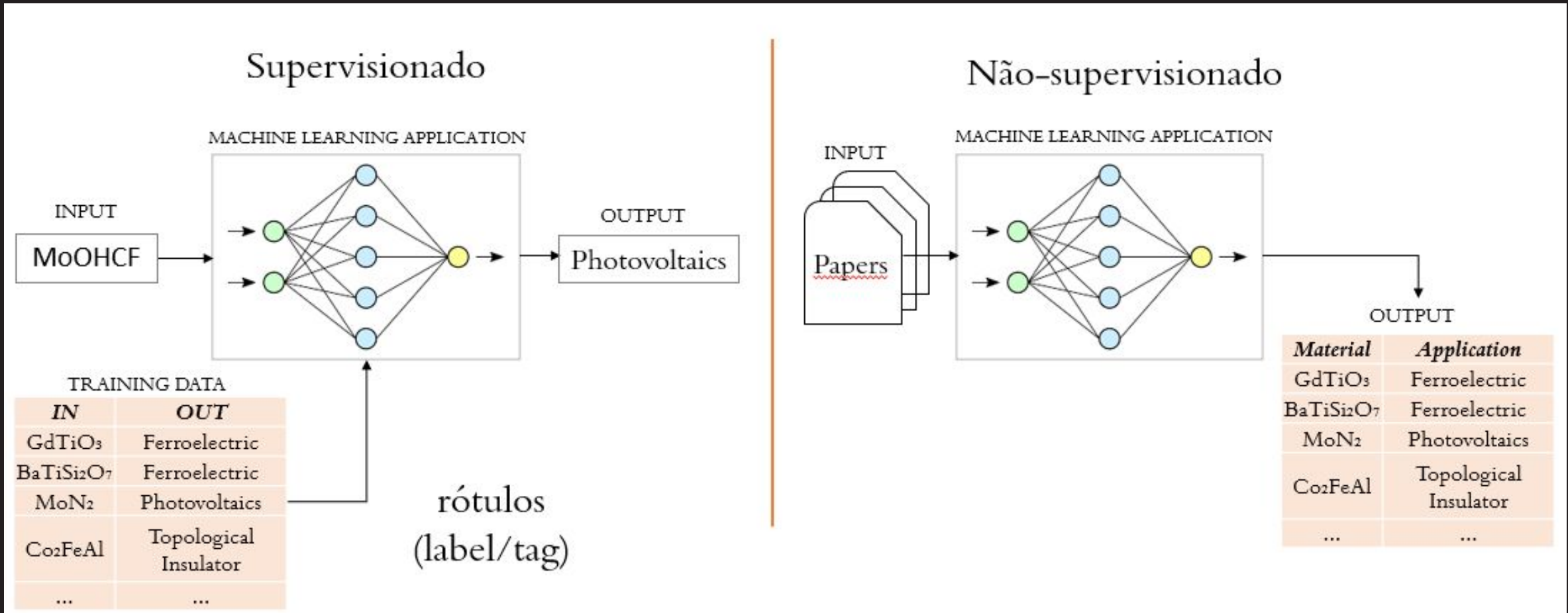
Aprendizado de Máquina



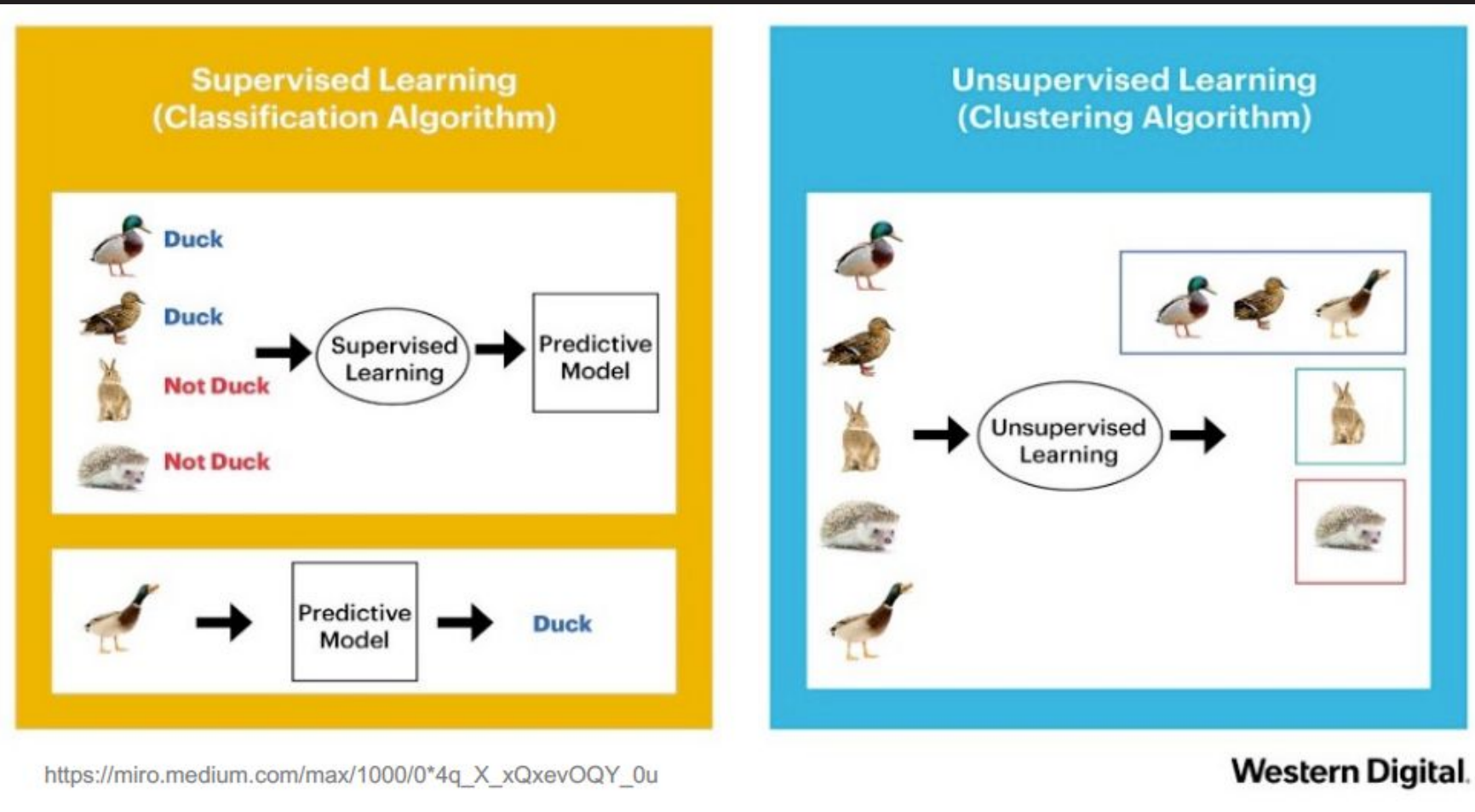
Visão Geral do ML www.researchgate.net/profile/Rory-Bunker/publication/319937079/figure/fig1/AS-542578279366656@1506372150206/Supervised-Learning-versus-Unsupervised-Learning-Mathworks-nd.png

Fluxo de trabalho típico e aplicações do aprendizado de máquina em diferentes domínios

Aprendizado de Máquina – Supervisionado vs Não Supervisionado



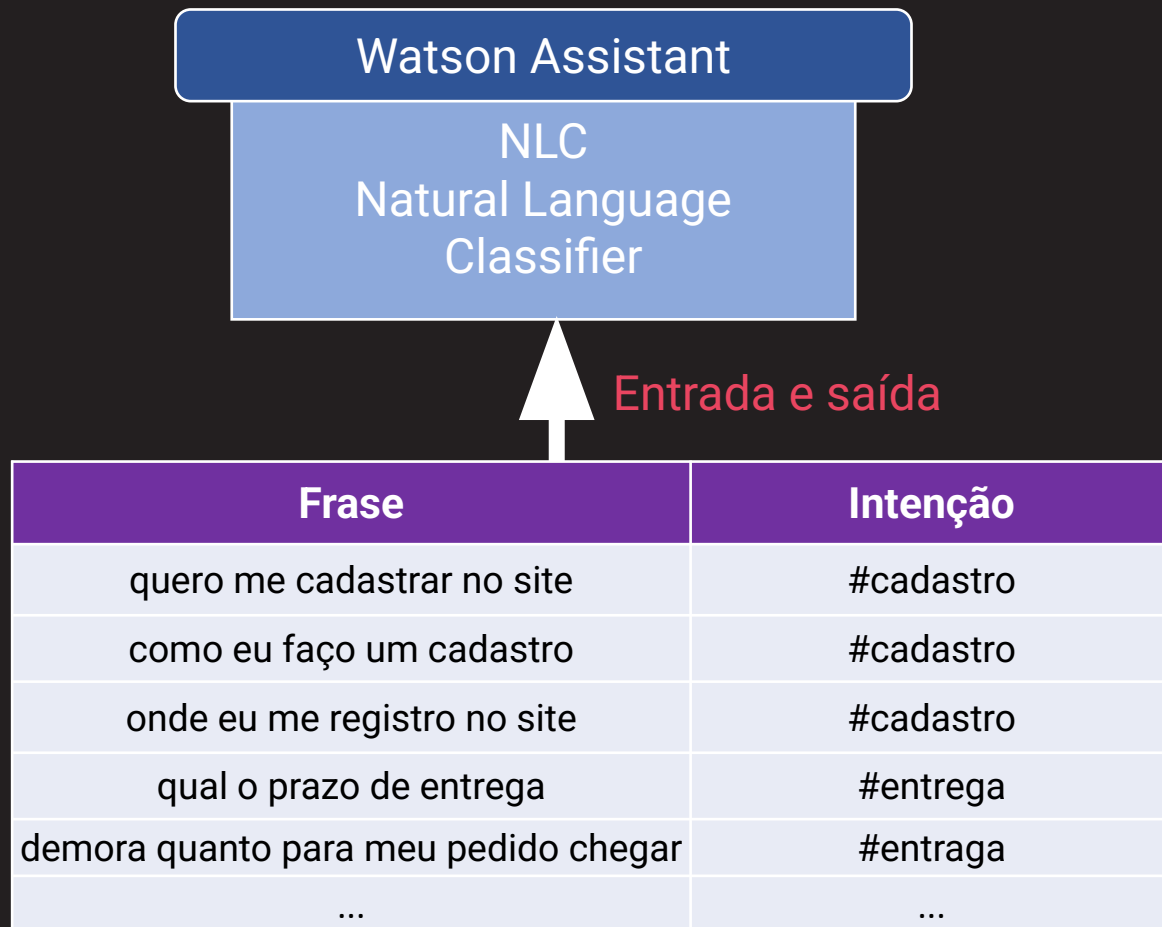
Aprendizado de Máquina – Supervisionado vs Não Supervisionado



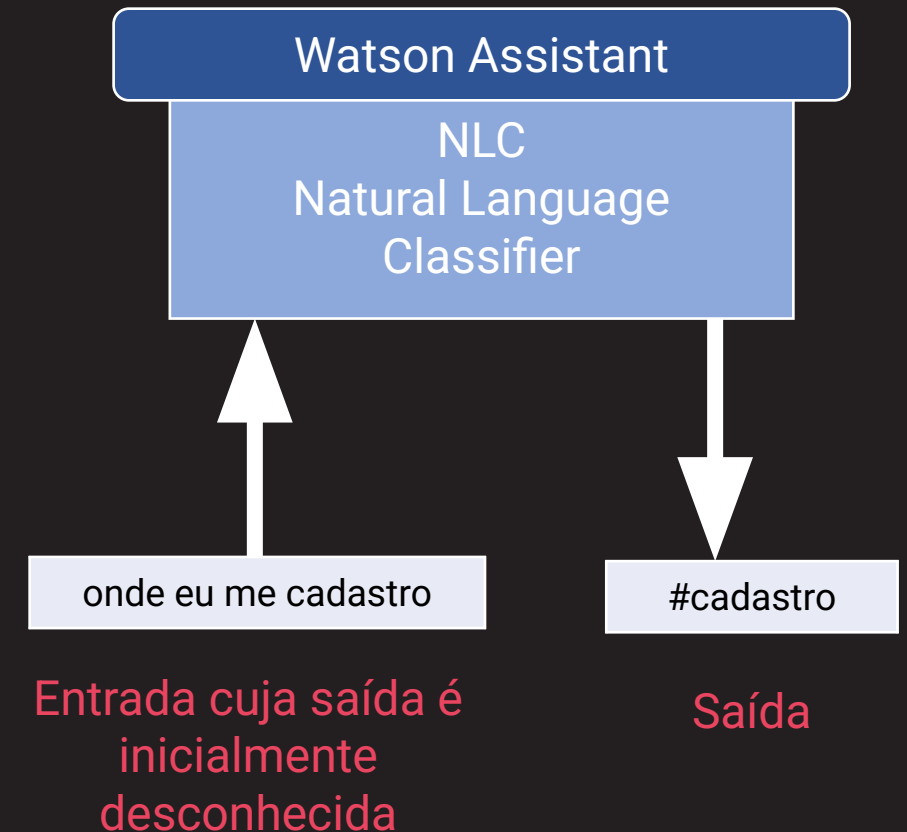
Aprendizado de Máquina – Exemplo: Classificação supervisionada



Treinamento

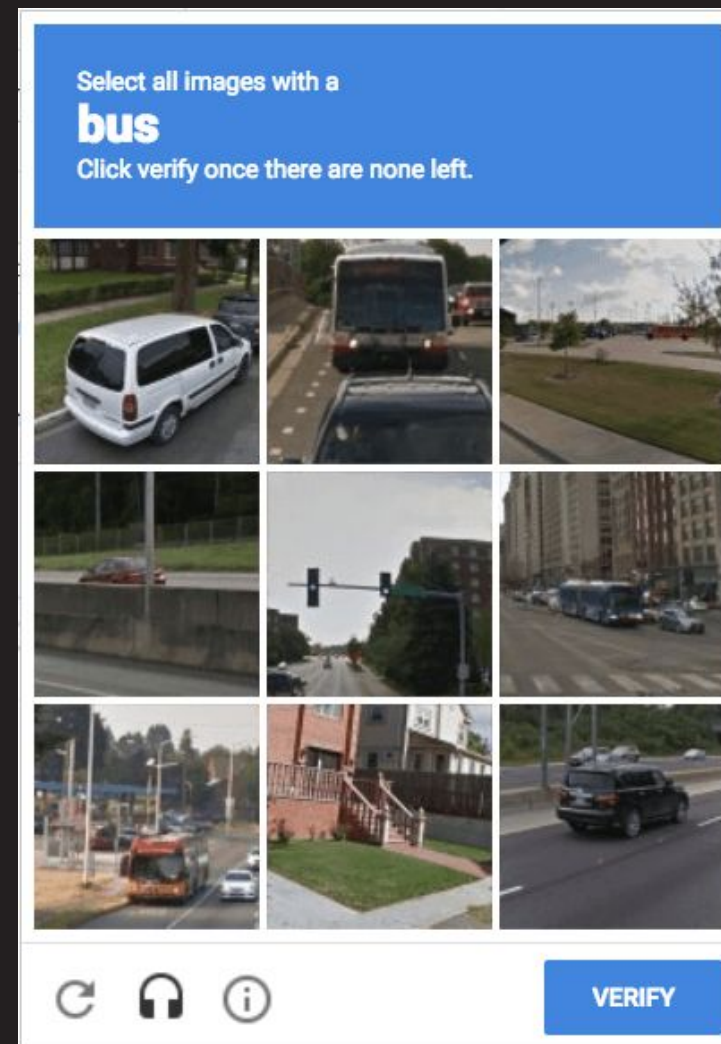


Uso



Aprendizado de Máquina – Supervisionado

- O aprendizado supervisionado tende a ser **mais preciso do que o aprendizado não supervisionado** (desempenho melhor);
- Entretanto, para se fazer treinamento supervisionado é **necessário ter os rótulos** (labels) dos dados;
- O processo de **rotulação dos dados é feito por seres humanos e pode ser altamente custoso e demorado** para determinado tipos de problemas;
- Diversas abordagens podem ser usadas para rotular dados, por exemplo, os CAPTCHAs:



Aprendizado de Máquina



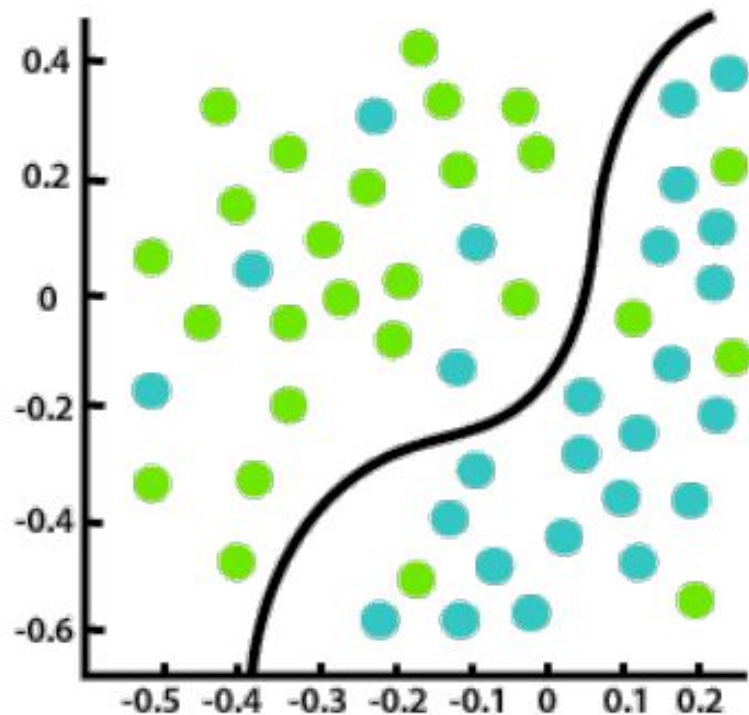
Aprendizado Supervisionado na Prática

Exemplos visuais de classificação e regressão em problemas reais de machine learning

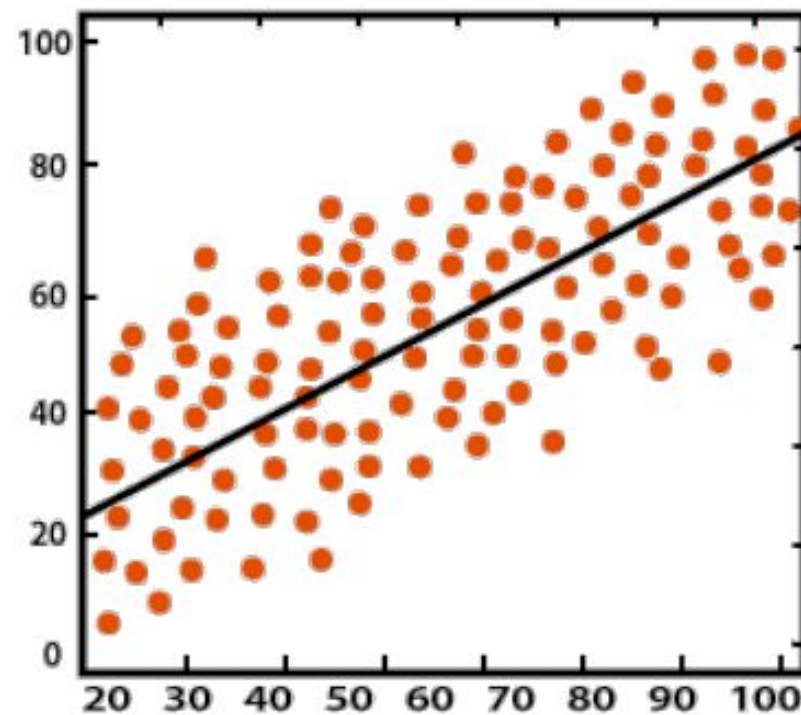


Aprendizado de Máquina – Supervisionado: Regressão vs Classificação

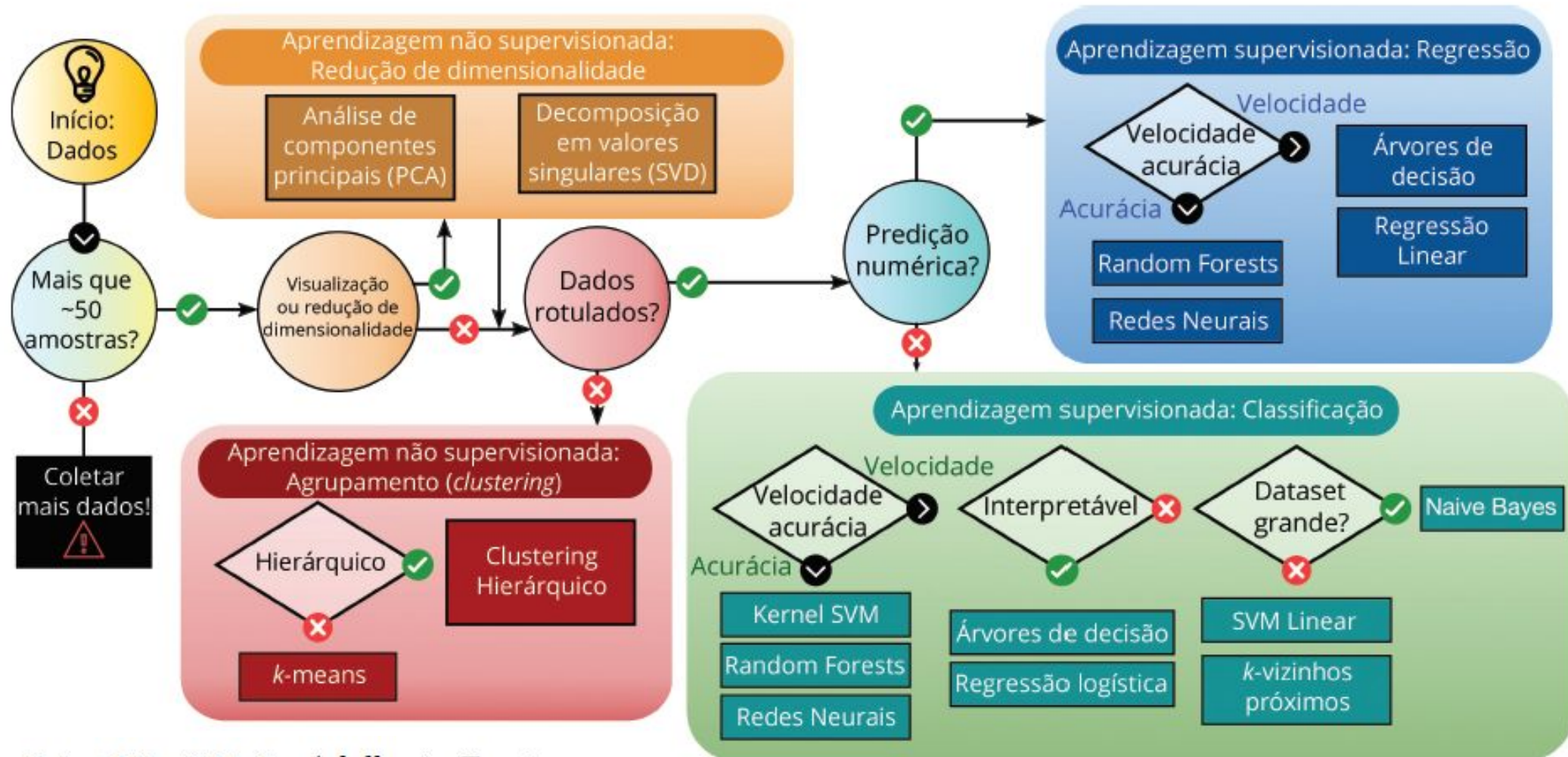
Classificação: predição de classe



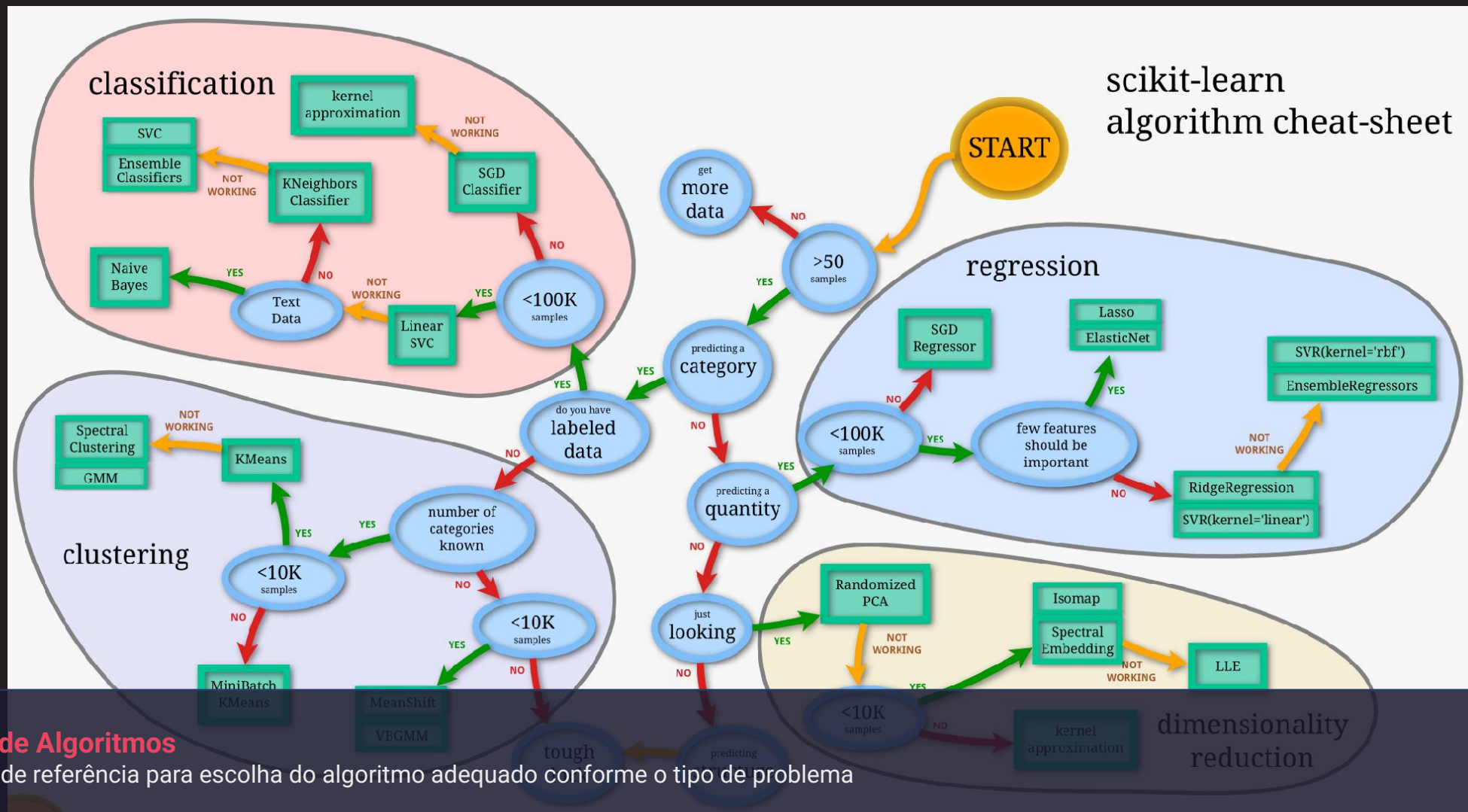
Regressão: predição de valor



Aprendizado de Máquina: guias



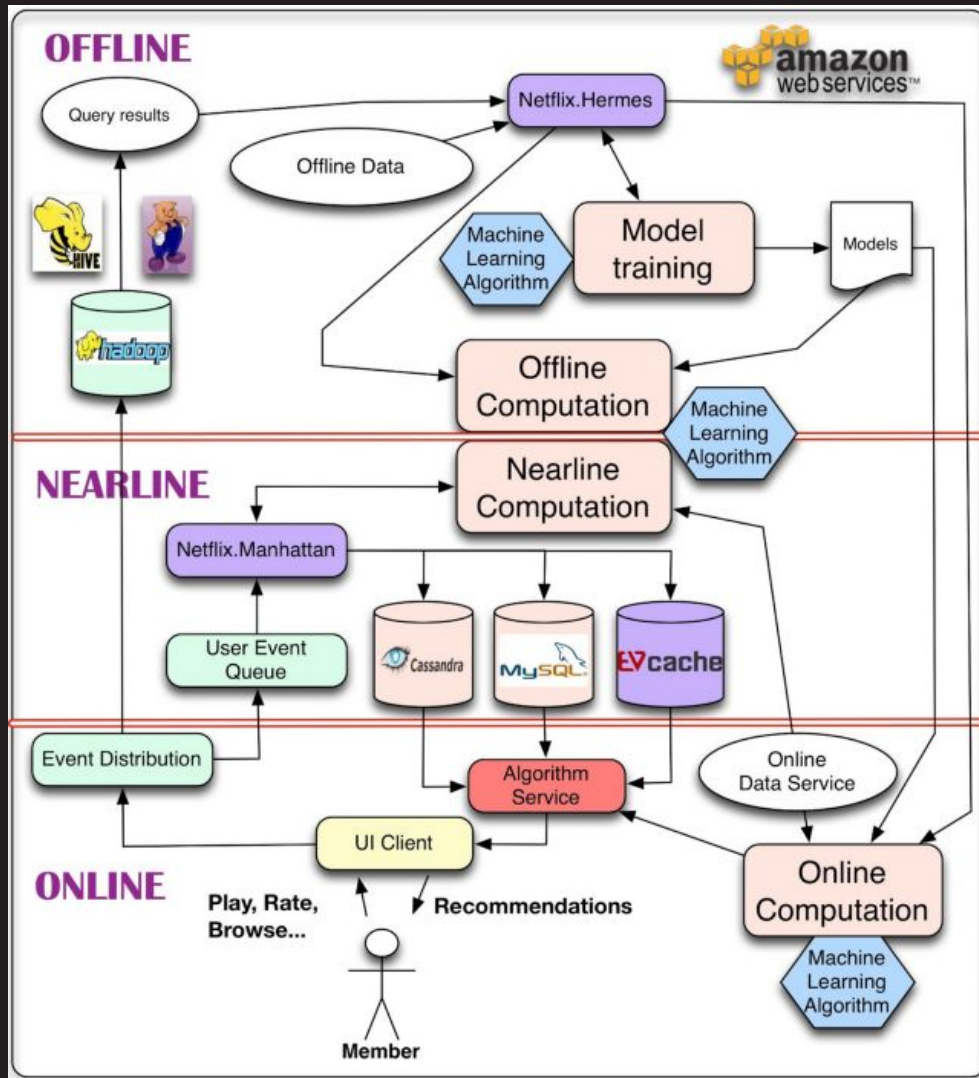
Aprendizado de Máquina: guias



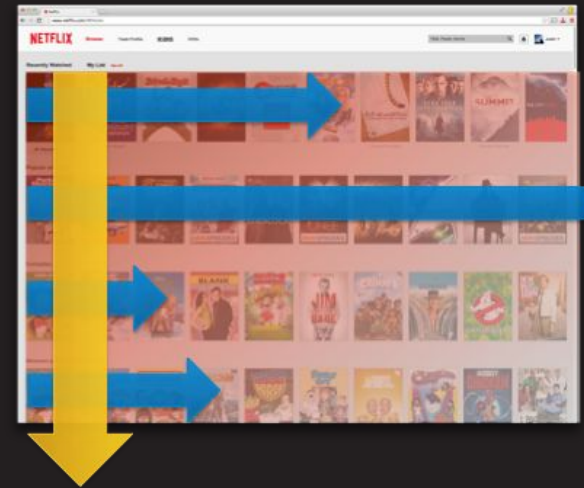
Guia de Algoritmos

Mapa de referência para escolha do algoritmo adequado conforme o tipo de problema

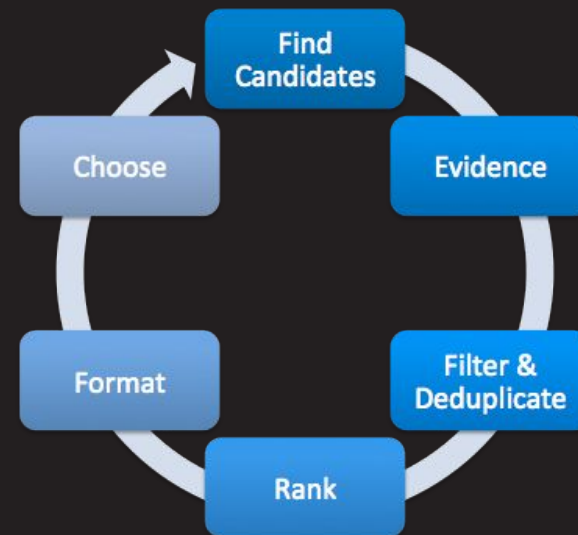
Exemplo aplicação: Sistemas de Recomendação



More likely
to see



Less likely

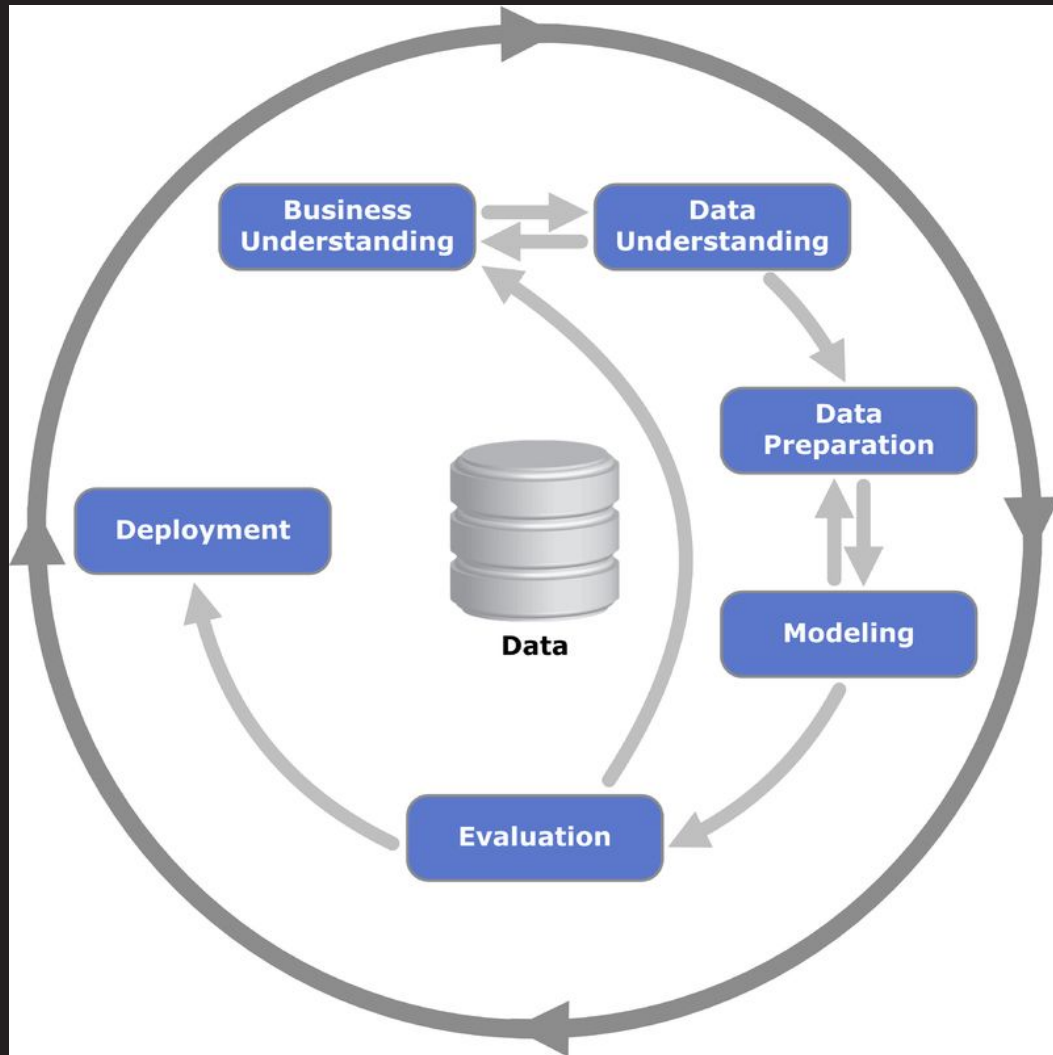


NETFLIX

amazon

TikTok

Metodologias de Trabalho em Aprendizado de Máquina

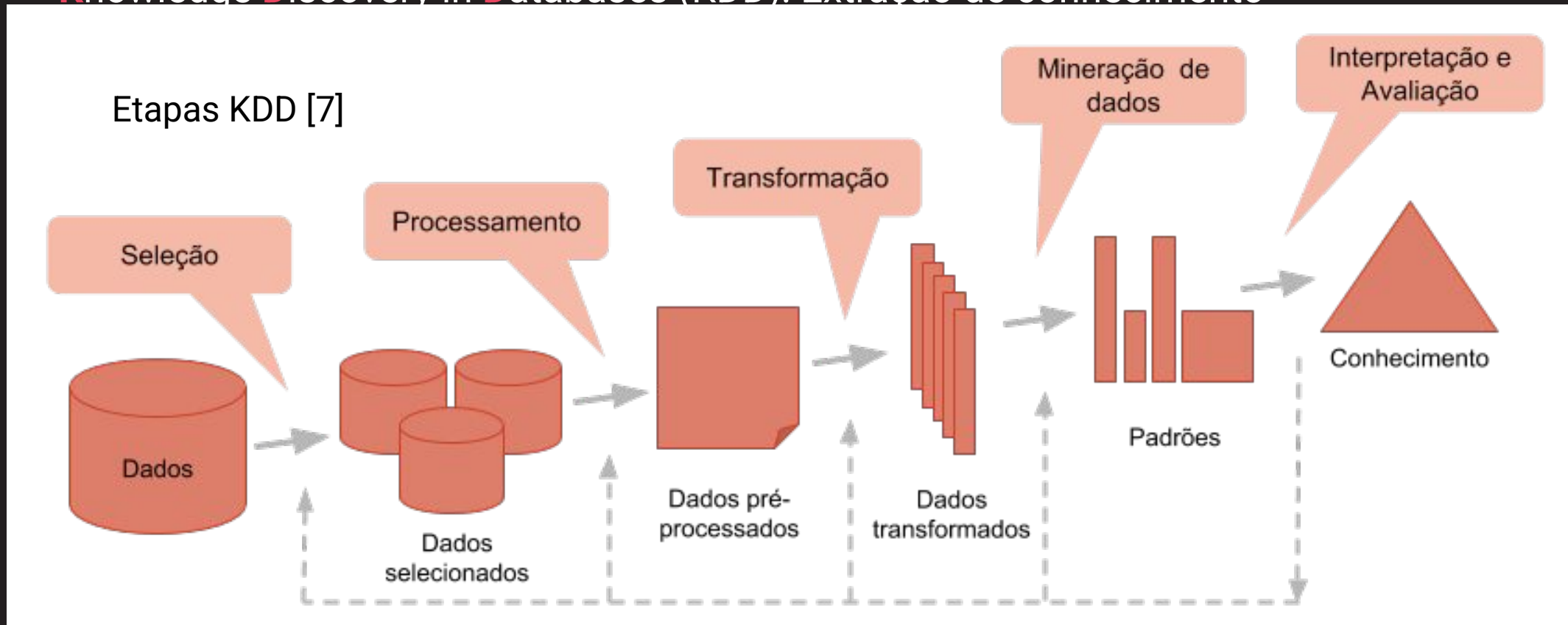


- Ao resolver problemas usando algoritmos de aprendizado de máquina, é comum seguir um certo número de passos, baseados em uma metodologia de trabalho.
- Algumas das metodologias muito usadas são:

Cross **I**ndustry **S**tandard **P**rocess for **D**ata **M**ining (CRISP-DM): Processo Padrão Inter-Indústrias para Mineração de Dados

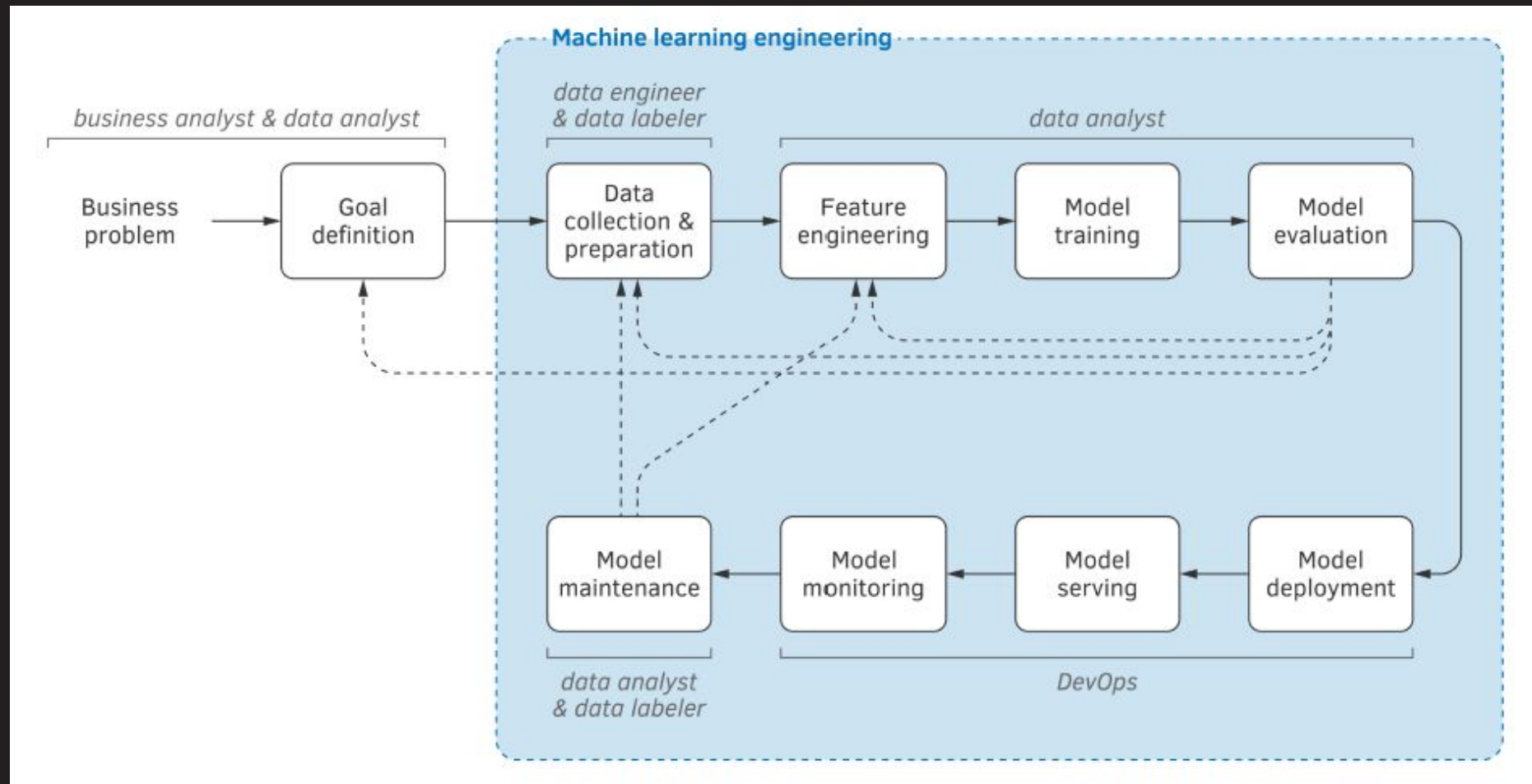
Metodologias de Trabalho em Aprendizado de Máquina

Knowledge Discovery in Databases (KDD): Extração de conhecimento

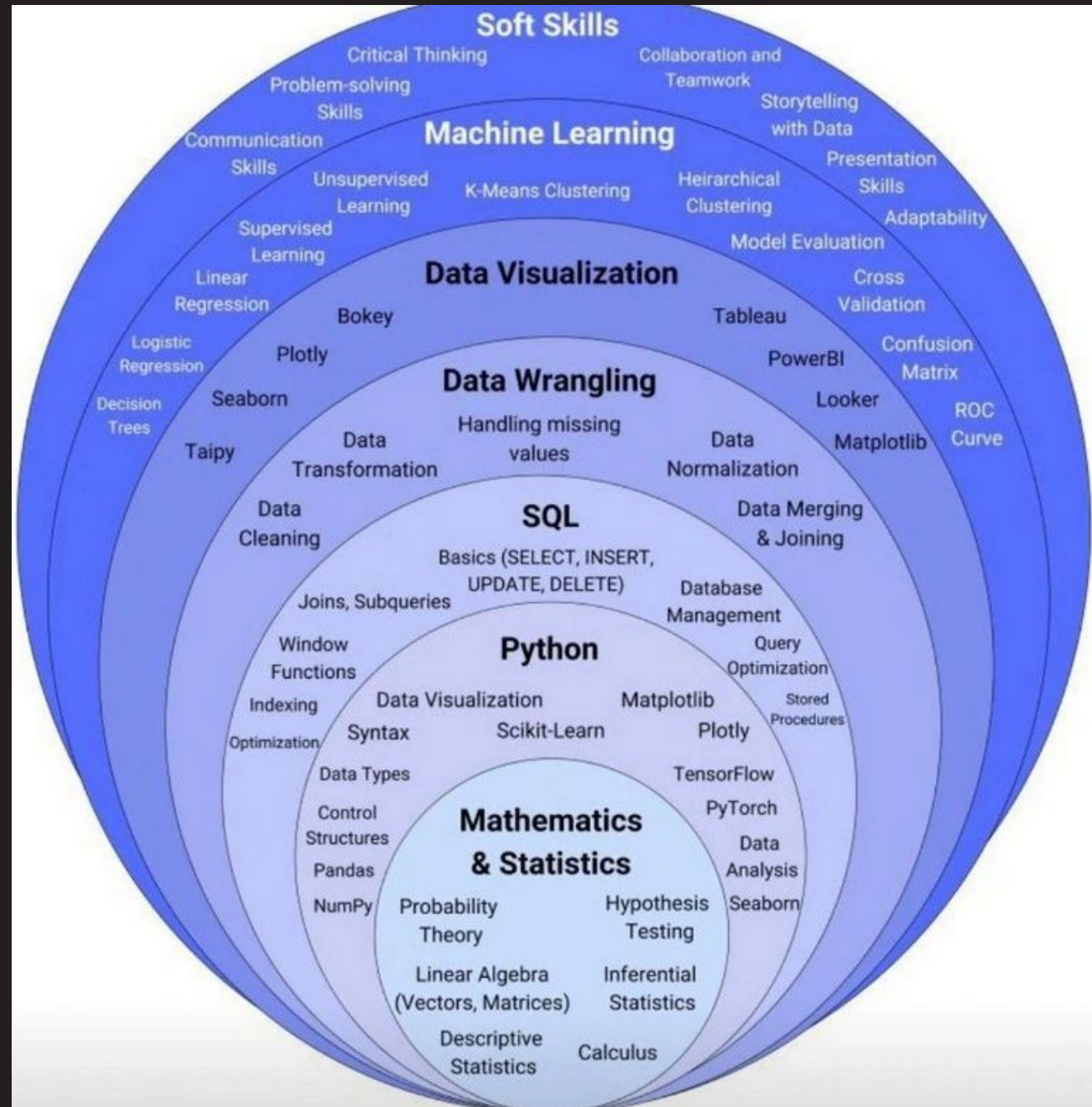


Metodologias de Trabalho em Aprendizado de Máquina

Na engenharia de um sistema que usa Aprendizado de Máquina existem diversas passagens específicas; Há relação direta entre algumas etapas mostradas abaixo e os fluxos KDD e CRISP-DM:



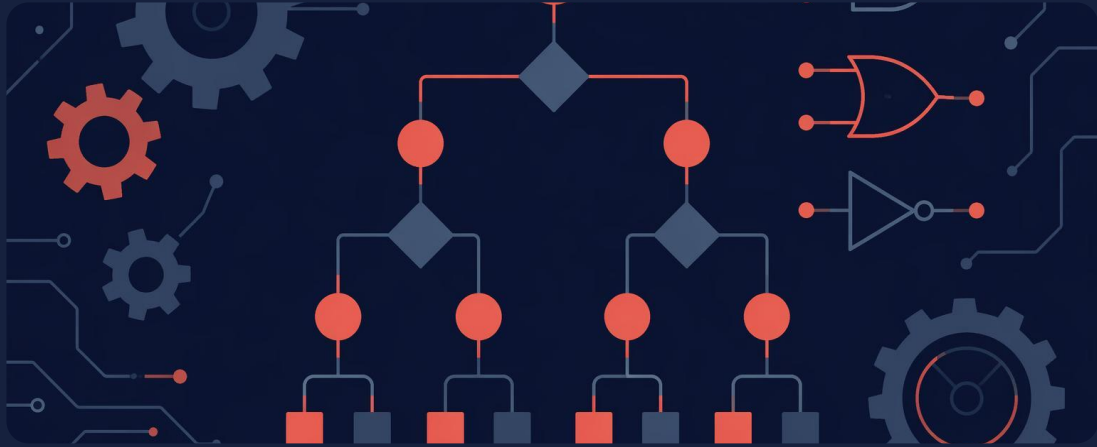
Data Scientist Roadmap



Penso, logo dados!

Representando dados

IA Clássica vs Aprendizado de Máquina



IA Clássica

- Inteligência nos algoritmos
- Programação explícita
- Requer forte adaptação para cada problema
- Capacidade de generalização limitada

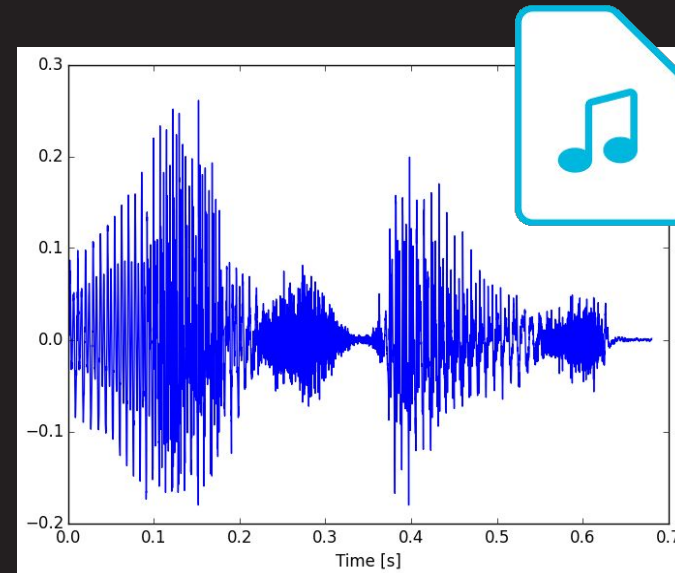
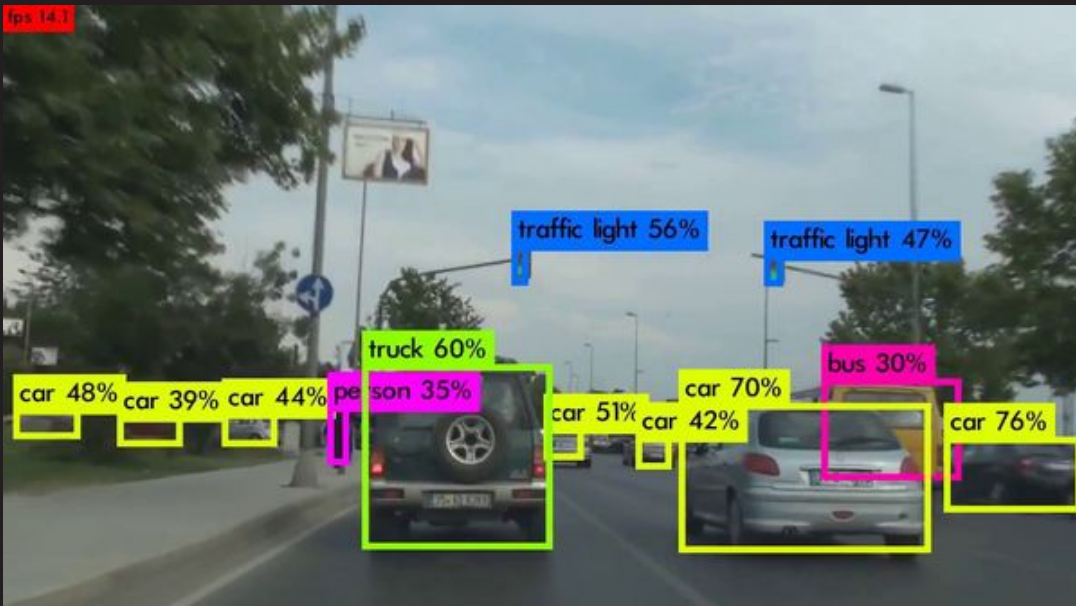


Aprendizado de Máquina

- Inteligência extraída dos dados
- Aprendizado por exemplos
- Alta capacidade de generalização
- Dados são o recurso mais valioso

Dados: Estrutura/Tipos/Representação

- Os dados podem estar armazenados de diferentes maneiras:
 - Estruturada: tabelas;
 - Semiestruturada: HTML, XML, JSON;
 - Não estruturada: texto, imagens, áudio;

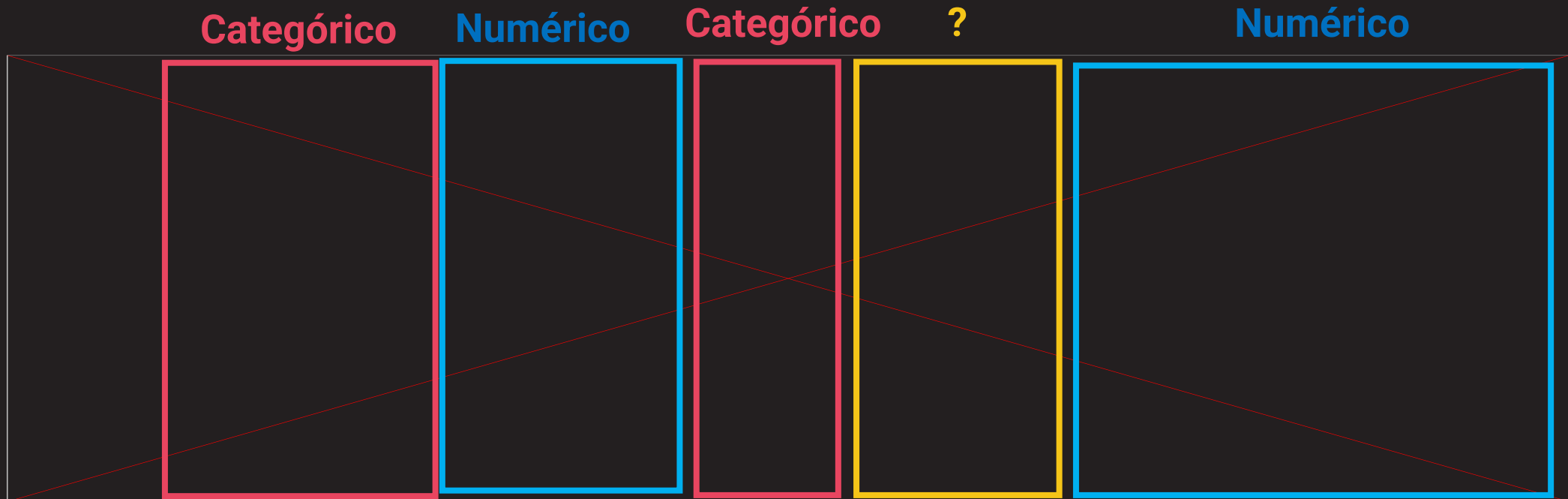


```
<!DOCTYPE html>
<html lang=en>
<meta charset=UTF-8>
<title>Introduction to
The Mating Rituals of
Bees</title>
<h1>Introduction</h1>
<p>This companion guide to
the highly successful
<cite>Introduction to
Medieval Beekeeping</cite>...
```

HTML

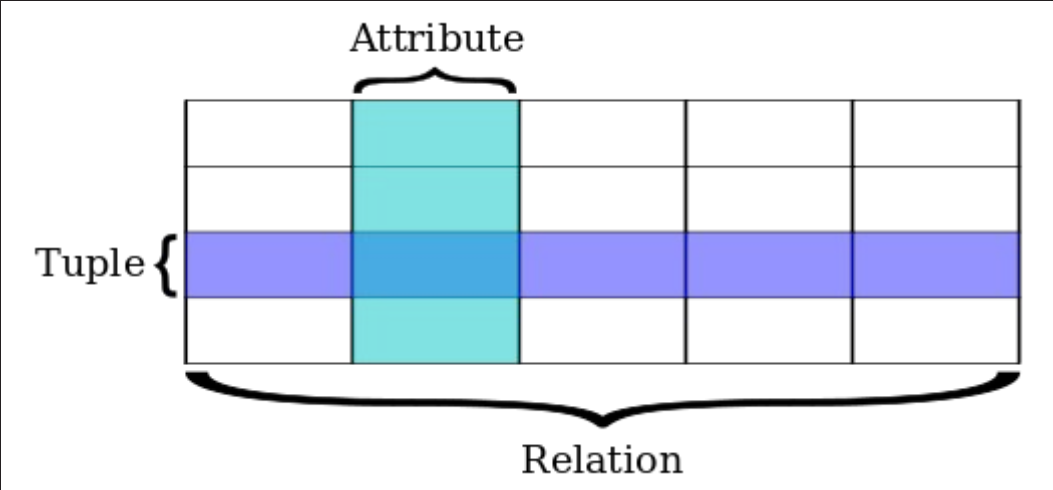
Dados: Estrutura/Tipos

- Além disso, dados podem ser:
 - Numéricos
 - Categóricos



Dados: Tabelas

- Para visualizar dados, TABELAS são a forma tradicional



$$f(x_1, x_2, \dots, x_N)$$

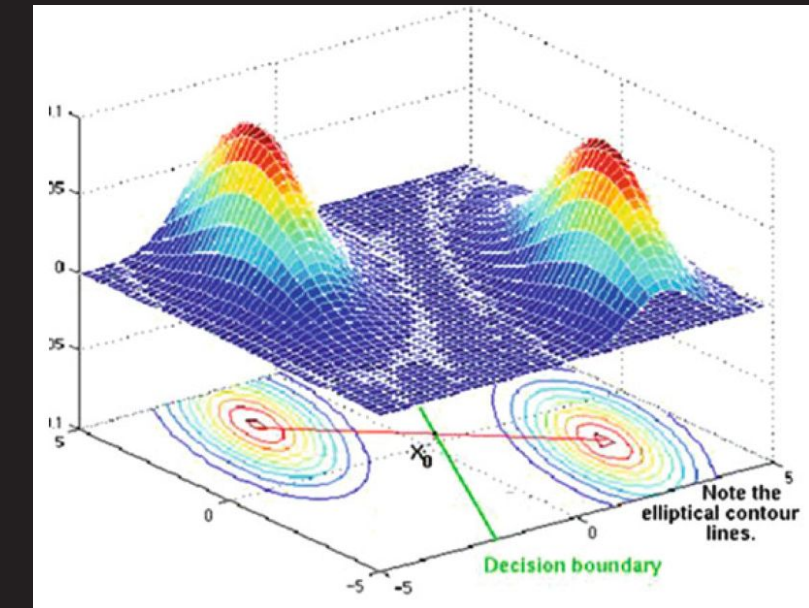
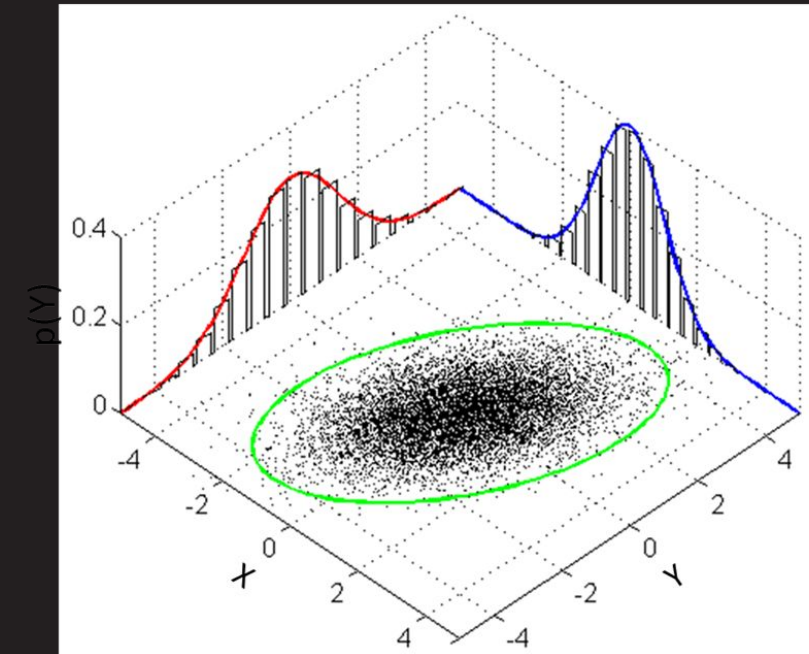
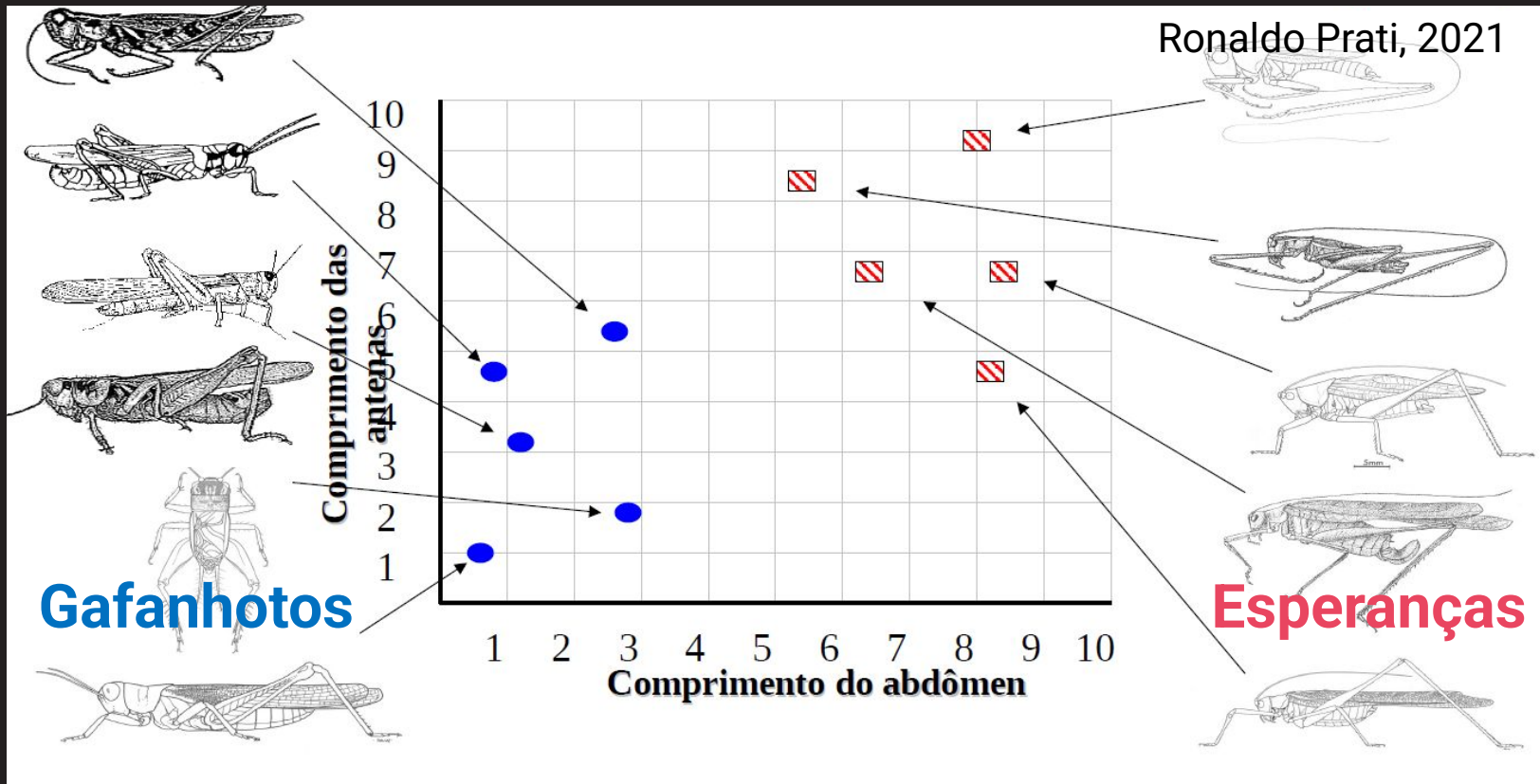
Atributos descritivos

Atributo alvo

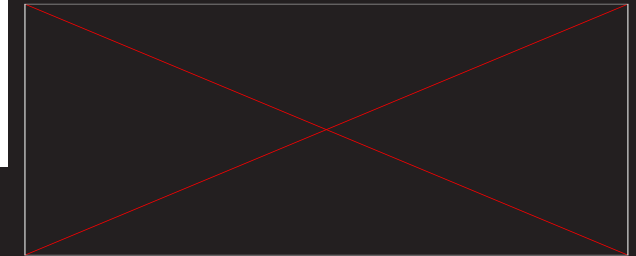
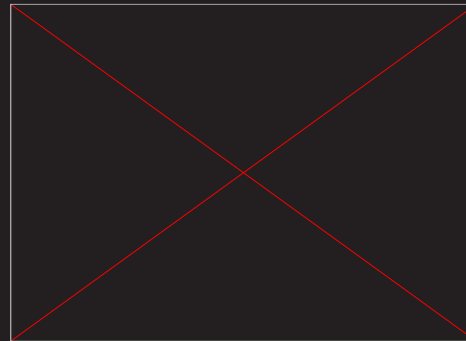
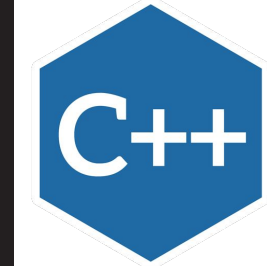
Índice da linha	x1	x2	...	xn	y
1	548.4	-9789	...	0.4875	-7595.28
2	689.4	-10235		-0.358	-7468.82
3	3154.8	-1031858		-0.1458	-101923 2
...	...			...	
k	803.54	-20000		1.054	-16791.4

Visualização de Dados

- Existem muitas formas de visualizar dados através de gráficos. Cada gráfico tem um objetivo!



Linguagens, Frameworks e Bibliotecas



Entre
outros...

Profissionais em IA

O mercado está aquecido! Conheça os principais perfis profissionais



Analista de Dados

Coleta, organiza e interpreta dados para gerar insights de negócio



Cientista de Dados

Modela problemas complexos usando estatística e machine learning



Engenheiro de IA

Desenvolve e implementa modelos de inteligência artificial em produção



Engenheiro de ML

Cria pipelines de treinamento e deploy de modelos de aprendizado de máquina



Engenheiro de Dados

Constrói e mantém a infraestrutura de dados (ETL, pipelines, data lakes)

Ferramentas por Perfil

Analista de Dados

Excel, SQL, Tableau, Power BI

Cientista de Dados

Python, R, Pandas, Scikit-Learn, TensorFlow

Engenheiro de IA

Python, PyTorch, TensorFlow, Docker, Kubernetes

Engenheiro de ML

MLflow, Kubeflow, Airflow, Python, AWS/GCP

Engenheiro de Dados

Spark, Hadoop, Kafka, SQL, Airflow, AWS/GCP

A person with glasses is sitting at a desk in a dimly lit room, working on a multi-monitor setup. The top monitor shows a scatter plot and a bar chart. The middle monitor displays a code editor with Python code. The bottom monitor shows a Jupyter Notebook with code and a scatter plot. The person is looking at the middle monitor. The text "Ambiente de Trabalho" is overlaid on the image.

Ambiente de Trabalho

Python, Jupyter Notebook, Pandas, NumPy, ScikitLearn, Matplotlib, Seaborn

Setup do Ambiente

Instalando Python

1. Baixe em python.org/downloads
2. Escolha 32 ou 64 bits corretamente
3. Marque 'Adicionar ao PATH'
4. Verifique: abra o cmd e digite python

O PATH permite que o terminal reconheça o comando python de qualquer diretório.

Pip (Gerenciador de Pacotes)

Semelhante ao apt (Linux) ou npm (Node). Permite instalar bibliotecas facilmente com `pip install <pacote>`. Já vem com Python 3.4+

Verificação Rápida

```
python --version  
pip --version  
pip install jupyter
```

Se estes comandos funcionam, seu ambiente está pronto!

Jupyter Notebook

O que é?

Ambiente de desenvolvimento interativo muito usado em Data Science e IA

A interface roda no navegador (browser), comunicando-se com o servidor via WebSockets

O Kernel implementa o interpretador da linguagem (IPython, IRkernel, IJulia)

Como instalar?

`pip install jupyter`

Para iniciar:
`python -m notebook`

Acesse: jupyter.org

Alternativa na nuvem: Google Colab (colab.research.google.com)

Bibliotecas Essenciais



Pandas

Manipulação e análise de dados tabulares. A ferramenta essencial para trabalhar com tabelas e DataFrames.



NumPy

Operações com vetores e matrizes. Base matemática para computação científica em Python.



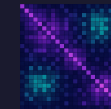
Scikit-Learn

Algoritmos de ML prontos para uso: classificação, regressão, clustering e pré-processamento.



Matplotlib

Criação de gráficos e visualizações. A biblioteca base para plots em Python.



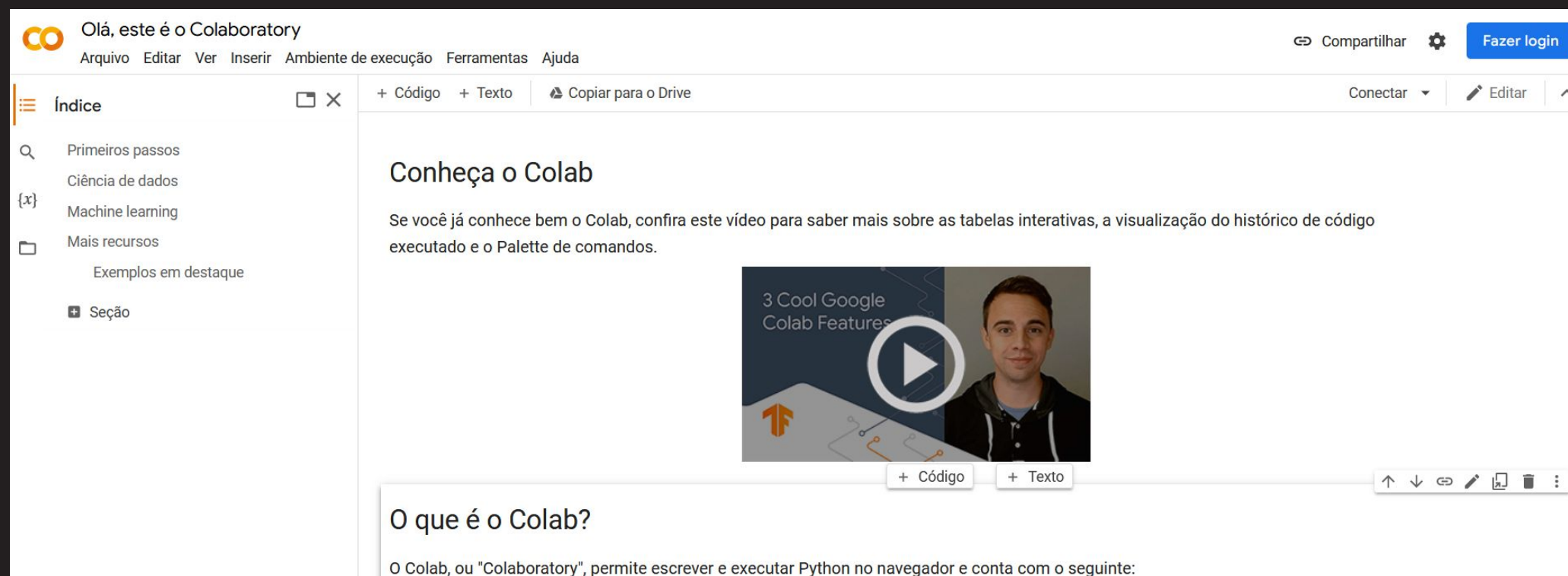
Seaborn

Gráficos estatísticos com estilos elegantes. Construído sobre Matplotlib.

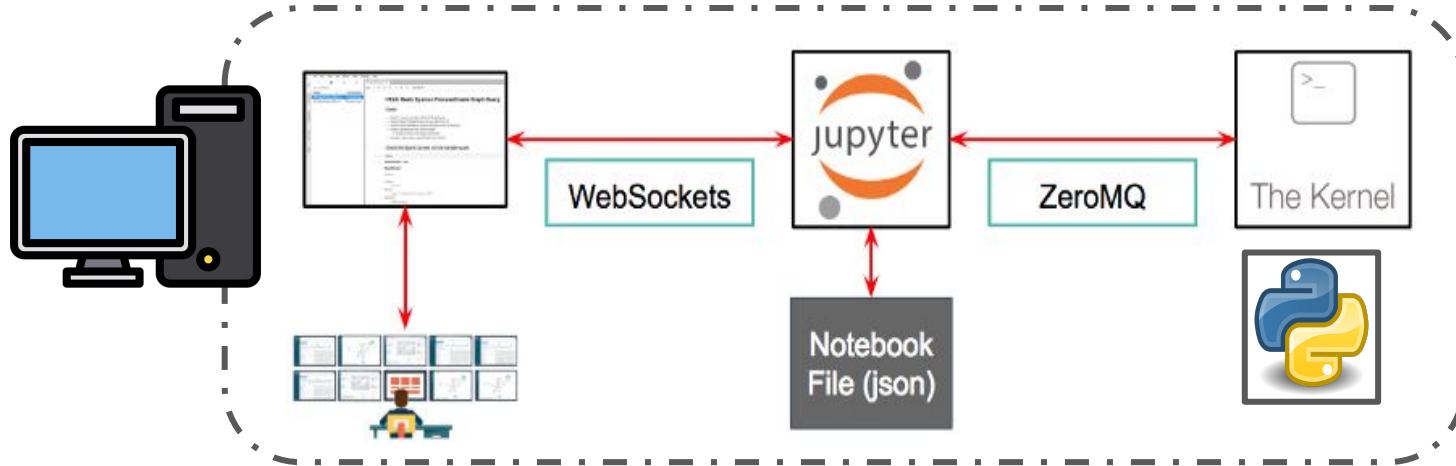
Google Colab



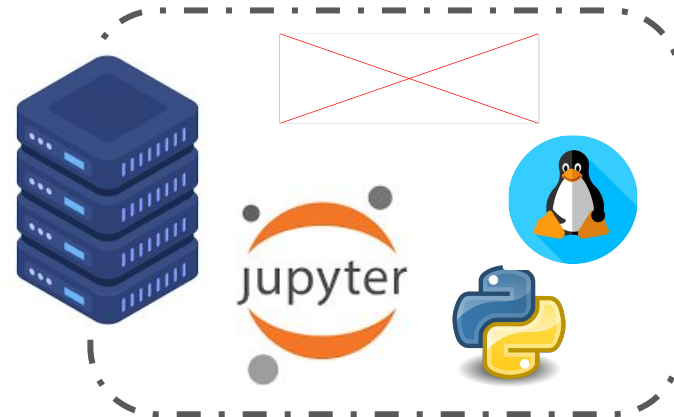
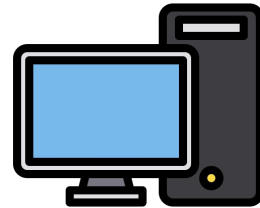
- Framework online: <https://colab.research.google.com/>
- Você vai precisar de um gmail para usar. Os scripts ficam salvos no Google Drive.

A screenshot of the Google Colab web interface. The top navigation bar includes the Colab logo, a welcome message "Olá, este é o Colaboratory", and links for "Arquivo", "Editar", "Ver", "Inserir", "Ambiente de execução", "Ferramentas", and "Ajuda". On the right, there are links for "Compartilhar", a settings gear, and a "Fazer login" button. The left sidebar shows a table of contents under "Índice" with items like "Primeiros passos", "Ciência de dados", "Machine learning", "Mais recursos", "Exemplos em destaque", and "Seção". The main content area is titled "Conheça o Colab" and contains a paragraph about the platform's features. Below the text is a video player with a play button and a thumbnail showing a man and the text "3 Cool Google Colab Features". At the bottom, there is a section titled "O que é o Colab?" with introductory text. The interface also includes a top toolbar with "+ Código", "+ Texto", and "Copiar para o Drive", and a bottom toolbar with various editing and navigation icons.

Nuvem vs Local



colab





Agora é com você!

Exercício

- 1) Vamos testar a instalação. Rode o Jupyter Notebook digitando `python -m notebook` no terminal de comando. Uma vez aberto, importe as bibliotecas para verificar se está tudo funcionando.
- 2) Abra o Google Colab e use o comando `!pip install` para instalar uma biblioteca.
- 3) Crie uma lista de 10 números em python em uma célula do Jupyter. Exiba a lista em forma decrescente em outra célula, sem usar sort e usando sort.
- 4) Crie uma célula de Markdown a escreva diferentes títulos e textos de apoio.
- 5) Defina uma função para computar a diferença entre duas listas de palavras em Python. Use o primeiro nome de todos os alunos da sua turma. Com a função feita, verifique quantas vezes cada nome se repete na turma.
- 6) Explore as documentações da bibliotecas instaladas.

Próximos Passos

O que veremos na próxima aula

Referências

Livros

- [1] Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow — Aurélien Géron, O'Reilly 2019
- [9] O Algoritmo Mestre — Pedro Domingos, Ed. Novatec 2015
- [10] G. Schleder et al. From DFT to Machine Learning: Recent Approaches to Materials Science — A Review (2019)

Artigos Acadêmicos

- [5] Reinforcement Learning as Supervised Learning on Optimized Data — BAIR Berkeley 2020
- [6] Kiran et al. Deep Reinforcement Learning for Autonomous Driving: A Survey — arXiv 2020

Tutoriais e Blogs

- [2] Introduction to Active Learning — Michelle Zhao
towardsdatascience.com/introduction-to-active-learning
- [3] Active Learning vs Weak Supervision — Appen
appen.com/blog/ml-techniques-active-learning-vs-weak-supervision
- [4] Supervised vs Unsupervised Learning — Isha Salian, NVIDIA 2018
blogs.nvidia.com/blog/supervised-unsupervised-learning
- [7] Ciclo de Vida dos Dados: KDD Process — Karina Moura
medium.com/@kvmoura/kdd-process
- [8] Data Leakage — Towards Data Science
towardsdatascience.com/data-leakage

Links Acadêmicos

Os links das referências [5] e [6] apontam para artigos no arXiv e BAIR Berkeley, abordando aprendizado por reforço e direção autônoma.

Todos os demais links direcionam para blogs especializados em ciência de dados e machine learning.

Acesso verificado em fevereiro de 2026. Links podem estar desatualizados.

Ciência de Dados e Visualização

Transformando dados em conhecimento e insights visuais

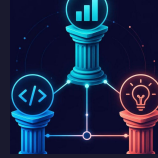
O que é Ciência de Dados?

Definição



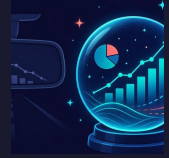
Área interdisciplinar que usa métodos científicos, algoritmos e sistemas para extrair conhecimento e insights de dados estruturados e não estruturados.

Pilares



Combina estatística, programação e conhecimento de domínio para resolver problemas complexos e apoiar decisões baseadas em evidências.

Diferença do BI



Enquanto Business Intelligence descreve o que aconteceu, Ciência de Dados prevê o que pode acontecer e prescreve ações.

Profissional



O cientista de dados coleta, limpa, analisa e modela dados. Usa Python, R, SQL e ferramentas de visualização no dia a dia.

Impacto Real



Netflix economiza US\$ 1 bilhão/ano com recomendações. Uber otimiza rotas em tempo real. Hospitais preveem surtos com dados.

O Pipeline da Ciência de Dados

Da coleta ao deploy: as 5 etapas fundamentais de todo projeto de Data Science



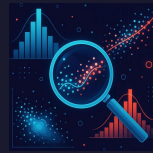
1. Coleta

Reunir dados de bancos, APIs, web scraping, sensores IoT e formulários



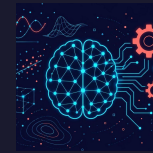
2. Limpeza

Tratar valores ausentes, duplicatas e formatos. 80% do tempo é gasto aqui



3. EDA

Investigar padrões, distribuições e correlações com estatísticas e gráficos



4. Modelagem

Aplicar algoritmos de ML ou estatística para criar modelos preditivos



5. Deploy

Validar com métricas, colocar em produção e monitorar continuamente

Pipeline: Coleta e Limpeza de Dados

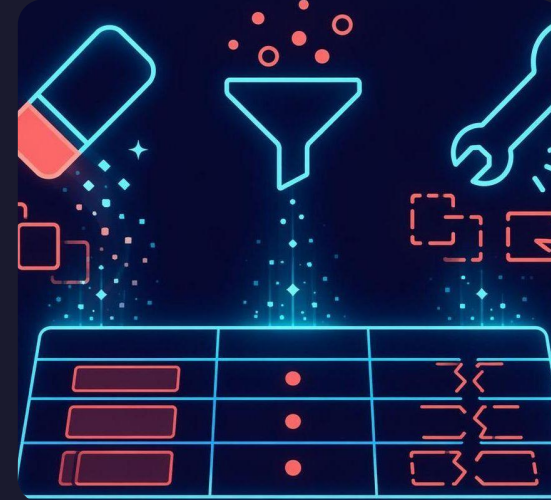


1. Coleta de Dados

Reunir dados de diversas fontes: bancos de dados relacionais (SQL), APIs REST, web scraping, sensores IoT e formulários online.

Fontes comuns:

- Bancos de dados (MySQL, PostgreSQL, MongoDB)
- APIs públicas (Twitter, Google, governo)
- Arquivos CSV, JSON e Excel
- Web scraping (Beautiful Soup, Selenium)



2. Limpeza e Preparação

Tratar valores ausentes, remover duplicatas e corrigir formatos. Cerca de 80% do tempo de um projeto é gasto aqui.

Técnicas essenciais:

- Tratar NaN: preencher com média/mediana ou remover
- Remover duplicatas: `df.drop_duplicates()`
- Normalizar e padronizar colunas numéricas
- Encoding de variáveis categóricas

Pipeline: Análise e Modelagem

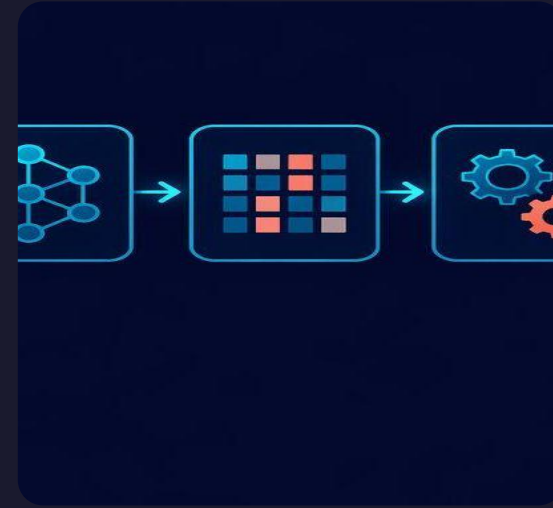


3. Análise Exploratória (EDA)

Investigar padrões, distribuições e correlações antes de modelar. O objetivo é entender os dados profundamente.

Técnicas-chave:

- `df.describe()` para estatísticas descritivas
- Histogramas e boxplots para distribuições
- Heatmap de correlação (`sns.heatmap`)
- Scatter plots para relações entre variáveis



4. Modelagem

Aplicar algoritmos de Machine Learning para criar modelos preditivos ou descritivos a partir dos dados preparados.

Etapas da modelagem:

- Dividir dados: `train_test_split` (treino/teste)
- Escolher algoritmo: regressão, árvore, SVM, rede neural
- Treinar: `model.fit(X_train, y_train)`
- Ajustar hiperparâmetros com `GridSearch/RandomSearch`

Pipeline: Avaliação e Deploy

5. Avaliação e Deploy

Validar o modelo com métricas objetivas e colocá-lo em produção para gerar valor real.

Métricas de Avaliação:

- Classificação: acurácia, precisão, recall, F1-score
- Regressão: MAE, RMSE, R^2
- Validação cruzada (cross-validation) para robustez

Deploy em Produção:

- Salvar modelo: pickle, joblib ou ONNX
- API REST com Flask/FastAPI para servir previsões
- Monitorar drift e retreinar periodicamente



Métricas de Avaliação de Modelos

Como medir se um modelo de Machine Learning realmente funciona

Por que Avaliar?

Um modelo que não é avaliado corretamente pode parecer bom no treino e falhar em produção.

Dois grandes grupos de métricas:

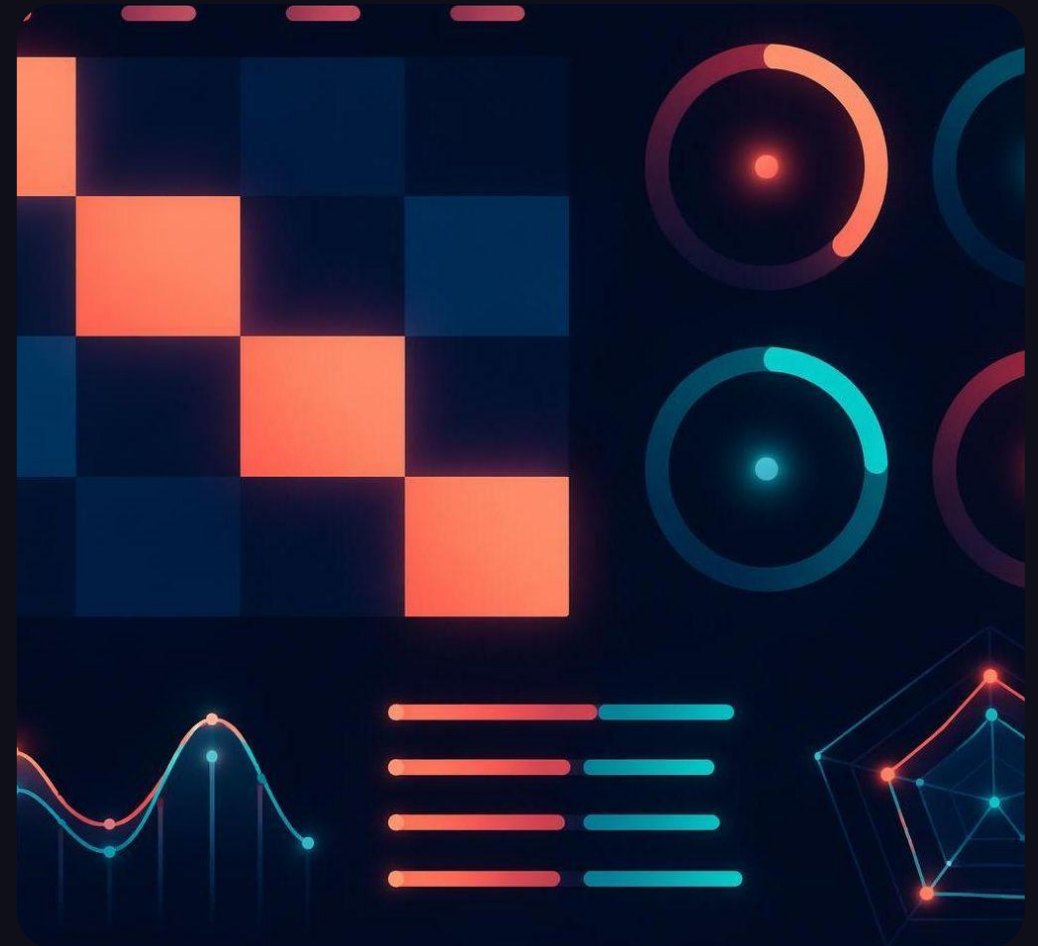
- Classificação: mede acertos em categorias (spam/não-spam, doente/saudável)
- Regressão: mede proximidade de valores numéricos (preço, temperatura)

Conceitos fundamentais:

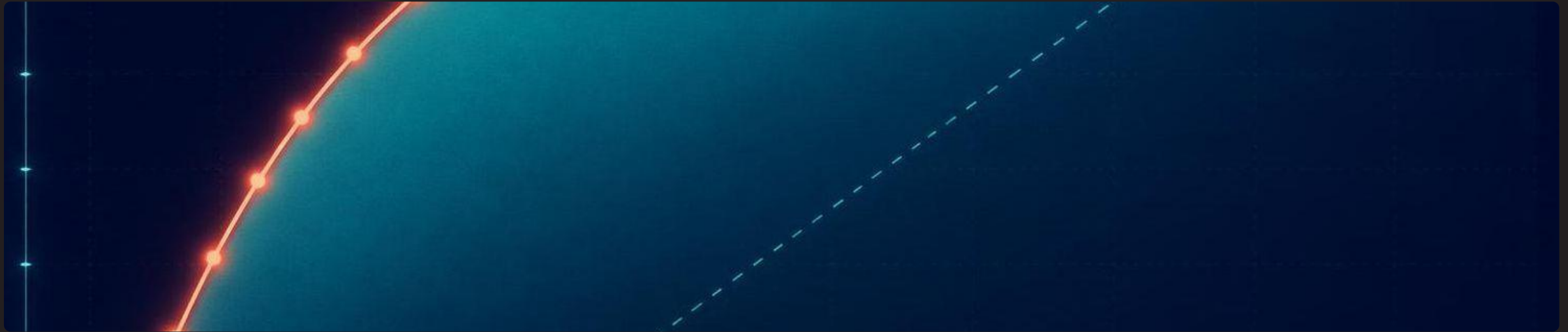
- Overfitting: modelo decora os dados de treino
- Underfitting: modelo simplifica demais
- Validação cruzada: técnica para testar generalização
- Trade-off viés-variância: equilíbrio essencial

Regra de Ouro

Nunca avalie um modelo nos mesmos dados usados para treiná-lo. Separe sempre treino, validação e teste.



Métricas de Classificação



Métricas Básicas

Acurácia: % de acertos totais. Cuidado com dados desbalanceados!

Precisão: dos que o modelo disse SIM, quantos realmente são SIM?
→ Importante quando falso positivo é caro (ex: spam)

Recall: dos que realmente são SIM, quantos o modelo encontrou?
→ Importante quando falso negativo é grave (ex: câncer)

F1-Score: média harmônica entre Precisão e Recall

Métricas Avançadas

Matriz de Confusão: tabela que mostra TP, TN, FP, FN
→ Visualização completa dos erros do modelo

Curva ROC: gráfico que mostra trade-off entre sensibilidade e especificidade

AUC (Área sob a Curva): quanto mais perto de 1, melhor
→ 0.5 = aleatório, 0.7-0.8 = bom, >0.9 = excelente

Em Python: `sklearn.metrics` (`accuracy_score`, `classification_report`, `roc_auc_score`)

Métricas de Regressão e Validação

Métricas de Regressão

MAE (Erro Médio Absoluto): média dos erros em valor absoluto

→ Fácil de interpretar, mesma unidade dos dados

MSE (Erro Quadrático Médio): penaliza erros grandes

→ Mais sensível a outliers que MAE

RMSE (Raiz do MSE): volta à unidade original

→ Métrica mais popular em competições

R^2 (Coeficiente de Determinação): % da variância explicada

→ 1.0 = perfeito, 0 = modelo não explica nada

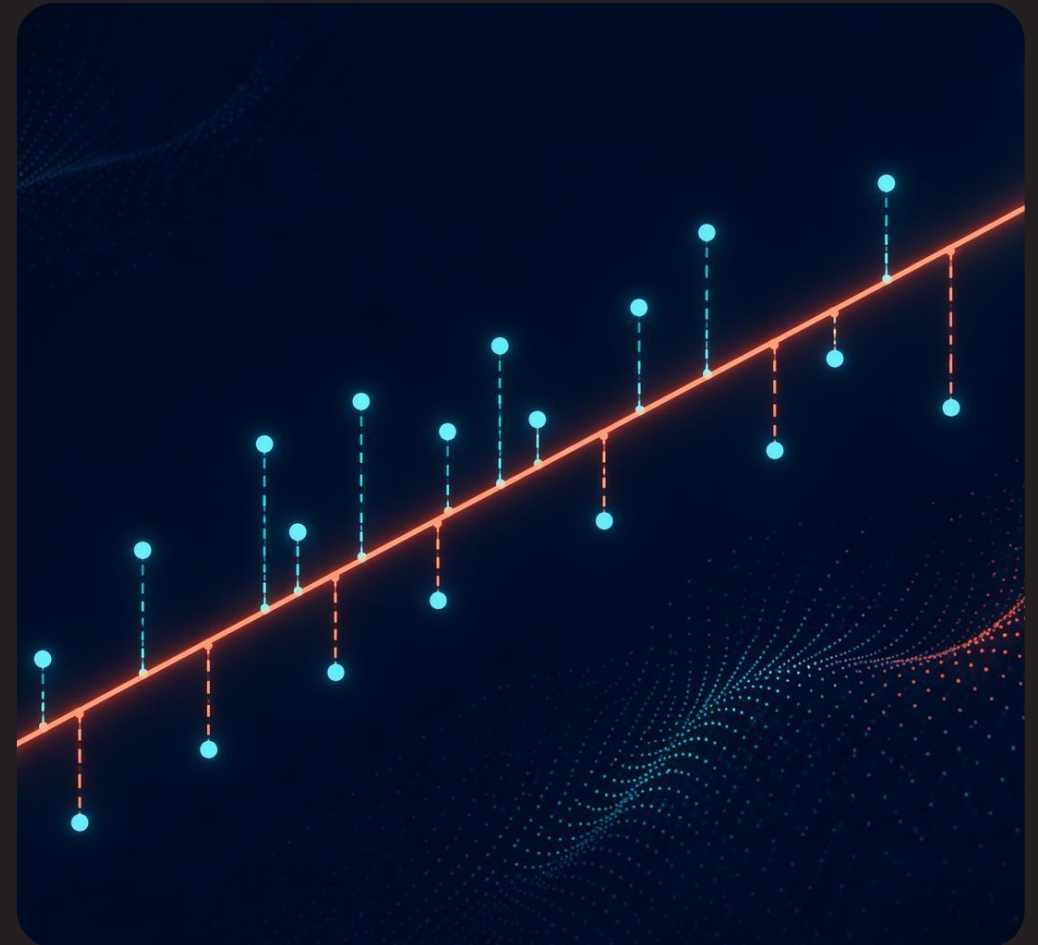
Em Python: `mean_absolute_error`, `mean_squared_error`, `r2_score`

Validação Cruzada (K-Fold)

Divide os dados em K partes, treina em K-1 e testa em 1. Repete K vezes.

Resultado: média das K avaliações — mais confiável que uma única divisão.

Em Python: `cross_val_score(model, X, y, cv=5)`



Análise Exploratória de Dados (EDA)

Entenda seus dados antes de modelar: as 4 dimensões fundamentais da EDA



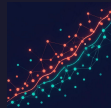
O que é EDA?

Processo de investigar dados antes de modelar. Entender estrutura, detectar anomalias e formular hipóteses.



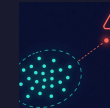
Estatísticas Descritivas

Média, mediana, moda, desvio padrão e quartis resumem distribuições de forma rápida e objetiva.



Correlações e Outliers

Identificar relações entre variáveis e valores atípicos que indicam erros ou fenômenos reais.



Ferramentas

Pandas (describe, info, corr), Matplotlib e Seaborn são essenciais para EDA em Python.

EDA: Definição e Estatísticas



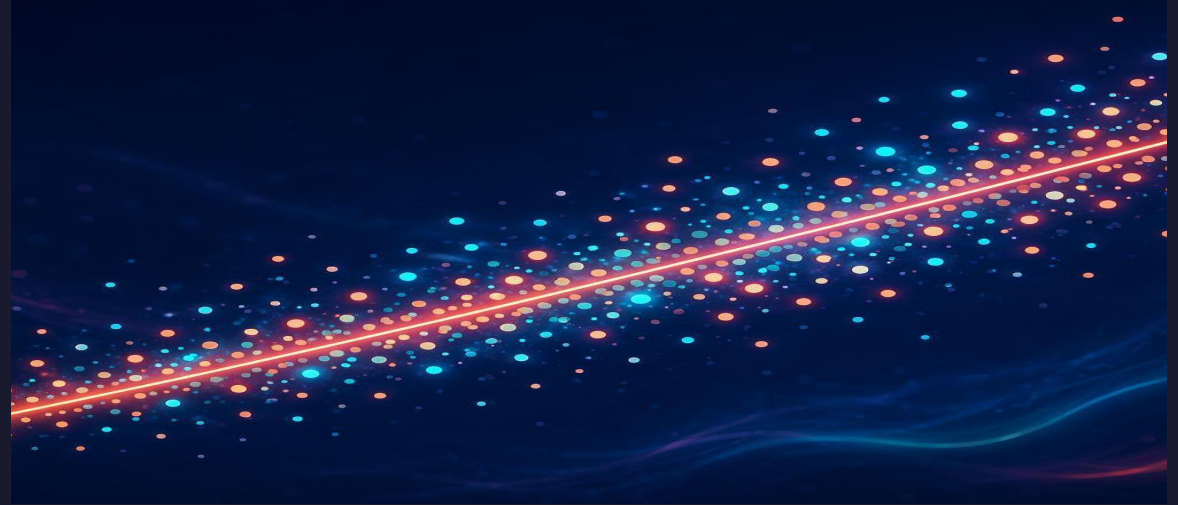
O que é EDA?

Explorar dados antes de aplicar qualquer modelo.

Objetivos principais:

- Entender a estrutura dos dados
- Detectar anomalias e valores atípicos
- Formular hipóteses iniciais
- Identificar variáveis relevantes

Exemplo: temperatura vs vendas de sorvete



Estatísticas Descritivas

Resumir distribuições de forma rápida e objetiva.

Medidas centrais:

- Média, mediana e moda
- Desvio padrão e variância

Medidas de posição:

- Quartis (Q1, Q2, Q3)
- Percentis e amplitude interquartil (IQR)

Em Python: `df.describe()`, `df.info()`

EDA: Correlações e Outliers



Correlações

Identificar relações entre variáveis numéricas.

Tipos de correlação:

- Positiva: temperatura $\uparrow \rightarrow$ vendas de sorvete $\uparrow$
- Negativa: preço $\uparrow \rightarrow$ demanda $\downarrow$
- Nula: sem relação aparente

Ferramentas:

- `df.corr()` para matriz de correlação
- `sns.heatmap()` para visualização
- Coeficiente de Pearson (-1 a +1)



Outliers (Valores Atípicos)

Valores que se desviam muito do padrão.

Como detectar:

- Boxplot (além dos whiskers)
- Z-score ($|z| > 3$)
- IQR: abaixo de $Q1 - 1.5 \times IQR$ ou acima de $Q3 + 1.5 \times IQR$

Cuidado:

- Nem todo outlier é erro
- Sempre investigue antes de remover
- Pode indicar fraude, bug ou fenômeno real

Por que Visualizar Dados?



Compreensão Rápida

O cérebro humano processa imagens 60.000x mais rápido que texto. Um bom gráfico comunica em segundos o que uma tabela leva minutos.



Descoberta de Padrões

Visualizações revelam tendências, ciclos e agrupamentos que ficam invisíveis em planilhas. O Quarteto de Anscombe prova isso.



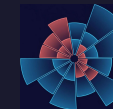
Comunicação Eficaz

Gráficos bem feitos contam histórias. Dashboards executivos transformam dados complexos em decisões acionáveis.



Detecção de Erros

Gráficos de dispersão e boxplots ajudam a encontrar outliers, dados faltantes e inconsistências rapidamente.



Exemplo Clássico

Florence Nightingale usou gráficos em 1858 para provar que higiene salvava vidas. Visualização muda políticas.

Tipos de Gráficos e Quando Usar



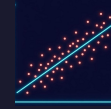
Barras e Colunas

Comparar categorias ou valores discretos. Ideal para ranking, comparações entre grupos e distribuições simples.



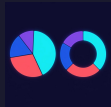
Linhas

Mostrar evolução ao longo do tempo. Perfeito para séries temporais como vendas mensais, temperatura ou preço de ações.



Dispersão (Scatter)

Explorar relações entre duas variáveis numéricas. Revela correlações, clusters e outliers de forma visual.



Pizza e Rosca

Mostrar proporções de um todo. Use com no máximo 5 categorias. Alternativa melhor: gráfico de barras horizontal.



Histograma e Boxplot

Entender distribuições. Histograma mostra frequências; boxplot revela mediana, quartis e outliers de uma só vez.

Caso Real: Spotify Wrapped

Como dados pessoais de escuta se transformam em uma experiência visual viral



O que é o Wrapped?

Campanha anual que transforma dados de escuta em cards visuais personalizados e compartilháveis.

Dados utilizados:

- Horas ouvidas e artistas favoritos
- Gêneros mais frequentes
- Horários de pico e faixas repetidas
- Evolução do gosto musical ao longo do ano

Design pensado para redes sociais: cards coloridos com tipografia grande e animações.

Impacto e Lição

60M+ compartilhamentos no Instagram Stories (2023). Viralização orgânica impulsionada por dados.

Lição: dados pessoais + visualização criativa = engajamento massivo.

Caso Real: Painei COVID-19

Como a Johns Hopkins rastreou a pandemia global com dados em tempo real

O Dashboard

Em janeiro de 2020, a Johns Hopkins criou um dashboard em tempo real para rastrear a pandemia.

Fontes de dados:

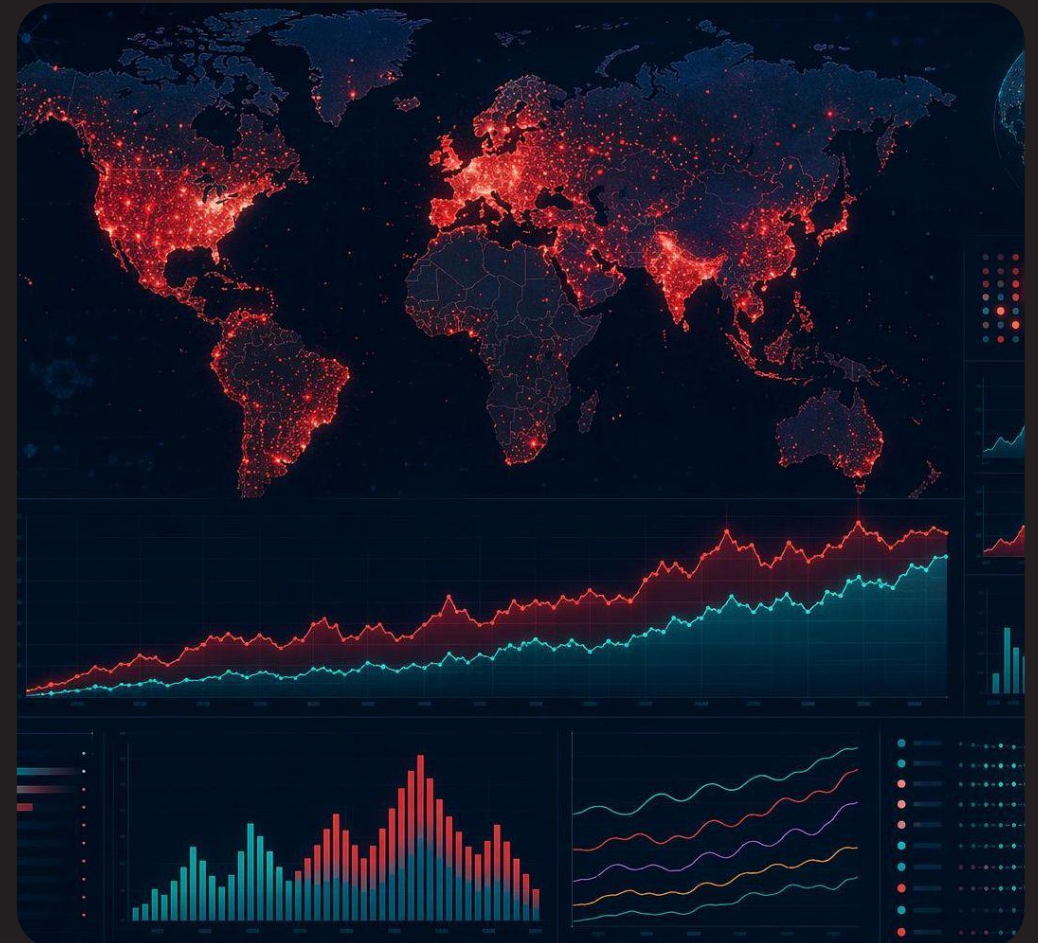
- OMS e ministérios da saúde de cada país
- Atualizações diárias com milhões de registros
- Dados por país, região e cidade

Tipos de visualização:

- Mapas de calor globais
- Gráficos de linha acumulativa
- Barras comparativas entre países
- Tabelas interativas com filtros

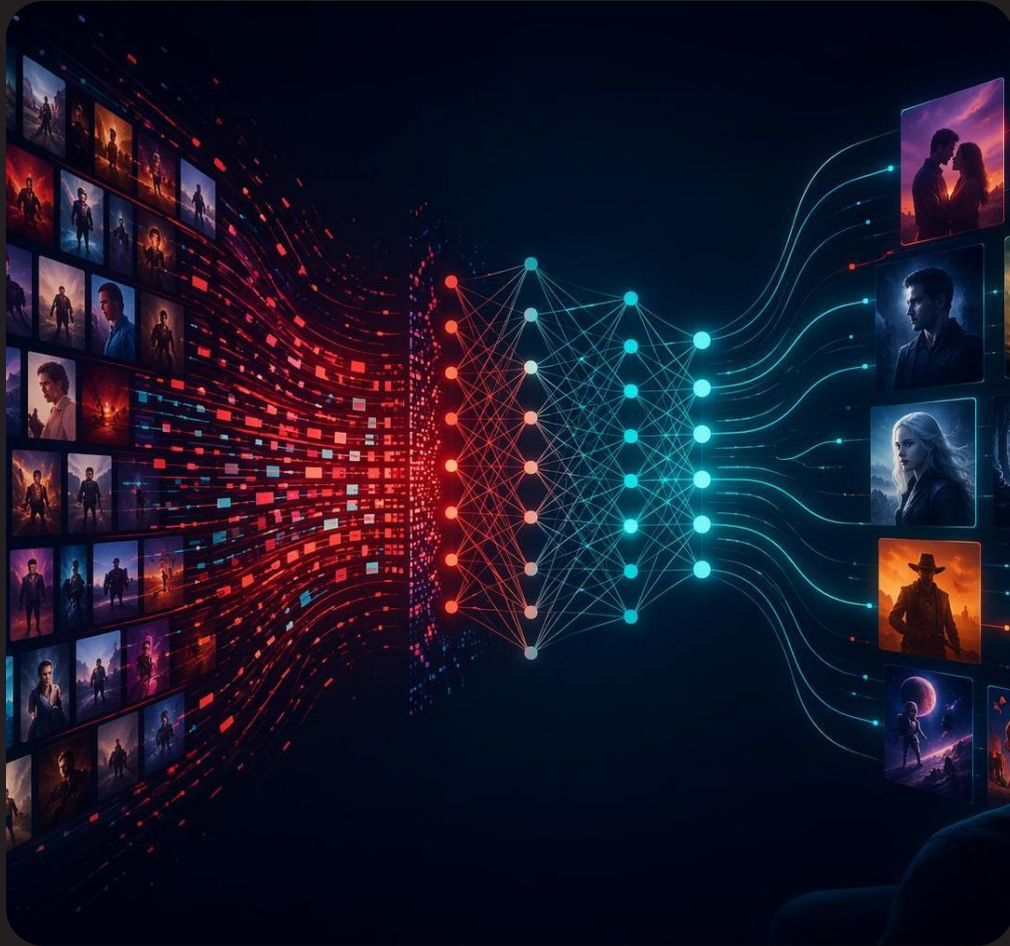
Impacto e Lição

4 bilhões+ de acessos. Referência mundial para governos e mídia.
Lição: visualização de dados em crise salva vidas. Transparência gera confiança.



Caso Real: Netflix

Como a Netflix economiza US\$ 1 bilhão/ano com ciência de dados



Sistema de Recomendação

O algoritmo de recomendação analisa o comportamento de 230 milhões de assinantes para personalizar a experiência.

Dados coletados:

- Histórico de visualização e avaliações
- Hora do dia, dispositivo e duração
- Padrões de pause, rewind e abandono
- Pesquisas e navegação no catálogo

Técnicas de Data Science:

- Filtragem colaborativa e baseada em conteúdo
- Deep Learning para thumbnails personalizadas
- Testes A/B com milhares de variações simultâneas

Impacto

80% do conteúdo assistido vem de recomendações. Economia de US\$ 1 bilhão/ano em retenção de assinantes. Cada detalhe visual é otimizado por dados.

Caso Real: Uber

Otimização de rotas e preço dinâmico com dados em tempo real

Dados em Tempo Real

O Uber processa bilhões de eventos por dia para otimizar a experiência de motoristas e passageiros.

Ciência de dados aplicada:

- ETA (tempo estimado de chegada) com ML
- Surge pricing baseado em oferta e demanda
- Otimização de rotas com grafos e GPS
- Detecção de fraude em tempo real

Fontes de dados:

- GPS de milhões de dispositivos simultâneos
- Dados de trânsito, clima e eventos
- Histórico de corridas e avaliações
- Dados de pagamento e comportamento

Impacto

Redução de 20% no tempo de espera. Preços ajustados em tempo real para equilibrar mercado. Presente em 10.000+ cidades no mundo.



Caso Real: iFood

Data Science no maior app de delivery da América Latina



Dados que Alimentam o App

O iFood usa ciência de dados em toda a cadeia: do pedido à entrega, passando por marketing e logística.

Aplicações de Data Science:

- Recomendação personalizada de restaurantes
- Previsão de demanda por região e horário
- Otimização de rotas de entregadores
- Precificação dinâmica de frete

Dados coletados:

- Histórico de pedidos e avaliações
- Localização em tempo real (GPS)
- Tempo de preparo dos restaurantes
- Comportamento de navegação no app

Impacto

75 milhões de pedidos/mês. Redução de 15% no tempo de entrega com ML. Mais de 300 mil restaurantes parceiros otimizados por dados.

Métricas na Prática

Como Netflix, Uber e iFood medem se seus modelos funcionam



KPIs de Data Science

As métricas técnicas (F1, RMSE) precisam se conectar a métricas de negócio.

Netflix: taxa de retenção de assinantes

→ Se a recomendação melhora, menos gente cancela

Uber: tempo médio de espera do passageiro

→ Otimização de rotas reduz ETA em 20%

iFood: tempo de entrega e satisfação do cliente

→ ML prevê demanda e aloca entregadores

Spotify: minutos de escuta por sessão

→ Recomendações mantêm o usuário engajado

Conexão Essencial

A métrica técnica só importa se mover uma métrica de negócio. Sempre pergunte: qual decisão este número vai influenciar?

Testes A/B e Experimentação

O que é Teste A/B?

Comparar duas versões (A e B) para medir qual performa melhor em uma métrica específica.

Como funciona:

1. Definir hipótese e métrica de sucesso
2. Dividir usuários aleatoriamente em dois grupos
3. Grupo A (controle) vê versão atual
4. Grupo B (variante) vê nova versão
5. Medir diferença com significância estatística

Exemplos reais:

- Netflix testa milhares de thumbnails simultaneamente
- Google testou 41 tons de azul para links
- iFood testa layouts de recomendação de restaurantes

Cuidados Importantes

- Tamanho da amostra: poucos usuários = resultados não confiáveis
- Duração: testes curtos demais capturam ruído, não sinal
- $p\text{-valor} < 0.05$: padrão para considerar resultado significativo



Boas Práticas em Visualização

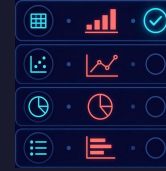
Simplicidade

Menos é mais. Remova elementos desnecessários (chartjunk). Cada pixel deve transmitir informação, não decoração.



Escolha Certa

Combine o tipo de gráfico com a pergunta. Comparação? Barras. Tendência? Linhas. Proporção? Pizza (com cautela).



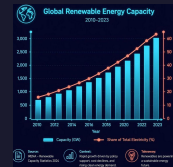
Cores com Propósito

Use cores para destacar, não para decorar. Daltonismo afeta 8% dos homens — use paletas acessíveis como ColorBrewer.



Contexto Sempre

Inclua títulos claros, rótulos nos eixos, unidades de medida e fonte dos dados. Sem contexto, o gráfico é ambíguo.



Não Distorça

Eixo Y deve começar em zero para barras. Não use 3D sem necessidade. Evite truncar eixos para exagerar diferenças.



Erros Comuns em Visualização

Evite estes 5 erros que comprometem a comunicação dos seus dados



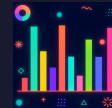
1. Gráfico Errado

Usar pizza com 15 categorias ou linhas para dados categóricos



2. Excesso de Info

Gráficos com muitas séries e eixos se tornam ilegíveis



3. Cores Sem Lógica

Arco-íris aleatório em vez de paleta sequencial ou divergente



4. Eixo Manipulado

Truncar o eixo Y para exagerar diferenças pequenas



5. Sem Fonte

Gráfico sem indicar origem dos dados perde credibilidade

Erros: Gráfico Errado e Excesso de Info

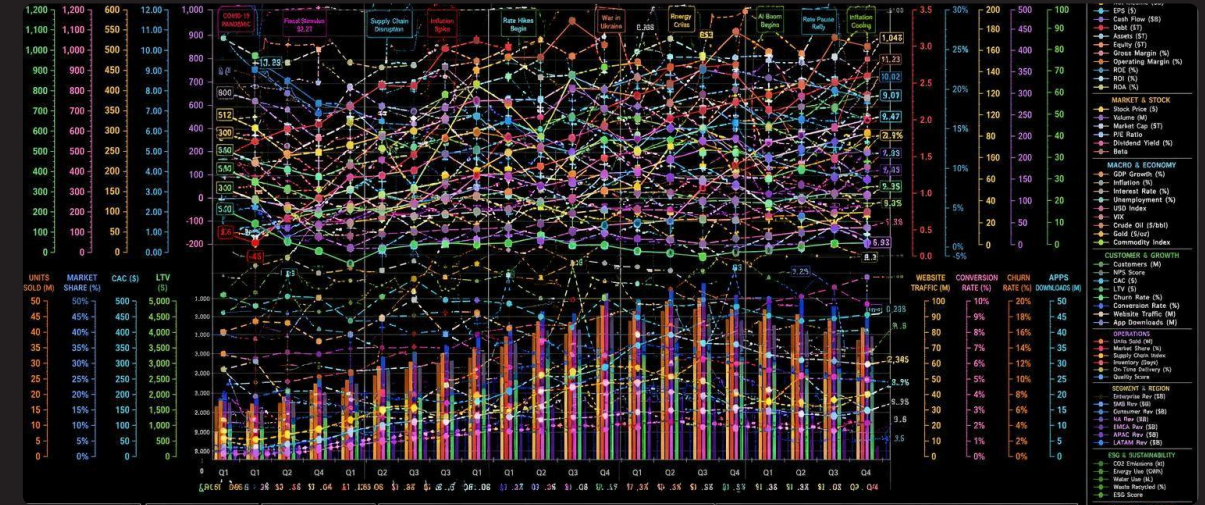
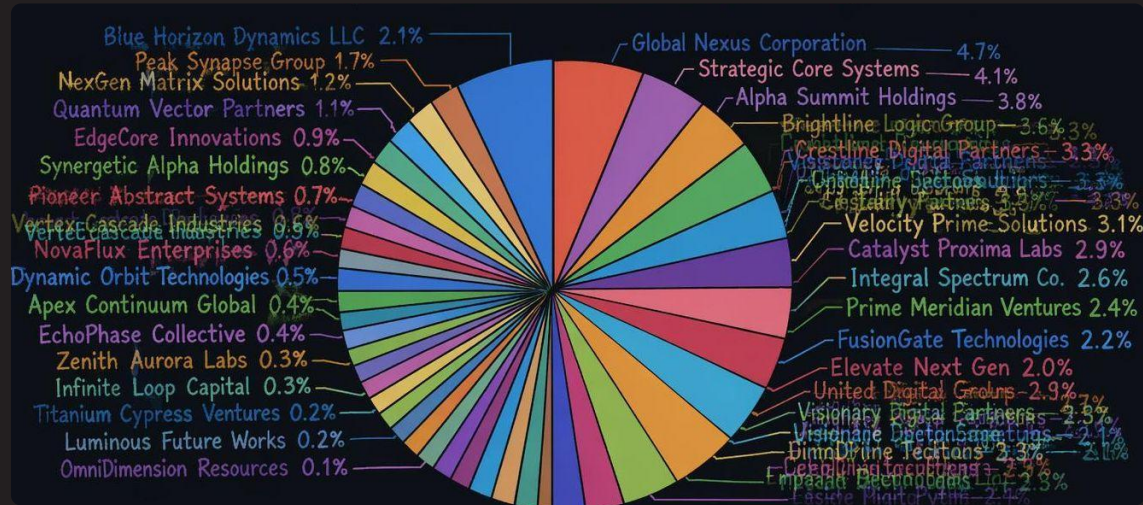


Gráfico Errado

Usar o tipo de gráfico inadequado distorce a mensagem.

Erros frequentes:

- Pizza com mais de 5-6 categorias
- Linhas para dados sem sequência temporal
- Barras 3D que distorcem proporções

Dica: sempre pergunte qual é a pergunta que quer responder.

Excesso de Informação

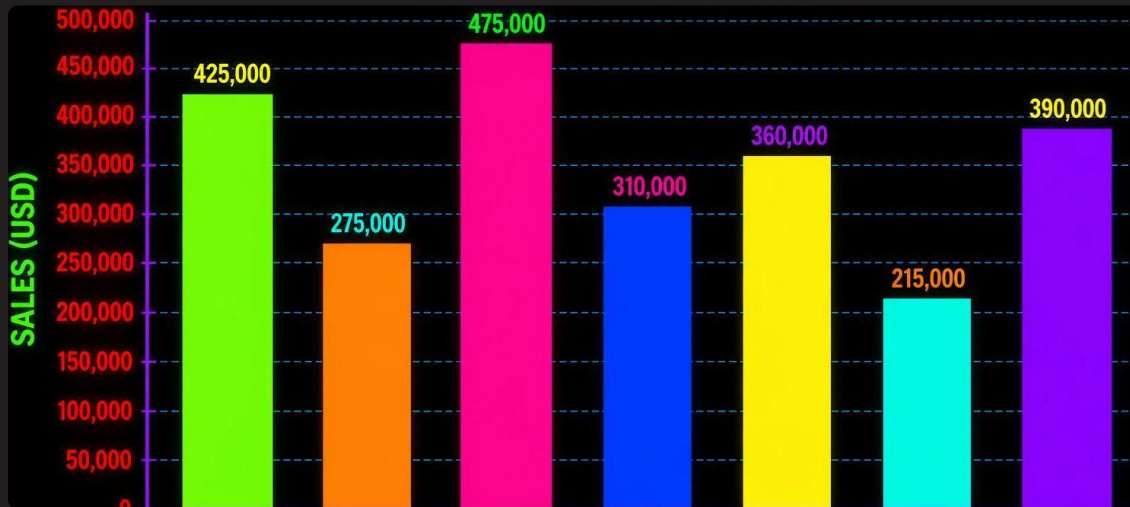
Gráficos sobrecarregados são ilegíveis.

Sinais de excesso:

- Mais de 5-7 séries no mesmo gráfico
- Legendas competindo com os dados
- Múltiplos eixos Y cruzando informações

Solução: divida em gráficos menores e focados. Menos é mais.

Erros: Cores e Eixos Manipulados



Cores Sem Lógica

Cores aleatórias confundem em vez de informar. Cada cor deve ter significado.

Erros comuns:

- Arco-íris aleatório sem hierarquia
- Vermelho e verde juntos (daltônicos!)
- Muitas cores saturadas competindo
- Mesma cor para categorias diferentes

Use paletas sequenciais para ordenação e divergentes para comparar extremos.



Eixo Manipulado

Truncar o eixo Y faz diferenças de 2% parecerem enormes. Isso é desonestidade visual.

Técnicas enganosas:

- Cortar o eixo Y para não começar em zero
- Alterar escala para exagerar tendências
- Usar eixos duplos com escalas incompatíveis
- Inverter eixos sem indicação clara

Regra: sempre comece o eixo Y em zero, salvo justificativa técnica clara.

Erros: Credibilidade e Resumo

Sem Fonte de Dados

Um gráfico sem indicar de onde vêm os números perde toda a credibilidade.

Por que a fonte importa:

- Permite verificação independente
- Dá contexto temporal e geográfico
- Aumenta a confiança do público
- É requisito ético e acadêmico

Boas práticas:

- Cite a organização e o ano dos dados
- Inclua link quando possível
- Diferencie dados primários de secundários
- Mencione limitações conhecidas da base

Checklist Anti-Erros

- ✓ O gráfico responde à pergunta certa?
- ✓ Está simples e legível?
- ✓ As cores têm significado?
- ✓ Os eixos são honestos?
- ✓ A fonte dos dados está citada?

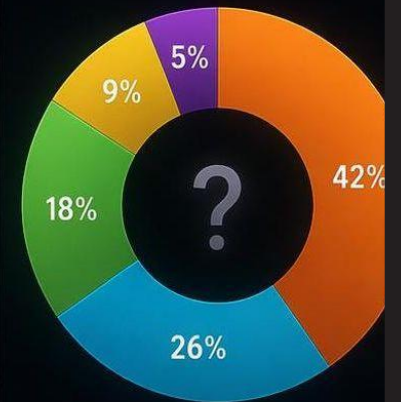
WORLD IS CHANGING FAST

GLOBAL TRENDS

POPULATION (IN BILLIONS)



ENERGY SOURCES IN THE WORLD



GLOBAL TEMPERATURE
INCREASE



68%

URBAN POPULATION
BY 2025



7%
GLOBAL GROWTH

THESE NUMBERS SHAPE OUR FUTURE. BUT CAN WE TRUST THEM?

Métricas de Qualidade Visual

Ferramentas quantitativas para avaliar a eficácia de uma visualização



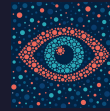
Data-Ink Ratio

Proporção de tinta usada para dados vs. decoração. Quanto mais alto, mais eficiente o gráfico.



Lie Factor

Mede distorção visual. Se a representação gráfica exagera ou minimiza a realidade dos dados.



Acessibilidade

Garantir que a visualização seja compreensível por todos, incluindo pessoas com daltonismo.



Checklist de Avaliação

Framework sistemático para revisar e pontuar a qualidade de qualquer visualização.

Data-Ink Ratio e Chart Junk

O Princípio de Tufte

Edward Tufte, estatístico de Yale, definiu que a excelência gráfica é a comunicação de ideias complexas com clareza e eficiência.

Data-Ink Ratio = tinta dos dados / tinta total

→ Quanto mais próximo de 1, melhor

→ Elimine tudo que não comunica informação

Chart Junk: elementos decorativos sem valor informativo

- Gradientes 3D desnecessários
- Bordas e sombras excessivas
- Imagens de fundo decorativas
- Grid lines pesadas e redundantes
- Legendas que repetem rótulos dos eixos

Na Prática: Antes e Depois

Antes: gráfico 3D com gradiente, grid pesado e legenda redundante

Depois: barras simples 2D com rótulos diretos e fundo limpo

Regra: se remover um elemento e a mensagem não muda, remova-o.



Lie Factor e Distorção Visual



O que é Lie Factor?

Criado por Edward Tufte, mede o quanto uma representação gráfica distorce a realidade dos dados.

Fórmula:

Lie Factor = tamanho do efeito no gráfico / tamanho do efeito nos dados

Interpretação:

- LF = 1.0: representação honesta
- LF > 1.0: gráfico exagera a diferença
- LF < 1.0: gráfico minimiza a diferença
- LF > 1.05 ou < 0.95: distorção significativa

Técnicas de distorção comuns:

- Eixo Y truncado (não começa em zero)
- Escala não-linear sem indicação
- Áreas/volumes usados para dados lineares
- Perspectiva 3D que distorce proporções

Exemplo Clássico

Dado real: aumento de 5% nas vendas (100 → 105)

Gráfico com eixo Y de 98 a 106: barra dobra de tamanho visual

Lie Factor = 100% (visual) / 5% (real) = 20 — distorção grave!

Acessibilidade e Checklist de Avaliação

Visualização Acessível

8% dos homens e 0.5% das mulheres têm algum grau de daltonismo. Sua visualização precisa funcionar para todos.

Princípios de acessibilidade:

- Não dependa apenas de cor para diferenciar dados
- Use formas, padrões e rótulos como redundância
- Evite vermelho-verde juntos (daltonismo mais comum)
- Garanta contraste mínimo de 4.5:1 para texto

Ferramentas úteis:

- Coblis: simulador de daltonismo online
- Paletas acessíveis: ColorBrewer, Viridis, Cividis
- Teste: peça para 3 pessoas descreverem seu gráfico
- Alt text: sempre adicione descrição textual

Checklist de Qualidade Visual

- ✓ Data-Ink Ratio alto? (sem chart junk)
- ✓ Lie Factor entre 0.95 e 1.05?
- ✓ Acessível para daltônicos?
- ✓ Fonte dos dados citada?
- ✓ Título claro e autoexplicativo?



Exercício Proposto

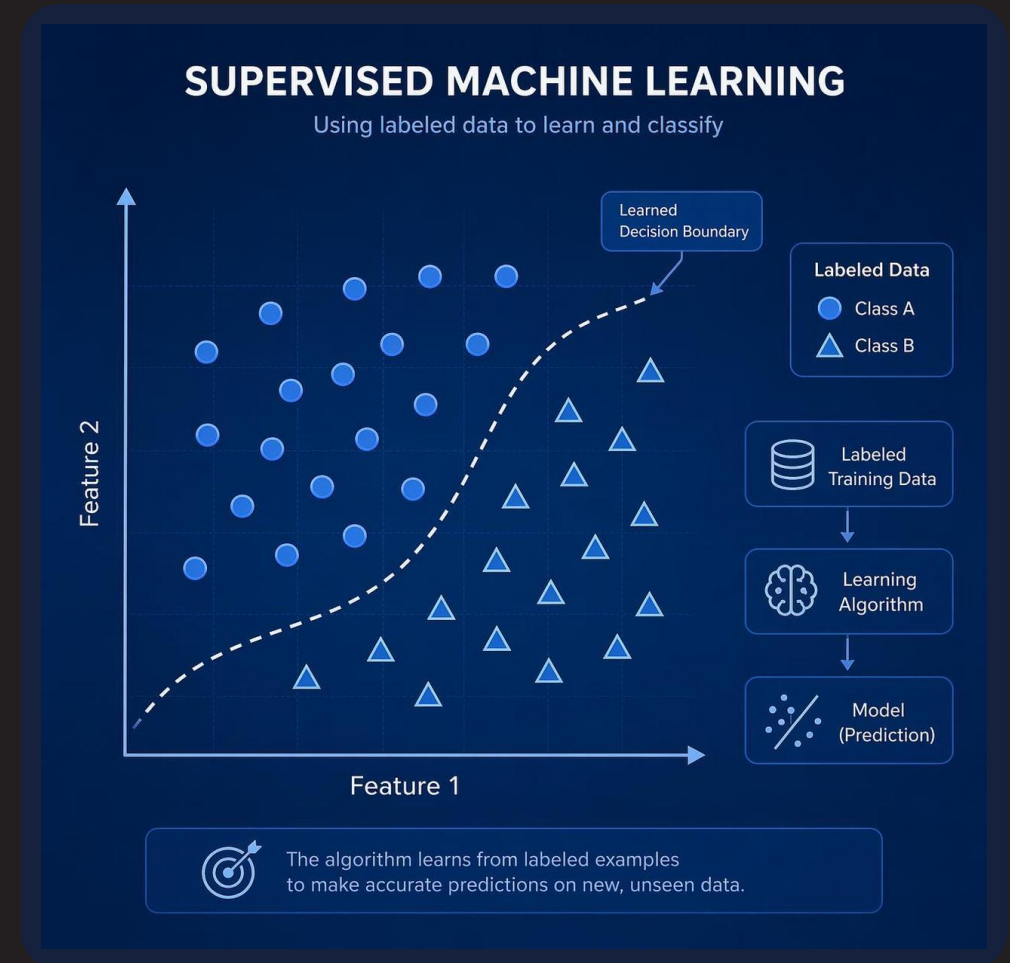
- 1) Abra o Google Colab e importe as bibliotecas: pandas, matplotlib e seaborn. Carregue o dataset Iris com `seaborn.load_dataset('iris')`.
- 2) Use `df.head()`, `df.describe()` e `df.info()` para fazer uma análise exploratória inicial. Anote suas observações sobre os dados.
- 3) Crie um histograma da coluna 'sepal_length' usando matplotlib. Adicione título, rótulos nos eixos e uma cor de sua escolha.
- 4) Crie um gráfico de dispersão (scatter plot) entre 'sepal_length' e 'petal_length', colorindo os pontos por espécie. Use seaborn.
- 5) Crie um boxplot comparando 'petal_width' entre as três espécies. Identifique visualmente se existem outliers.
- 6) Desafio: crie um painel com 4 gráficos usando `plt.subplots(2,2)`. Escolha os gráficos que melhor resumem o dataset e justifique suas escolhas.

Aprendizado de Máquina

Paradigmas e como as máquinas aprendem a partir de dados

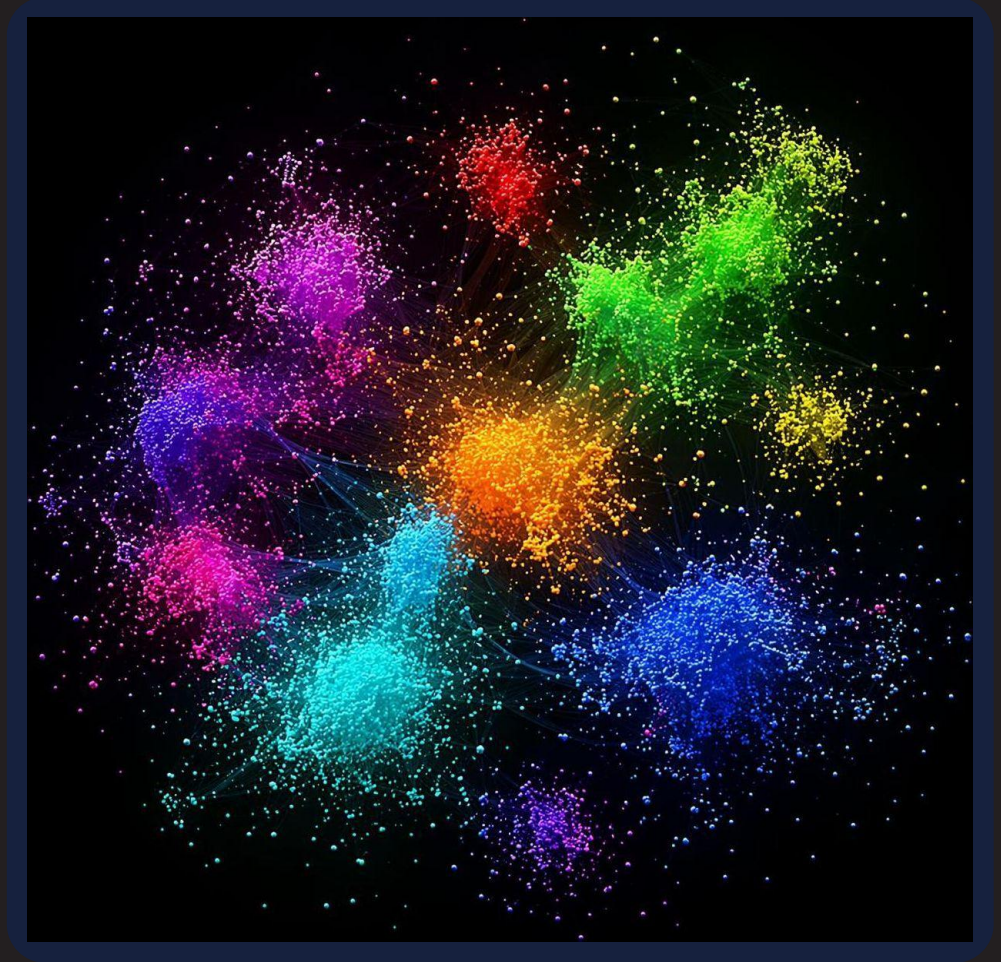
Aprendizado Supervisionado

- Como funciona o aprendizado supervisionado:
- **Definição:** o modelo aprende a partir de dados rotulados, onde cada exemplo tem entrada (X) e resposta esperada (Y).
- **Objetivo:** generalizar para prever a saída de novos exemplos não vistos.
- **Exemplos:** prever preço de imóveis, classificar e-mails como spam, reconhecer dígitos manuscritos.
- **Algoritmos típicos:** regressão linear, árvores de decisão, SVM e redes neurais.



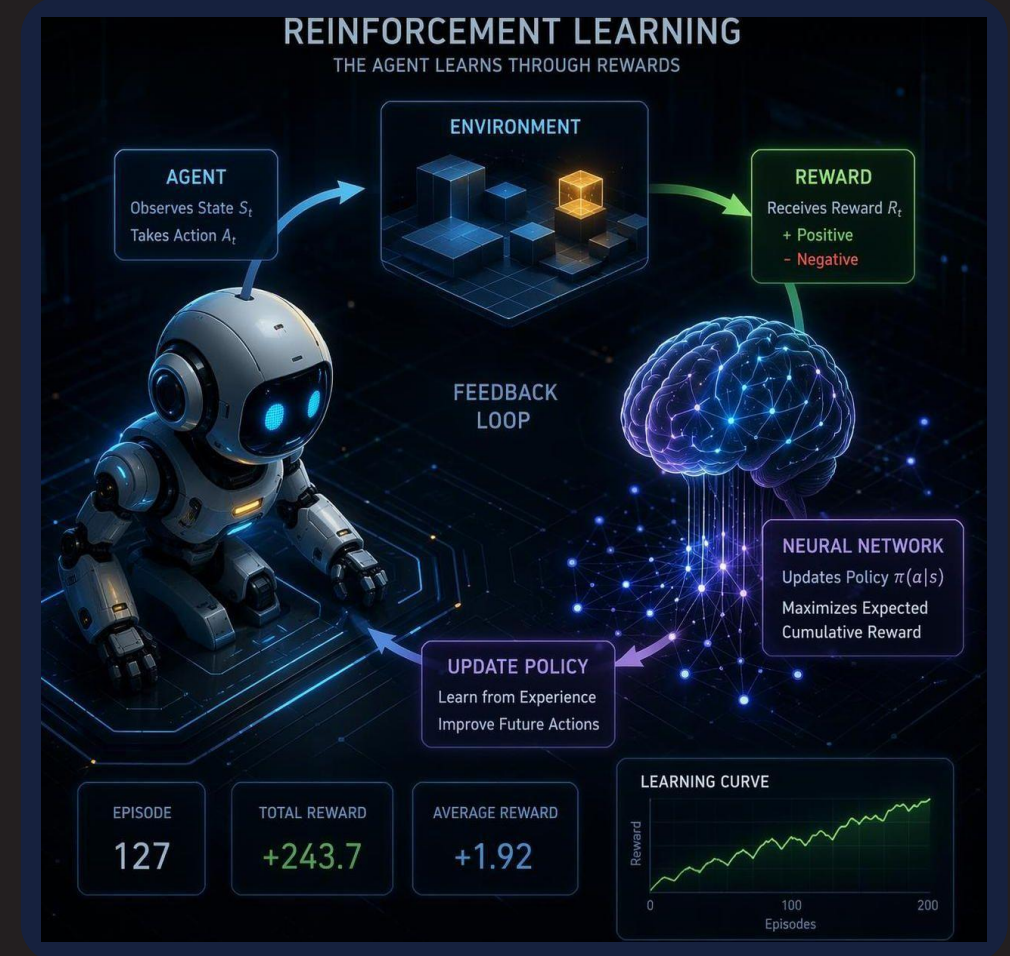
Aprendizado Não-Supervisionado

- Como funciona o aprendizado não-supervisionado:
- **Definição:** o modelo encontra padrões em dados sem rótulo, descobrindo estrutura por conta própria.
- **Objetivo:** agrupar, organizar ou simplificar dados sem orientação explícita.
- **Exemplos:** segmentação de clientes, detecção de anomalias, redução de dimensionalidade.
- **Algoritmos típicos:** K-Means, DBSCAN, PCA e autoencoders.



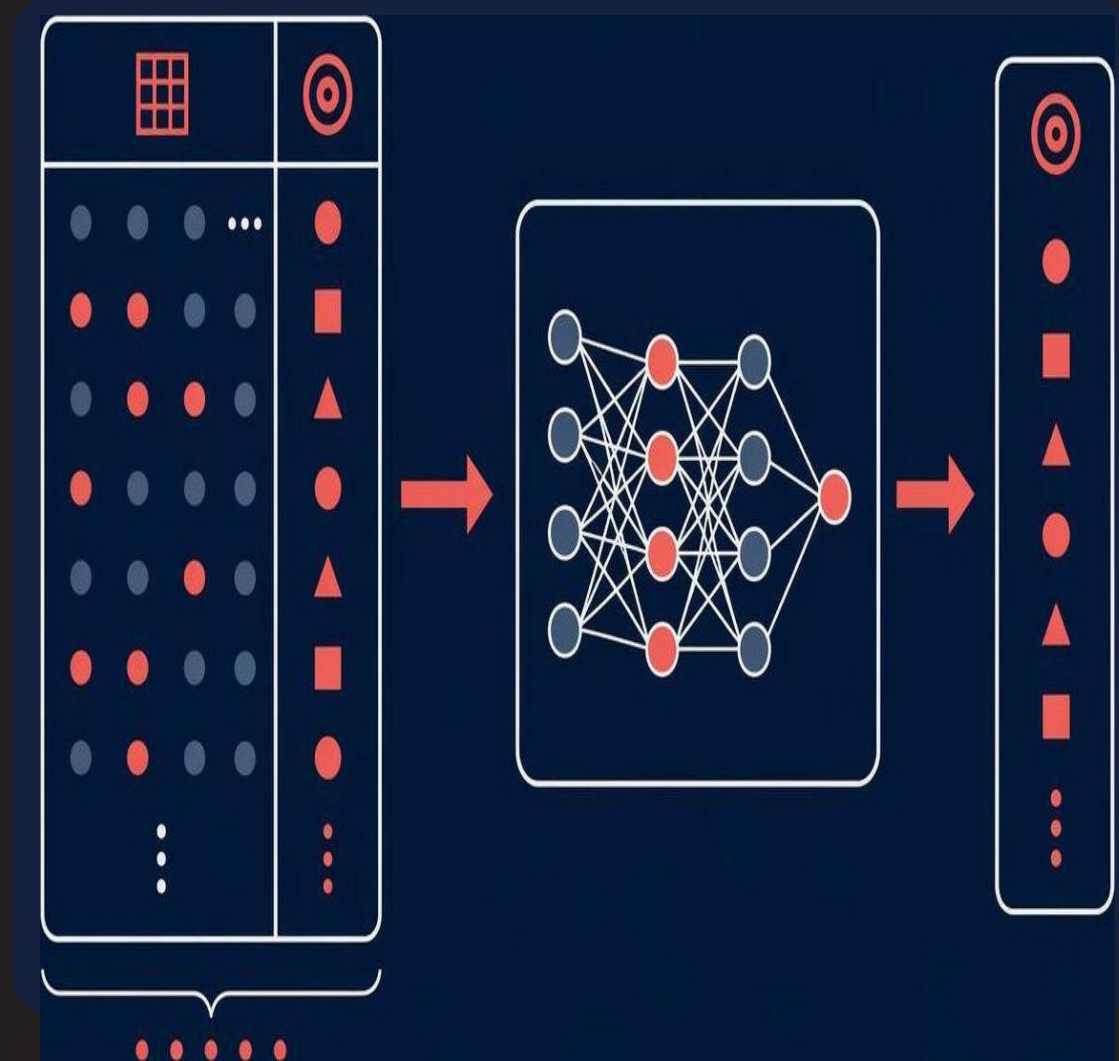
Aprendizado por Reforço

- Como funciona o aprendizado por reforço:
- **Definição:** um agente aprende interagindo com um ambiente, recebendo recompensas ou punições.
- **Objetivo:** aprender uma política que maximize a recompensa acumulada ao longo do tempo.
- **Características:** envolve exploração, exploração e decisões sequenciais.
- **Aplicações:** jogos (AlphaGo), robótica, veículos autônomos e sistemas de recomendação.



Aprendizado Supervisionado em Detalhe

- **Ideia central:** aprender uma função $f(X) \approx Y$ a partir de exemplos já rotulados pelo humano;
- **Dados de entrada:** pares (X, Y) — features X (atributos) e rótulo Y (resposta desejada);
- **Fluxo típico:** dividir em treino/teste → treinar o modelo → avaliar com métricas → ajustar;
- **Exemplos cotidianos:** filtro de spam, previsão de churn, diagnóstico médico, recomendação de preço;
- **Por que é tão popular?** resultados objetivos, métricas claras e fácil integração em produtos.



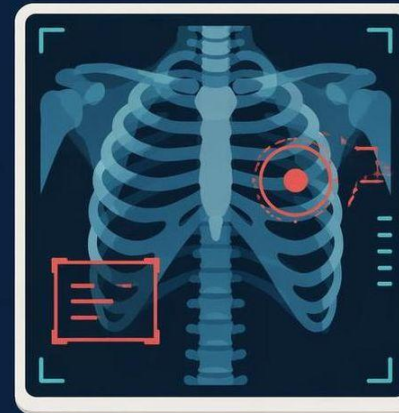
Aplicações Reais — Regressão

- **Preço de imóveis (Zillow, QuintoAndar):** estimar valor de venda a partir de área, localização, idade do imóvel e comparativos;
- **Previsão de demanda (Amazon, iFood):** antecipar pedidos por região e horário para dimensionar estoque e entregadores;
- **Risco de crédito (Nubank, Serasa):** calcular probabilidade de inadimplência e definir limite com base em histórico financeiro;
- **Energia e saúde:** prever consumo elétrico, tempo de internação ou a evolução de exames clínicos;
- **Marketing (Spotify, Netflix):** estimar lifetime value (LTV) e retorno esperado de campanhas para priorizar investimentos.



Aplicações Reais — Classificação

- **Filtro de spam (Gmail, Outlook):** separar e-mails legítimos de spam usando texto, remetente e padrões de envio;
- **Deteção de fraude (Visa, PicPay):** identificar transações suspeitas em tempo real a partir de padrões de gasto e localização;
- **Diagnóstico por imagem (radiologia, dermatologia):** classificar raio-X, ressonância ou foto de pele como saudável vs. patológica;
- **Previsão de churn (telecom, SaaS):** apontar clientes com alta chance de cancelar para acionar retenção antecipada;
- **Visão computacional (iPhone Face ID, Tesla):** reconhecer rostos, placas, pedestres e sinais de trânsito em imagens e vídeo.

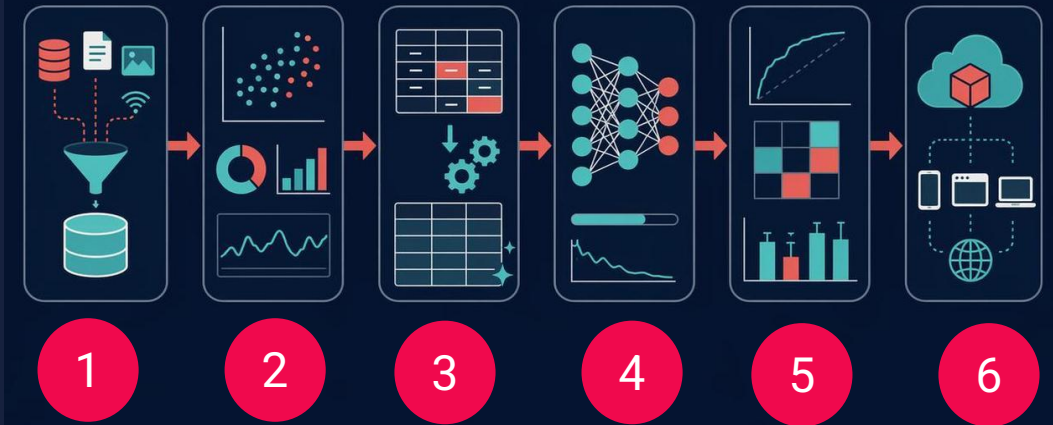


Tarefas Supervisionadas: Regressão vs Classificação

- **O tipo de Y define a tarefa:** contínuo → regressão; categórico → classificação;
- **Regressão:** prever um número — preço de imóvel, temperatura, nota, demanda;
- **Classificação:** prever uma classe — spam/não-spam, doente/saudável, gato/cachorro;
- **Métricas mudam:** regressão usa MAE, RMSE, R^2 ; classificação usa acurácia, precisão, recall, F1;
- **Próximo passo:** mergulhar em Regressão Linear — o algoritmo mais clássico de regressão.

Pipeline de Machine Learning

- **1. Coleta:** reunir dados de bancos, APIs, sensores ou web — qualidade conta mais que volume;
- **2. Exploração (EDA):** entender distribuições, correlações, lacunas e outliers antes de modelar;
- **3. Pré-processamento:** limpar, normalizar, codificar variáveis e criar novas features;
- **4. Treinamento:** escolher o algoritmo, ajustar hiperparâmetros e otimizar a função de perda;
- **5. Avaliação:** medir o desempenho em dados não vistos com métricas adequadas à tarefa;
- **6. Deploy + monitoramento:** colocar em produção e acompanhar para evitar degradação ao longo do tempo.



Pré-Processamento e Engenharia de Atributos

- **Dados faltantes:** preencher com média/mediana, KNN-imputer ou descartar linhas/colunas;
- **Outliers:** identificar com IQR ou Z-score e tratar (cap, transformar, remover);
- **Normalização:** Min-Max ou Z-score para algoritmos sensíveis à escala (KNN, SVM, redes neurais);
- **Variáveis categóricas:** Label Encoding (ordinais) ou One-Hot Encoding (sem ordem natural);
- **Feature Engineering:** criar atributos novos (razões, agregações, datas, interações) — alavanca de performance.

													
	1/5/23	\$1,200	Premium	Y			2023-01-05	1200.00	1	0	0	1	0
	2023-01-07	1200.00	premium	Yes			2023-01-07	1200.00	1	0	0	1	0
	?	\$ 950	PREMIUM	1			2023-01-12	950.00	1	0	0	1	0
	03-12-23	1000	Basic	N			2023-01-13	1000.00	0	1	0	0	1
	Jan 13 2023	\$1.5K	basic	No			2023-01-15	1500.00	0	1	0	0	1
	2023/01/15		Standard	1			2023-01-16	1100.00	0	0	1	1	0
		850.5	STD	0			2023-01-17	850.50	0	0	1	0	1
	1-17-23	\$ -		yes			2023-01-17	900.00	0	1	0	1	0
	2023.01.19	1,300.00	Basic				2023-01-19	1300.00	0	1	0	0	1
	not available	N/A	standard	No			2023-01-20	800.00	0	0	1	0	1

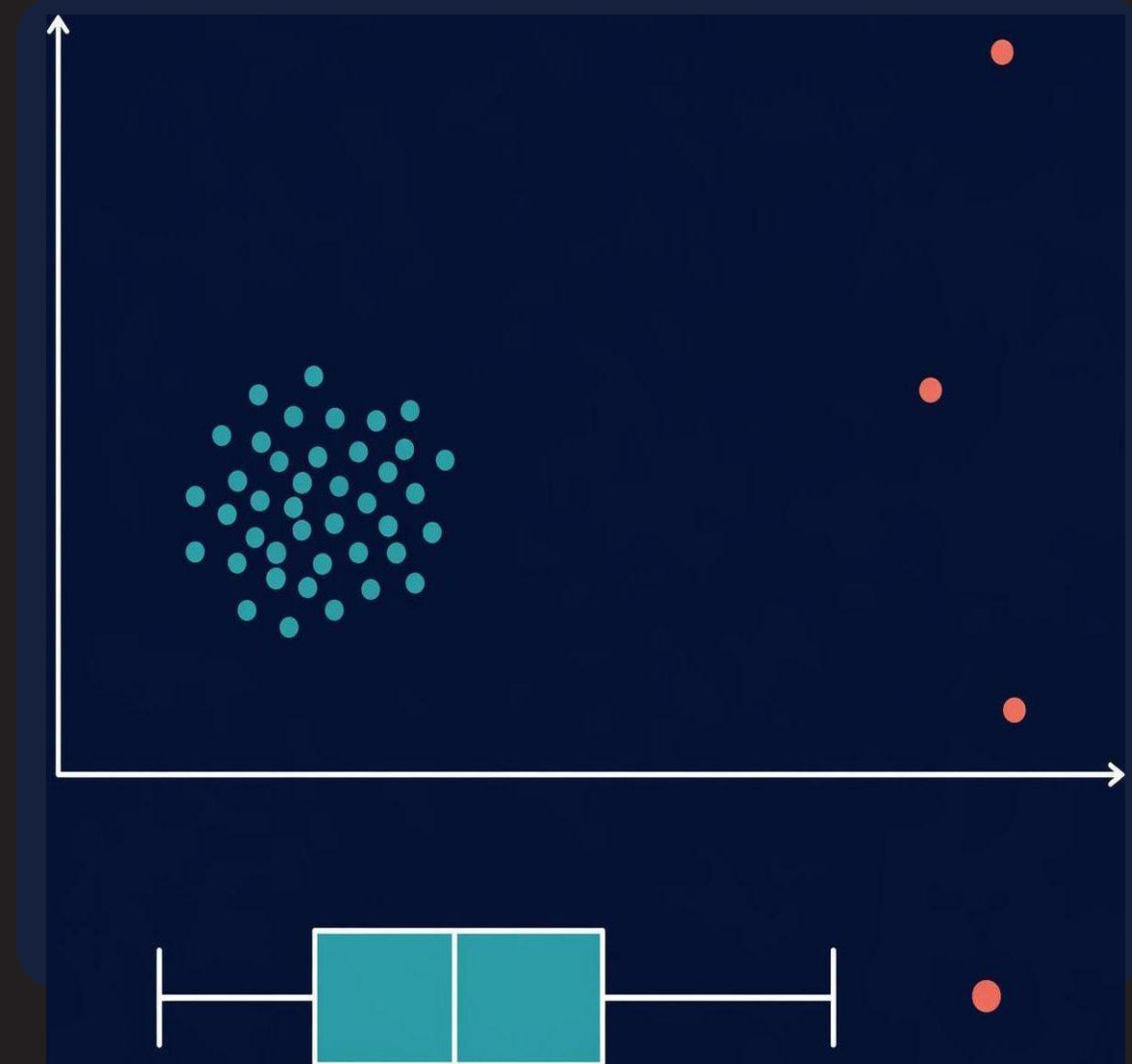
Dados Faltantes — A Primeira Limpeza

- **Por que aparecem:** sensor falhou, cliente não respondeu, sistema legado descontinuado — todo dataset real tem buracos;
- **Diagnóstico:** mapear % de NaN por coluna e visualizar com heatmap antes de decidir o tratamento;
- **Estratégias simples:** preencher com média (numéricos), mediana (quando há outliers) ou moda (categóricos);
- **Estratégias avançadas:** KNN-Imputer, regressão para prever o faltante ou modelos iterativos como MICE;
- **Quando descartar:** coluna com mais de 50% de NaN ou linhas pontuais isoladas — sempre documente a decisão.

					→					
23	67		88	12	→	23	67	39	88	12
45		32	76	19	→	45	63	32	76	19
31	58	41		17	→	31	58	41	78	17
29	73		64	15	→	29	73	39	64	15
50	61	38	91		→	50	61	38	91	16
36		45	72	20	→	36	63	45	72	20
42	66		80	18	→	42	66	39	80	18
28	54	37		13	→	28	54	37	78	13
33		39	70	16	→	33	63	39	70	16
47	69	43	85		→	47	69	43	85	16

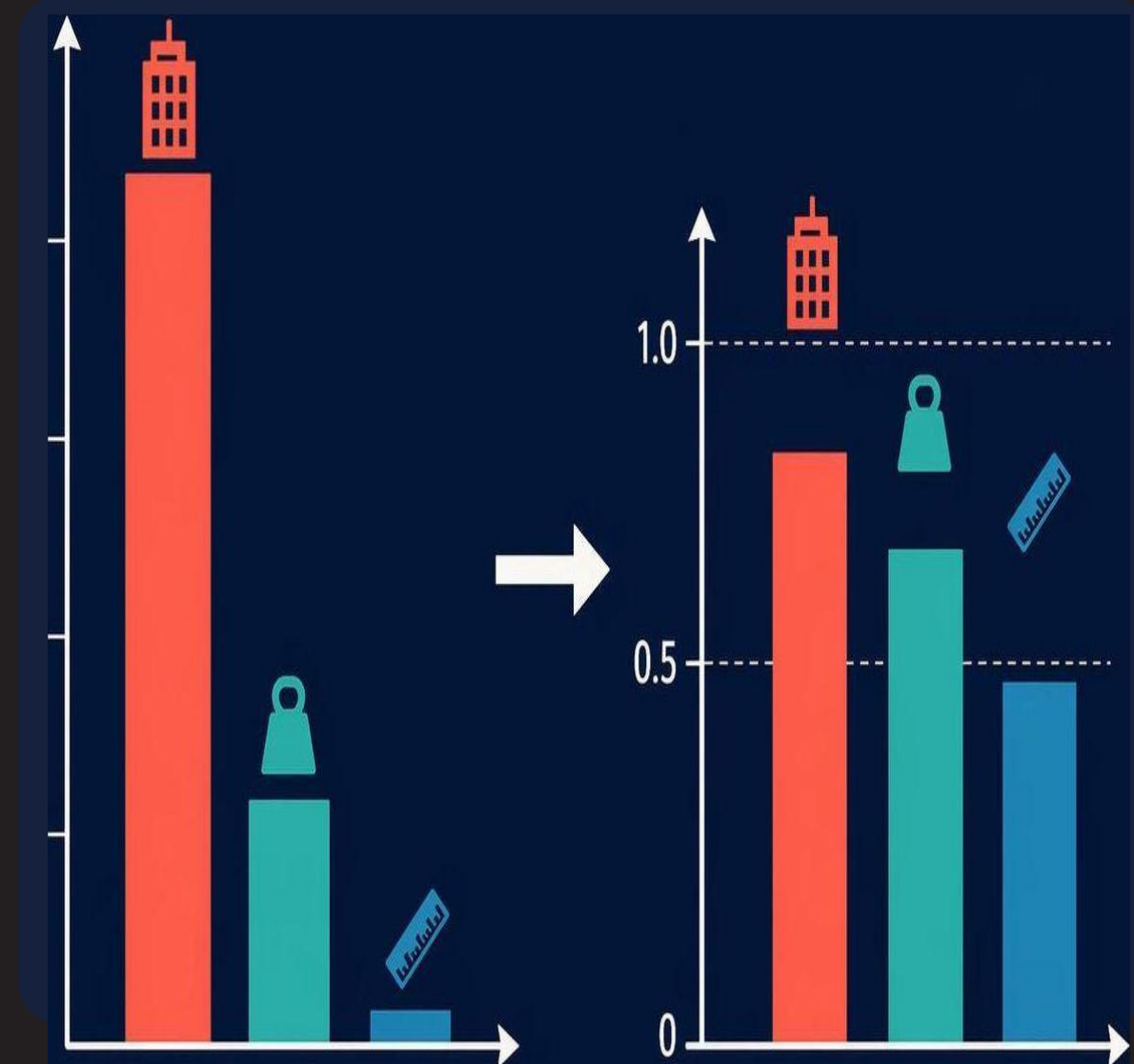
Outliers — Quando o Estranho Importa

- **O que são:** observações muito distantes do padrão — podem ser erro de digitação ou evento raro genuíno;
- **IQR:** ponto fora de $[Q1 - 1,5 \cdot IQR ; Q3 + 1,5 \cdot IQR]$ é candidato — funciona bem em distribuições simétricas;
- **Z-score:** $|z| > 3$ sugere outlier sob distribuição aproximadamente normal;
- **Tratamento:** cap (winsorização), transformação (log/raiz), remoção pontual ou usar modelos robustos;
- **Atenção:** em fraude ou medicina, o outlier costuma ser o sinal mais valioso — investigue antes de remover.



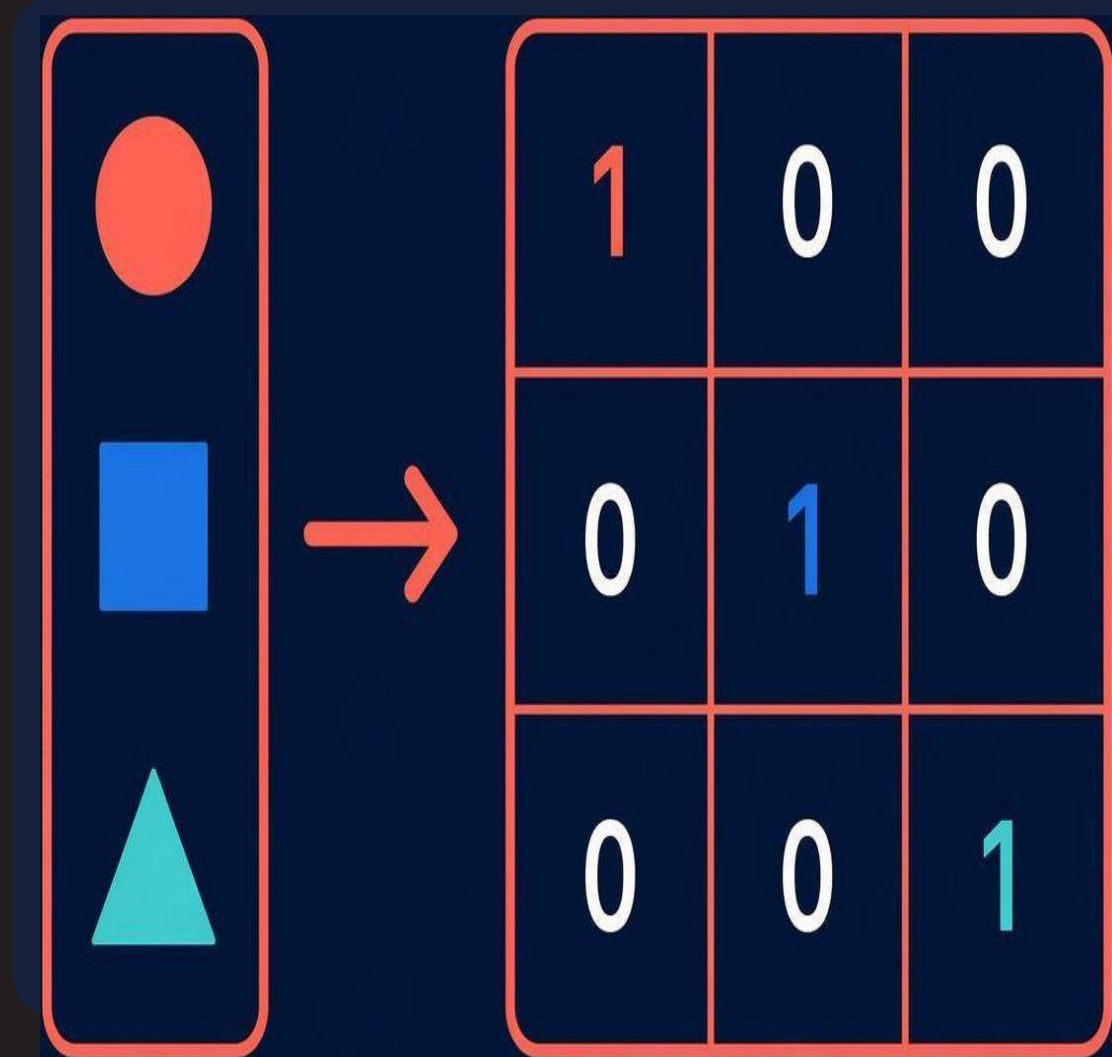
Normalização & Padronização

- **Por que escalar:** KNN, SVM, PCA e redes neurais são sensíveis à escala — features grandes dominam o cálculo;
- **Min-Max [0, 1]:** preserva a forma da distribuição; ideal quando os limites são conhecidos;
- **Z-score (StandardScaler):** centra em 0 com desvio 1; robusto quando não há limites fixos;
- **RobustScaler:** usa mediana e IQR — sobrevive bem a outliers extremos;
- **Boa prática:** ajustar o scaler SÓ no treino e aplicar (transform) no teste — evita data leakage.



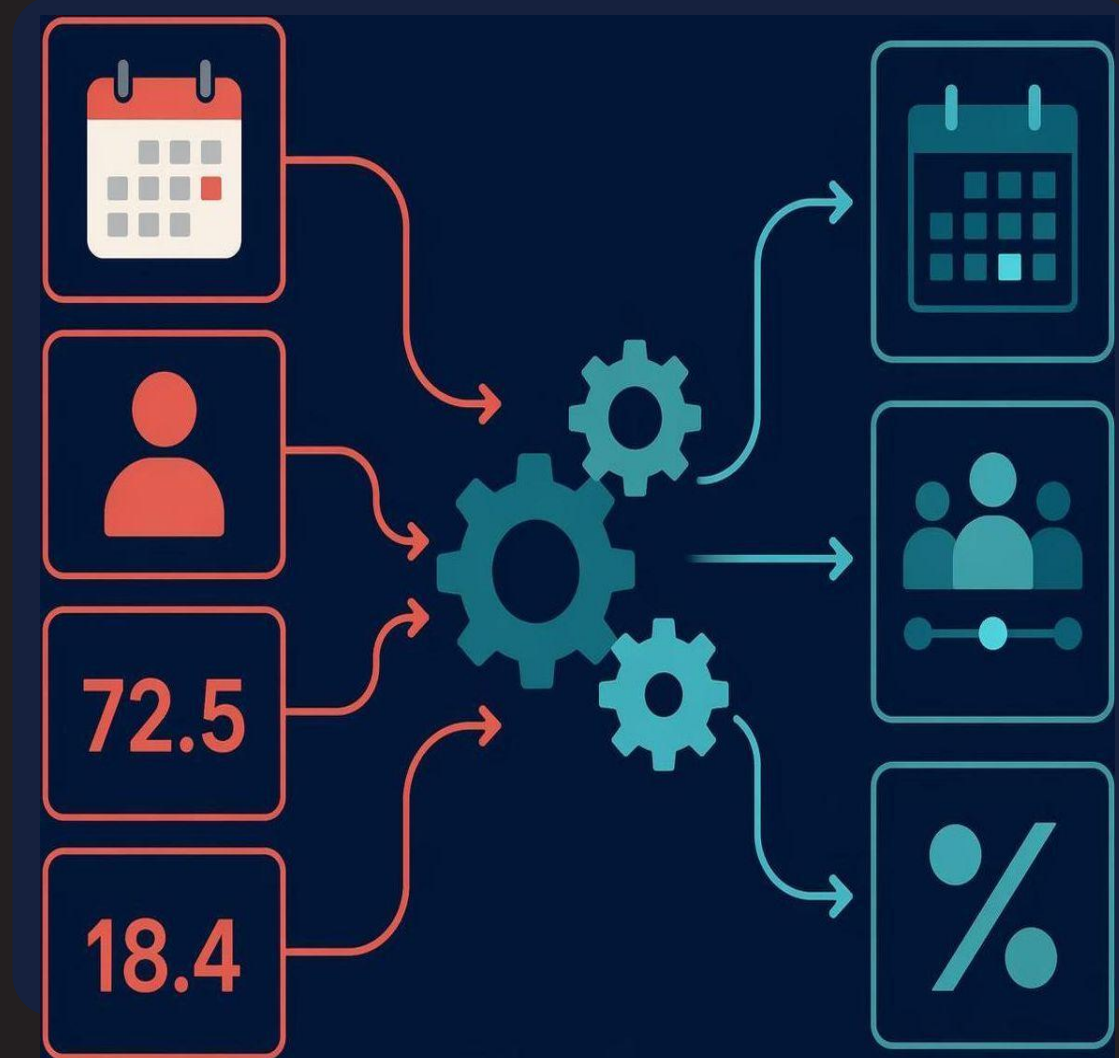
Codificando Variáveis Categóricas

- **O problema:** algoritmos exigem números — texto não entra direto no modelo;
- **Label Encoding:** mapeia cada categoria para um inteiro — útil em variáveis ORDINAIS (pequeno < médio < grande);
- **One-Hot Encoding:** cria uma coluna binária por categoria — padrão para variáveis sem ordem natural;
- **Target Encoding:** substitui categoria pela média do alvo — poderoso, mas exige cuidado com vazamento;
- **Alta cardinalidade:** com muitas categorias, considere embeddings, hashing ou agrupar por frequência.



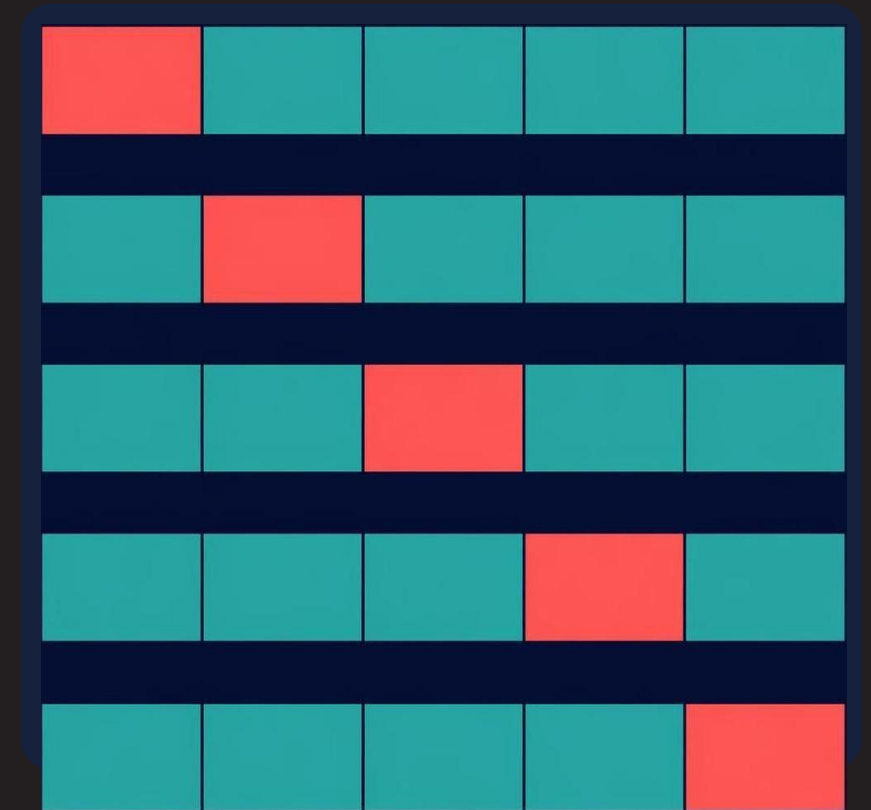
Feature Engineering — Onde Mora a Mágica

- **Definição:** criar novos atributos a partir dos dados brutos — onde a criatividade do cientista aparece;
- **Datas:** extrair dia da semana, mês, sazonalidade, feriado e hora do dia transforma uma coluna em várias;
- **Razões e agregações:** preço por m², ticket médio do cliente, média móvel dos últimos 7 dias;
- **Interações:** combinações entre variáveis (ex.: renda × escolaridade) capturam efeitos não-lineares;
- **Domínio importa:** modelo simples + ótimas features quase sempre vence modelo complexo + features ruins.



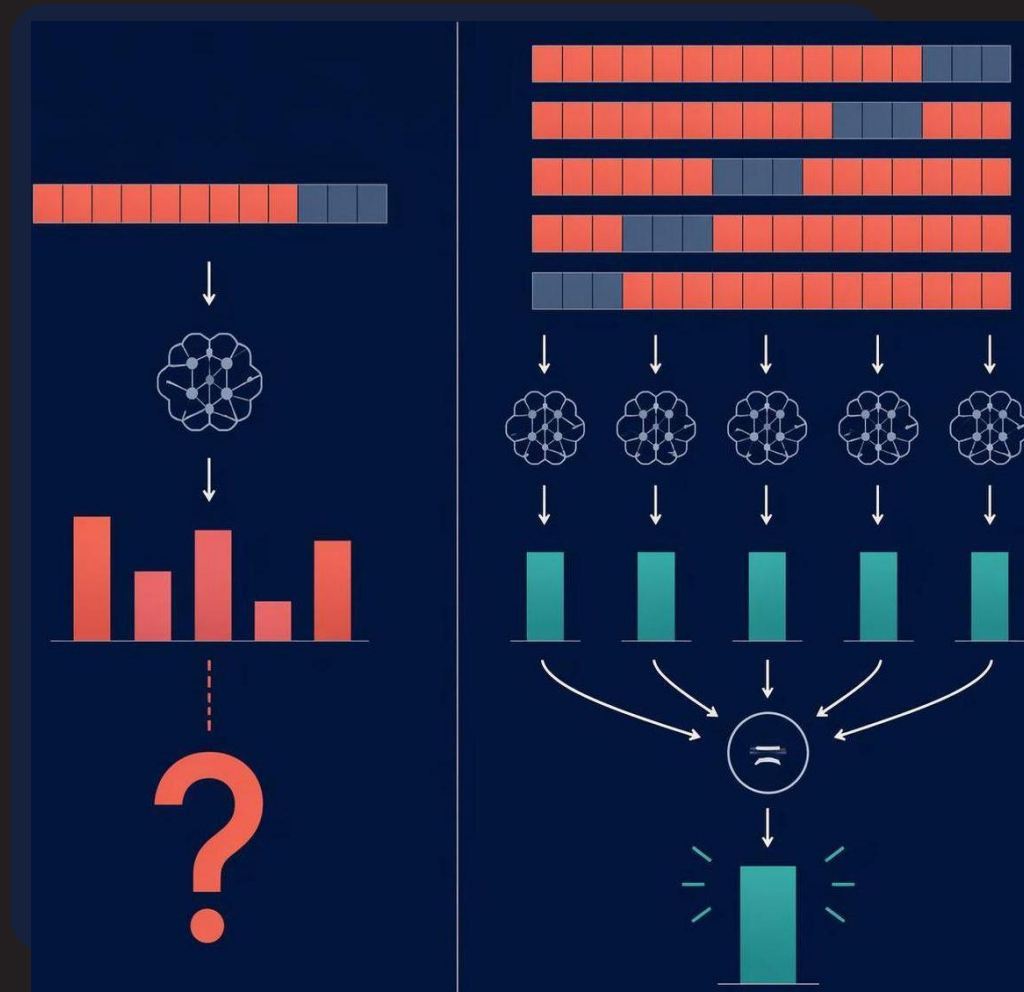
Divisão de Dados e Validação Cruzada

- **Train/Test split:** 70–80% para treinar e 20–30% para testar; o teste só é tocado no final;
- **Validação:** parte do treino vira “validação” para escolher hiperparâmetros sem espiar o teste;
- **K-Fold Cross-Validation:** divide em K partes (ex.: K=5) e treina K vezes alternando o fold de validação;
- **Vantagem:** usa todos os dados para treino e validação e reduz a variância da métrica;
- **Stratified K-Fold:** preserva a proporção das classes — essencial em classificação desbalanceada.



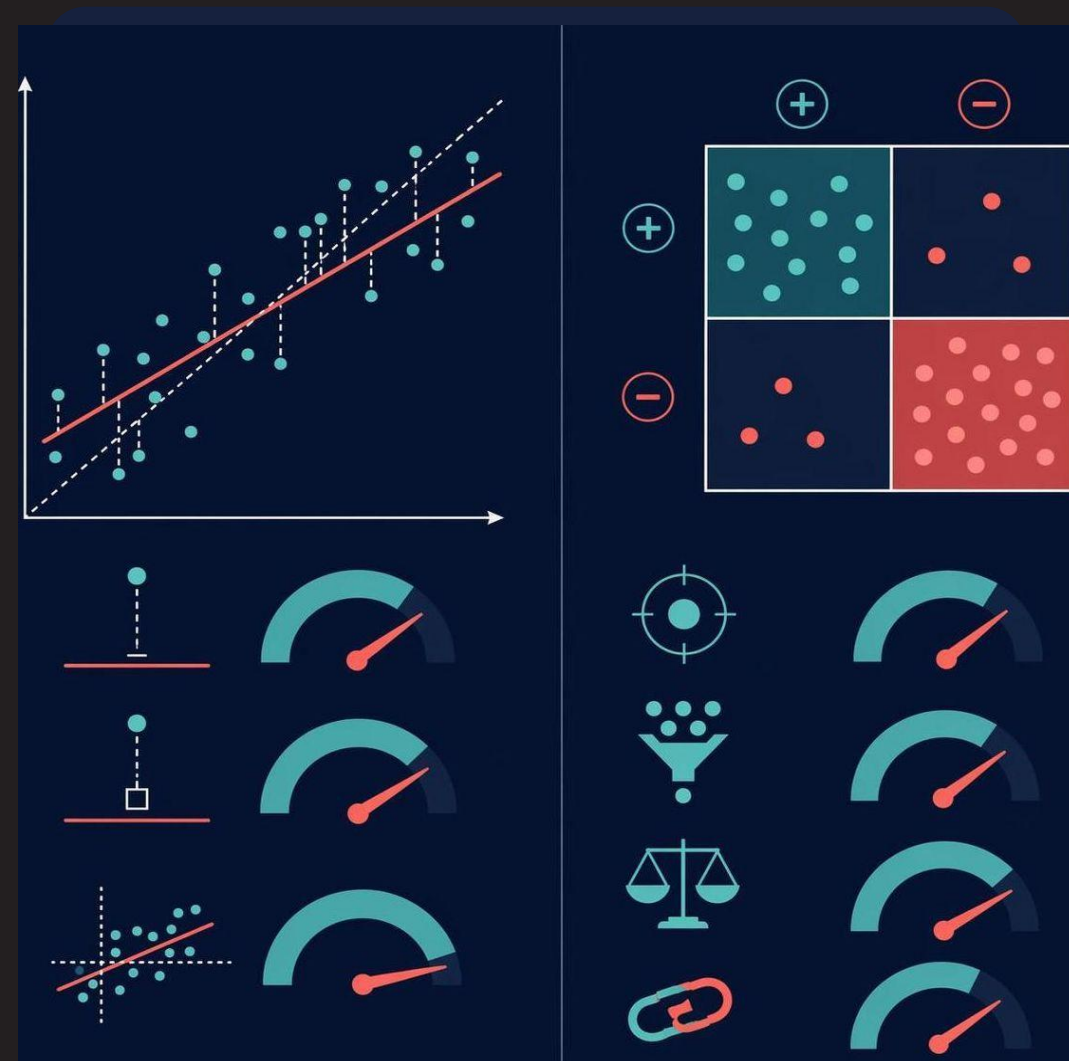
Por Que Usar Validação Cruzada?

- **Um split sozinho engana:** a métrica obtida em um único train/test depende do sorteio — pode ser sorte ou azar;
- **Estimativa mais confiável:** a média de várias rodadas reduz a variância e revela o desempenho real do modelo;
- **Aproveita todos os dados:** cada exemplo entra no treino em $K-1$ rodadas e na validação em uma — nada é desperdiçado;
- **Detecta instabilidade:** métricas muito diferentes entre folds avisam que o modelo é sensível ao subconjunto de treino;
- **Compara modelos com justiça:** avaliar todos sobre os mesmos folds elimina vantagem aleatória de um modelo “sortudo”;
- **Protege o conjunto de teste:** permite escolher hiperparâmetros sem encostar no teste — evita decisões enviesadas.



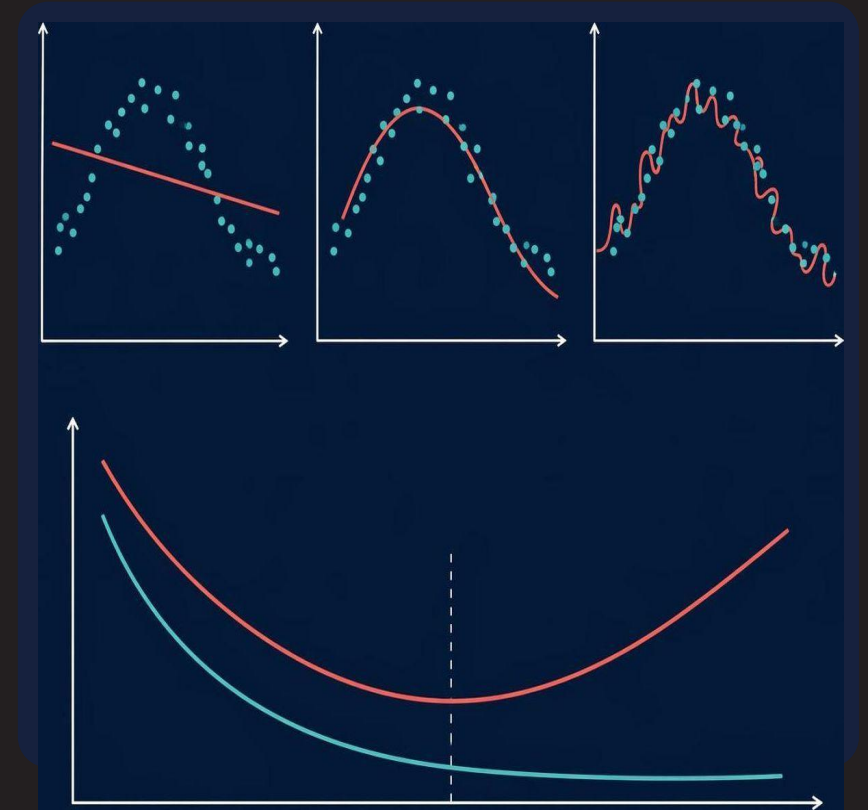
Métricas por Tipo de Tarefa

- **Regressão — MAE:** erro médio absoluto, fácil de explicar (“erro de X unidades”);
- **Regressão — RMSE/MSE:** penaliza erros grandes; use RMSE para resultado na unidade original;
- **Regressão — R^2 :** fração da variação de Y explicada pelo modelo, entre 0 e 1;
- **Classificação:** matriz de confusão, acurácia, precisão, recall, F1 e ROC-AUC;
- **Cuidado:** acurácia engana em dados desbalanceados — prefira F1 ou ROC-AUC nesses casos.



Overfitting vs Underfitting (Viés-Variância)

- **Underfit (alto viés):** modelo simples demais — erra muito no treino E no teste;
- **Overfit (alta variância):** decora o treino mas falha em dados novos — generalização ruim;
- **Ponto ideal:** mínimo do erro de validação — modelo equilibra viés e variância;
- **Combatendo overfit:** mais dados, regularização (Ridge/Lasso), early stopping, dropout, simplificar o modelo;
- **Combatendo underfit:** aumentar a complexidade, adicionar features relevantes ou treinar mais tempo.



Métricas de Avaliação de Modelos de ML

Como medir a qualidade das previsões do seu modelo

Por Que Avaliar?

- Um modelo que parece bom no treino pode falhar em produção
- Métricas quantificam o desempenho de forma objetiva
- Diferentes problemas exigem diferentes métricas
- O que vamos cobrir:
 - Matriz de Confusão — base para entender erros
 - Precisão, Recall, F1-Score — métricas de classificação
 - Curva ROC e AUC — análise visual de desempenho
 - MSE, RMSE, MAE, R^2 — métricas de regressão
 - Guia prático para escolher a métrica certa

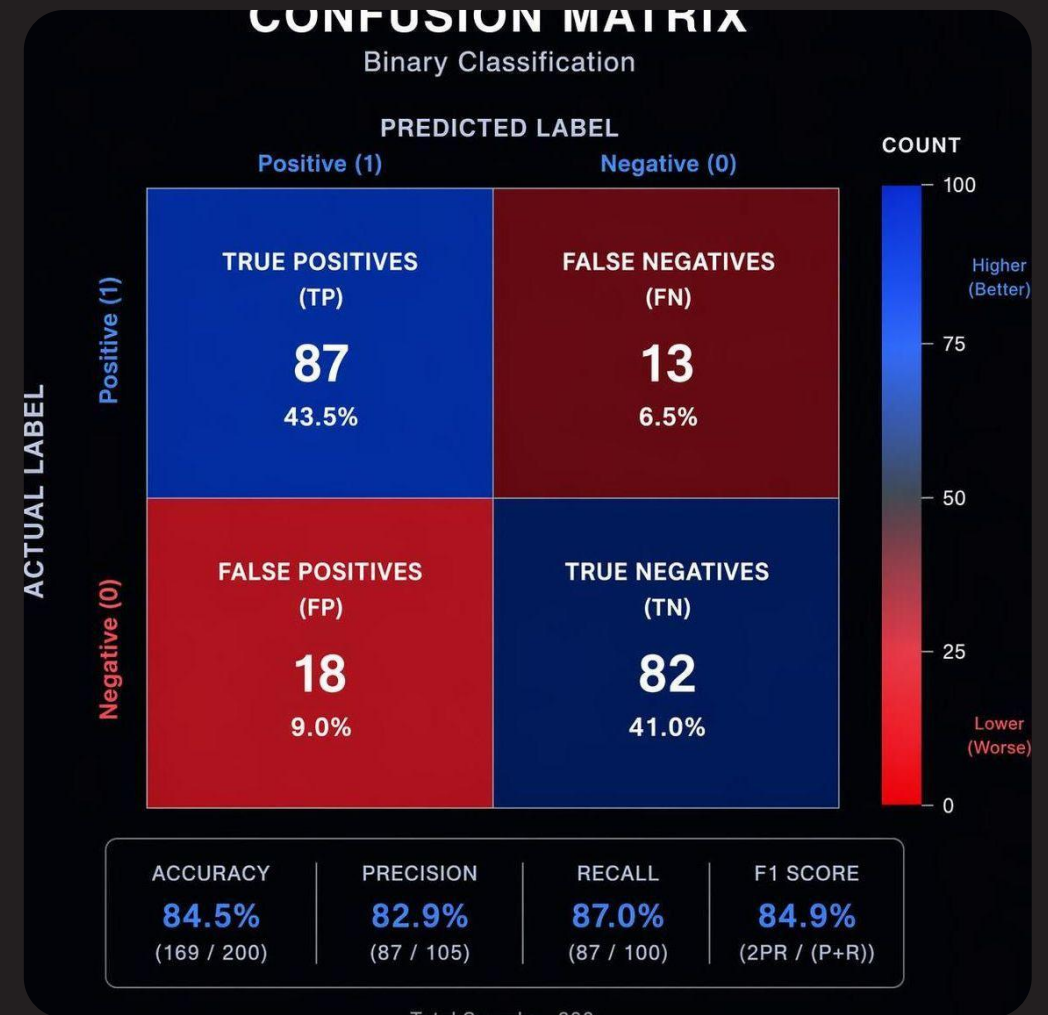


Matriz de Confusão

A base para entender os erros do modelo de classificação

Os 4 Quadrantes

- Verdadeiro Positivo (VP): modelo previu positivo e acertou
- Verdadeiro Negativo (VN): modelo previu negativo e acertou
- Falso Positivo (FP): modelo previu positivo mas errou — “alarme falso”
- Falso Negativo (FN): modelo previu negativo mas errou — “caso perdido”
- Exemplo: Diagnóstico médico
- FP = paciente saudável diagnosticado doente (custo: exames extras)
- FN = paciente doente não detectado (custo: risco de vida!)
- O equilíbrio entre FP e FN depende do contexto do problema.



Métricas de Classificação

As quatro métricas fundamentais para modelos de classificação

Acurácia

- Fórmula: $(VP + VN) / \text{Total}$
- Proporção de previsões corretas
- ⚠ Cuidado com dados desbalanceados! 95% de acurácia pode significar que o modelo ignora a classe rara

Precisão

- Fórmula: $VP / (VP + FP)$
- Dos que o modelo disse “positivo”, quantos realmente são?
- Use quando falsos positivos custam caro (ex: spam, fraude)

Recall (Sensibilidade)

- Fórmula: $VP / (VP + FN)$
- Dos positivos reais, quantos o modelo encontrou?
- Use quando falsos negativos custam caro (ex: câncer, segurança)

F1-Score

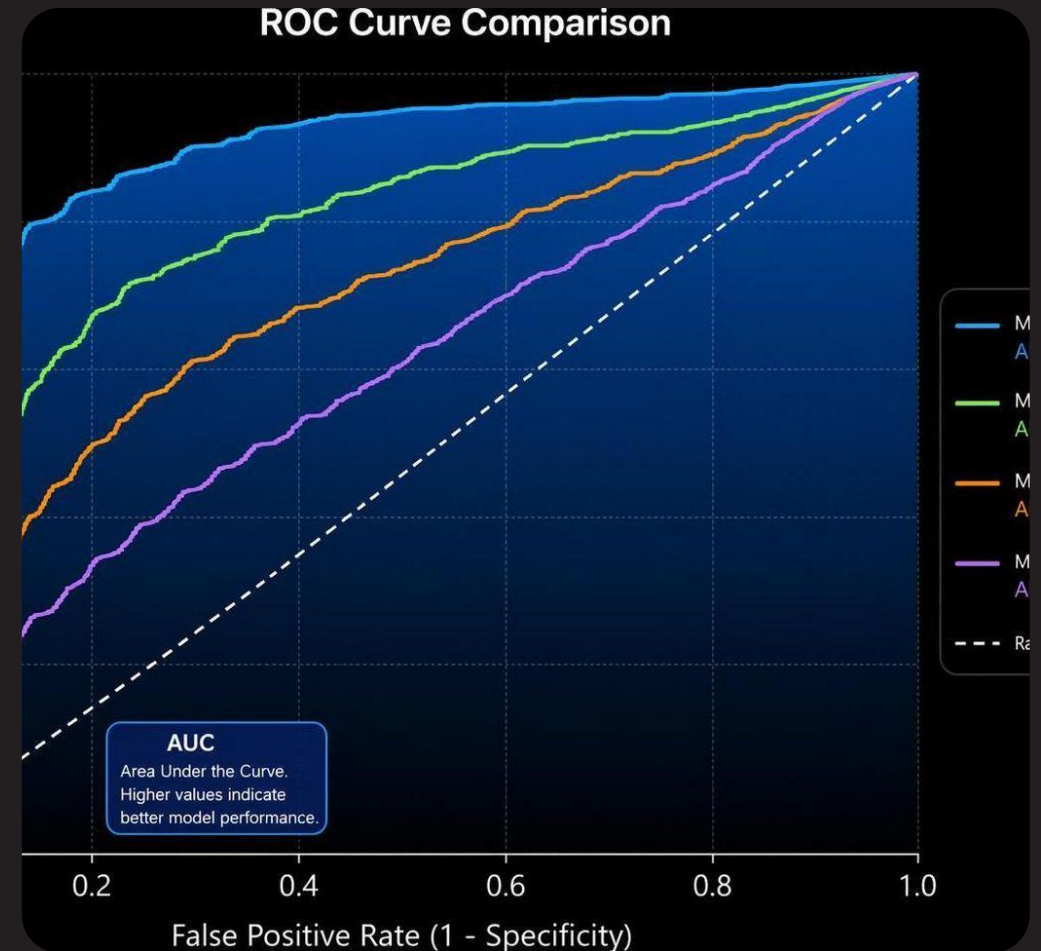
- Fórmula: $2 \times (\text{Precisão} \times \text{Recall}) / (\text{Precisão} + \text{Recall})$
- Média harmônica entre precisão e recall
- Ideal quando há desbalanceamento e ambos os erros importam
- Varia de 0 (pior) a 1 (perfeito)

Curva ROC e AUC

Visualizando o trade-off entre sensibilidade e especificidade

Entendendo a Curva ROC

- A curva ROC plota Taxa de VP (Recall) vs Taxa de FP
- Cada ponto na curva representa um limiar de decisão diferente
- Interpretando a AUC (Área Sob a Curva):
 - $AUC = 1.0$ → modelo perfeito
 - $AUC = 0.5$ → modelo aleatório (linha diagonal)
 - $AUC < 0.5$ → pior que aleatório
- Quando usar:
 - Comparar modelos independente do limiar
 - Dados desbalanceados (melhor que acurácia)
 - Escolher o limiar ideal para o seu contexto

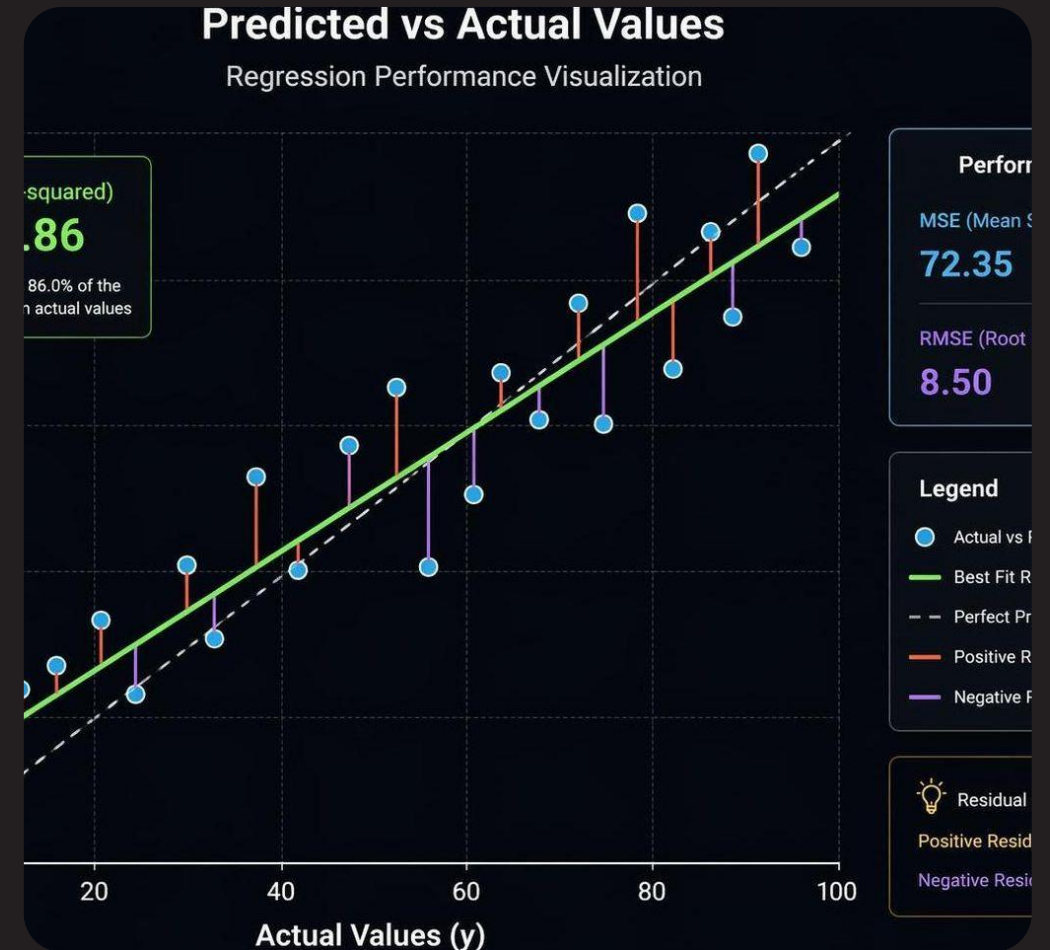


Métricas de Regressão

Quantificando o erro de modelos que preveem valores contínuos

As Métricas Essenciais

- MSE (Erro Quadrático Médio)
- Média dos erros ao quadrado — penaliza erros grandes
- RMSE (Raiz do MSE)
- Na mesma unidade da variável alvo — mais interpretável
- MAE (Erro Absoluto Médio)
- Média dos erros absolutos — robusto a outliers
- R^2 (Coeficiente de Determinação)
- Proporção da variância explicada pelo modelo
- $R^2 = 1 \rightarrow$ modelo perfeito; $R^2 = 0 \rightarrow$ modelo não explica nada
- Pode ser negativo se o modelo for pior que a média



Guia de Seleção de Métricas

Escolha a métrica certa para o seu problema

Classificação Balanceada

- Métrica principal: Acurácia
- Apoio: F1-Score
- Quando usar:
 - Classes com proporções similares
 - Ex: sentimento positivo/negativo
 - Ex: classificação de imagens

Classificação Desbalanceada

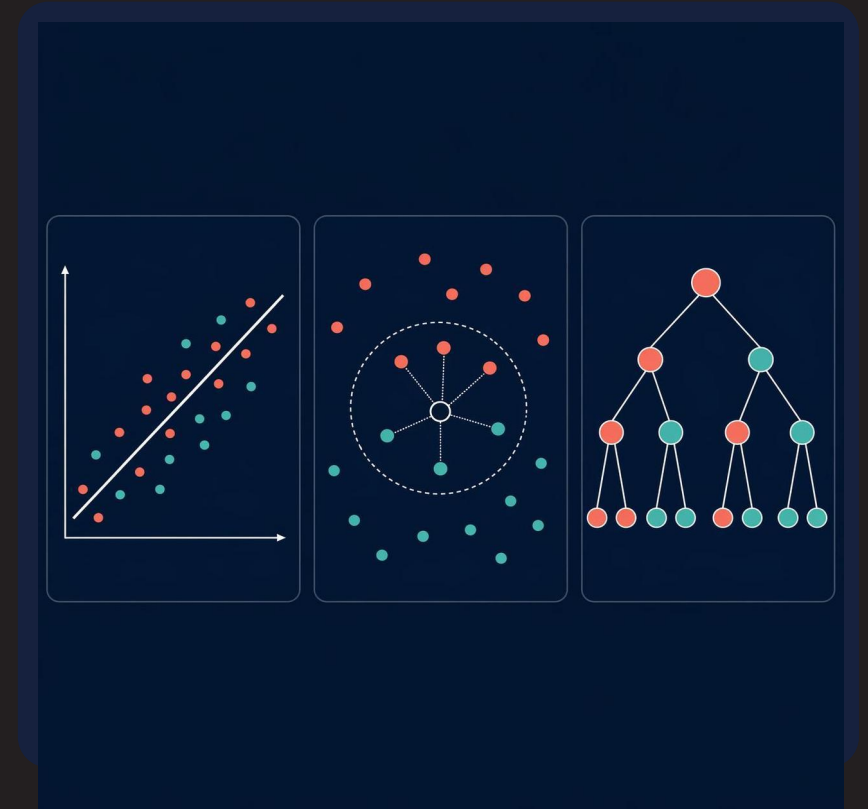
- Métrica principal: F1-Score ou AUC
- Apoio: Precisão e Recall
- Quando usar:
 - Classe rara (fraude, doença)
 - Custo assimétrico de erros
 - FN grave → priorize Recall
 - FP grave → priorize Precisão

Regressão

- Métrica principal: RMSE ou MAE
- Apoio: R^2
- Quando usar:
 - RMSE: penalizar erros grandes
 - MAE: robustez a outliers
 - R^2 : explicabilidade do modelo
 - Sempre compare com baseline simples (média)

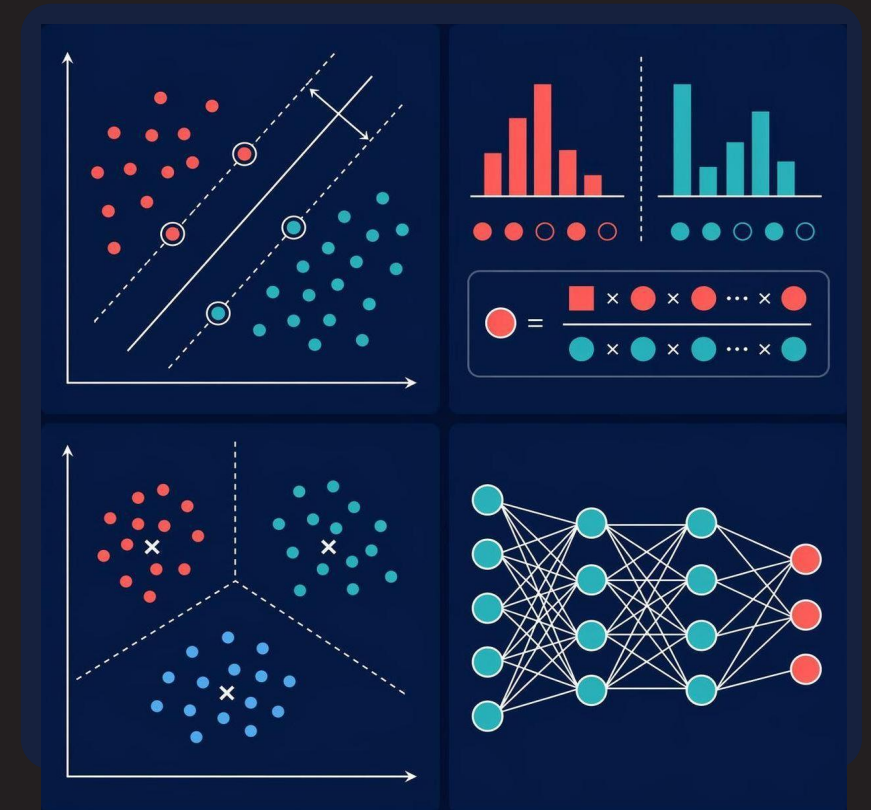
Panorama de Algoritmos — Parte 1

- **Regressão Linear / Logística:** simples e interpretáveis — ótimos como baseline para prever números ou classes binárias.
- **K-Nearest Neighbors (KNN):** classifica pelo voto dos vizinhos mais próximos; sensível à escala — sempre normalize antes.
- **Árvores e Florestas:** Decision Trees, Random Forest e Gradient Boosting dominam dados tabulares e lidam bem com features mistas.



Panorama de Algoritmos — Parte 2

- **SVM (Support Vector Machine):** encontra a fronteira com margem máxima entre classes — forte em alta dimensionalidade.
- **Naive Bayes:** probabilístico, leve e rápido; brilha em classificação de texto e filtragem de spam.
- **Não-supervisionados (K-Means, PCA):** K-Means agrupa por similaridade; PCA reduz dimensionalidade preservando variância.
- **Redes Neurais:** base do Deep Learning — dominam imagem, áudio e linguagem natural quando há muitos dados.



Ética e Responsabilidade em IA

- **Viés algorítmico:** dados históricos enviesados reproduzem desigualdades — o modelo aprende o passado;
- **Fairness:** medir e mitigar disparidades entre grupos (gênero, raça, renda) com métricas próprias;
- **Privacidade:** respeitar LGPD/GDPR; anonimização, minimização de dados e consentimento informado;
- **Transparência:** usar técnicas de explicabilidade (SHAP, LIME) para abrir a “caixa-preta”;
- **Responsabilidade:** definir quem responde por decisões automatizadas — accountability humana sempre presente.



Aprendizado de Máquina – Classificação

Algoritmos para prever a categoria de novos exemplos

RAW DATA

CLASSIFICATION
MODEL

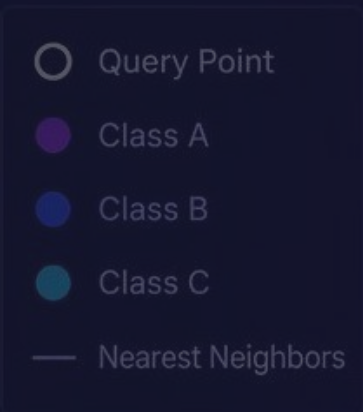
CLASS A

CLASS B

CLASS C

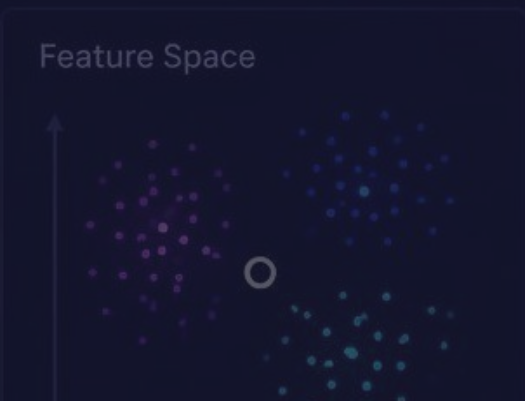


$k = 5$



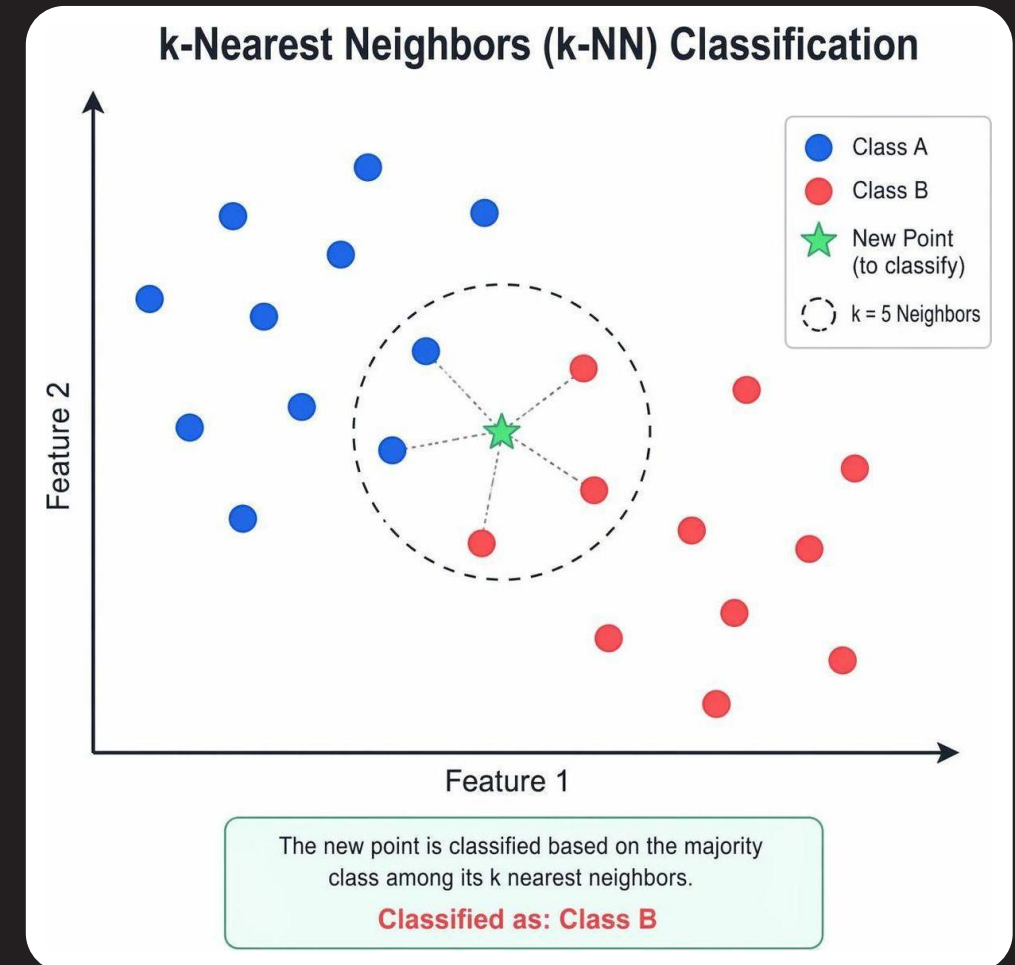
K-Nearest Neighbors (KNN)

Classificando pelo voto dos vizinhos mais próximos



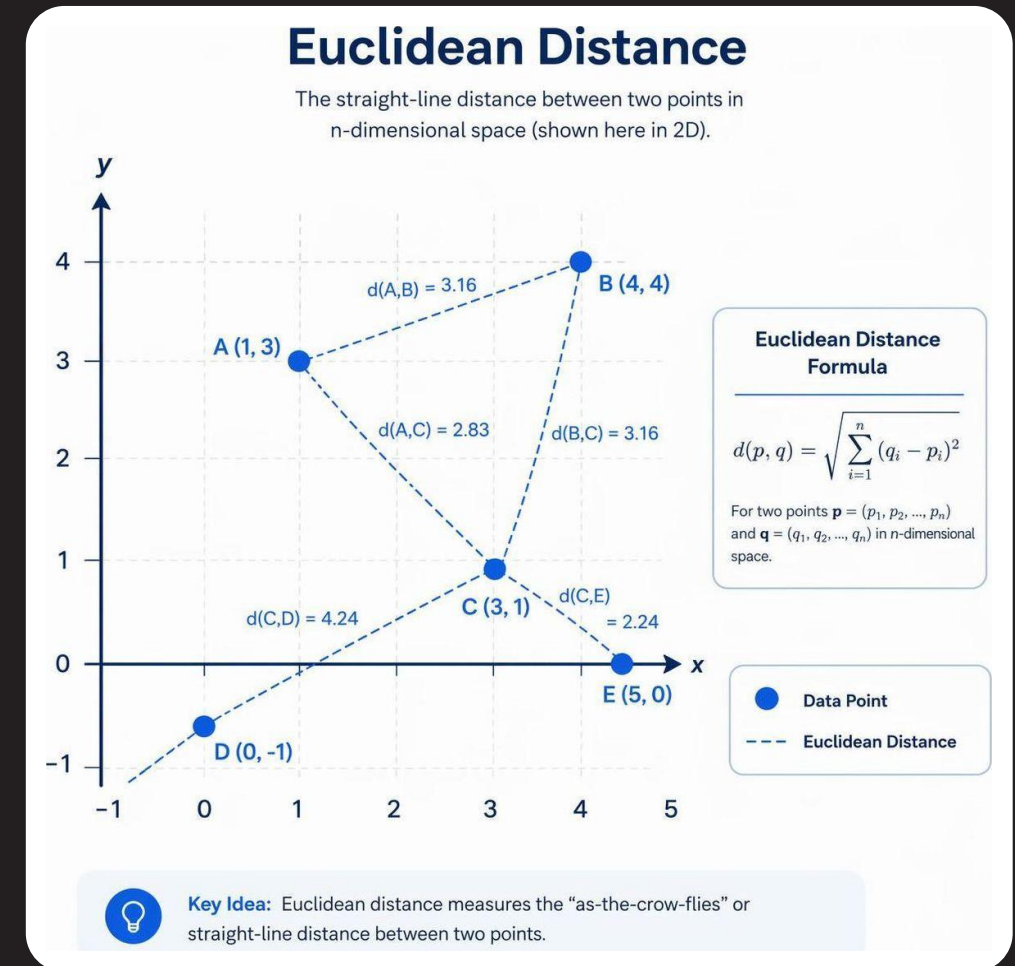
KNN – Intuição

- **Ideia central:** classificar um novo ponto pelo voto majoritário dos seus k vizinhos mais próximos.
- **Lazy learning:** não há fase de treino real — o modelo apenas guarda os dados.
- **Não-paramétrico:** não assume nenhuma forma para a fronteira de decisão.
- **Usos:** classificação, regressão (média dos vizinhos) e detecção de anomalias.
- **Exemplos:** recomendação por similaridade, reconhecimento de padrões, diagnósticos clínicos.



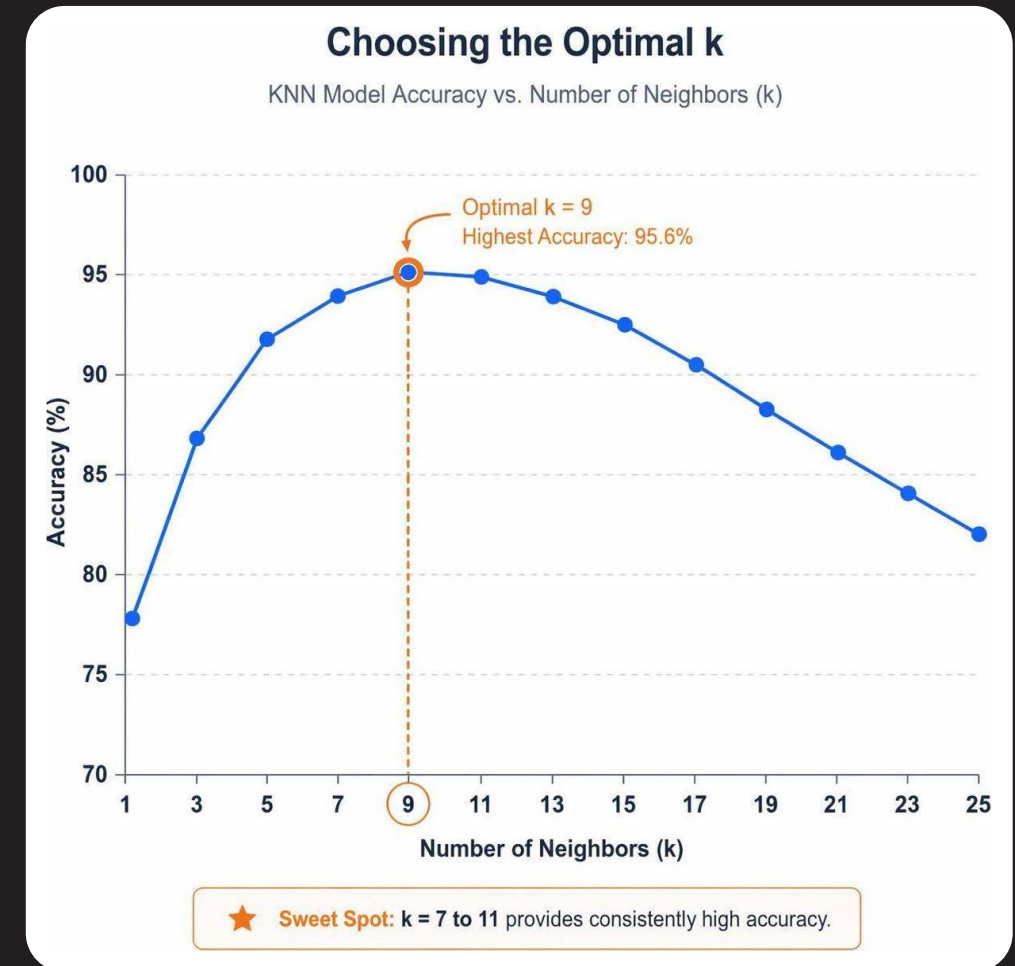
Distância e Escala das Variáveis

- **Distância Euclidiana:** $d(p,q) = \sqrt{\sum (p_i - q_i)^2}$ — a mais comum.
- **Outras métricas:** Manhattan, Minkowski, cosseno — escolha conforme o domínio.
- **Escala importa:** variáveis com magnitudes diferentes dominam a distância.
- **Padronização:** normalize (z-score ou min-max) antes de treinar — passo obrigatório.
- **Atenção:** em alta dimensionalidade, todas as distâncias ficam parecidas (curse of dimensionality).



Escolhendo o Valor de k

- **k pequeno:** modelo sensível a ruído, fronteiras irregulares — risco de overfitting.
- **k grande:** fronteiras mais suaves, mas pode “engolir” classes minoritárias (underfitting).
- **Regra prática:** começar com $k = \sqrt{n}$ e ajustar; preferir valores ímpares em classificação binária.
- **Validação cruzada:** selecione k testando a acurácia em diferentes valores.
- **Ponderação:** vizinhos mais próximos podem receber maior peso (weighted KNN).



Implementação em Python (scikit-learn)

- **Importação:** from sklearn.neighbors
import KNeighborsClassifier.
- **Treino:** modelo =
KNeighborsClassifier(n_neighbors=5);
modelo.fit(X_train, y_train).
- **Predição:** modelo.predict(X_test) retorna
a classe pelos votos dos vizinhos.
- **Pré-processamento:** use StandardScaler
ou MinMaxScaler antes de treinar.
- **Avaliação:** classification_report,
accuracy_score e matriz de confusão do
sklearn.metrics.

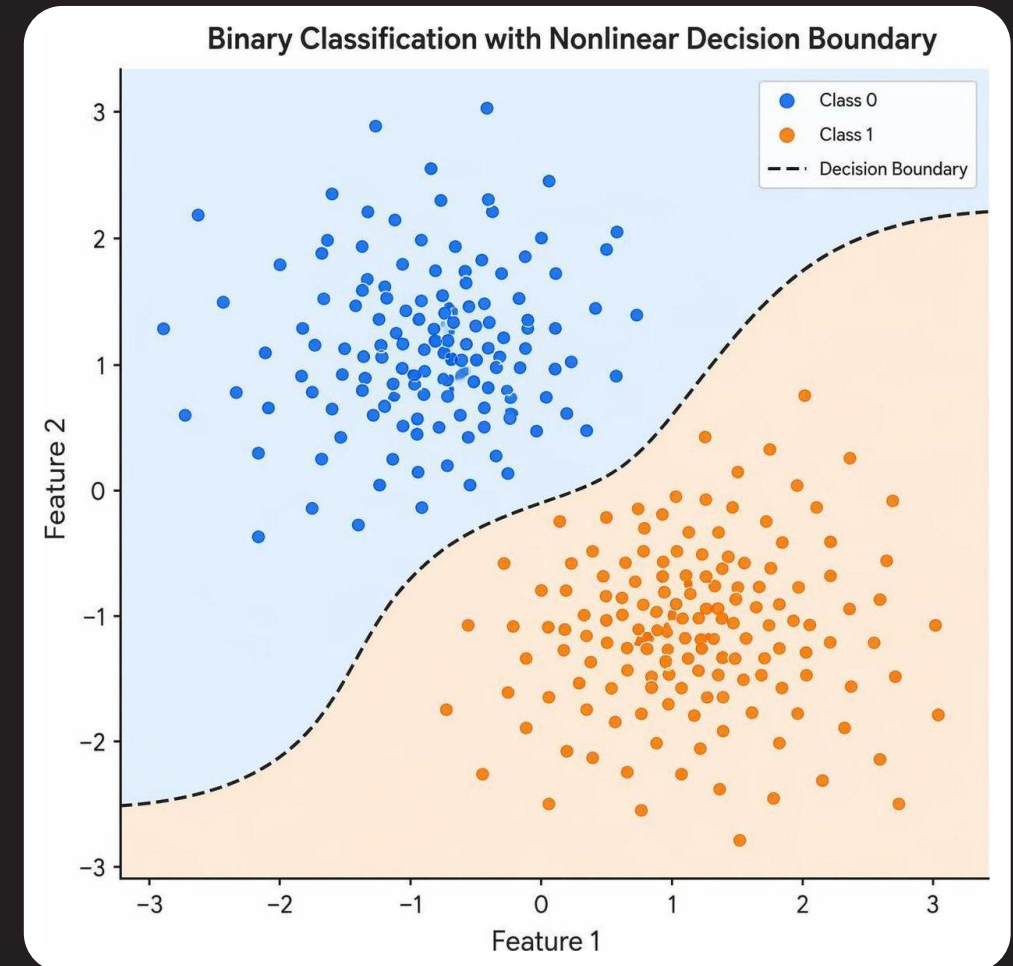
```
knn_example.py x Python ▾ ▶ ⋮  
  
1 # K-Nearest Neighbors (KNN) example with scikit-learn  
2 import numpy as np  
3 from sklearn.datasets import load_iris  
4 from sklearn.model_selection import train_test_split  
5 from sklearn.preprocessing import StandardScaler  
6 from sklearn.neighbors import KNeighborsClassifier  
7 from sklearn.metrics import accuracy_score, classification_report  
8  
9 # 1. Load dataset  
10 iris = load_iris()  
11 X = iris.data  
12 y = iris.target  
13  
14 # 2. Train-test split  
15 X_train, X_test, y_train, y_test = train_test_split(  
16     X, y, test_size=0.2, random_state=42, stratify=y  
17 )  
18  
19 # 3. Feature scaling  
20 scaler = StandardScaler()  
21 X_train_scaled = scaler.fit_transform(X_train)  
22 X_test_scaled = scaler.transform(X_test)  
23  
24 # 4. Train KNN model  
25 knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)  
26 knn.fit(X_train_scaled, y_train)  
27  
28 # 5. Predict and evaluate  
29 y_pred = knn.predict(X_test_scaled)  
30 acc = accuracy_score(y_test, y_pred)  
31 print(f"Accuracy: {acc:.4f}")  
32 print("\nClassification Report:\n")  
33 print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Regressão Logística

Prevendo classes a partir de probabilidades

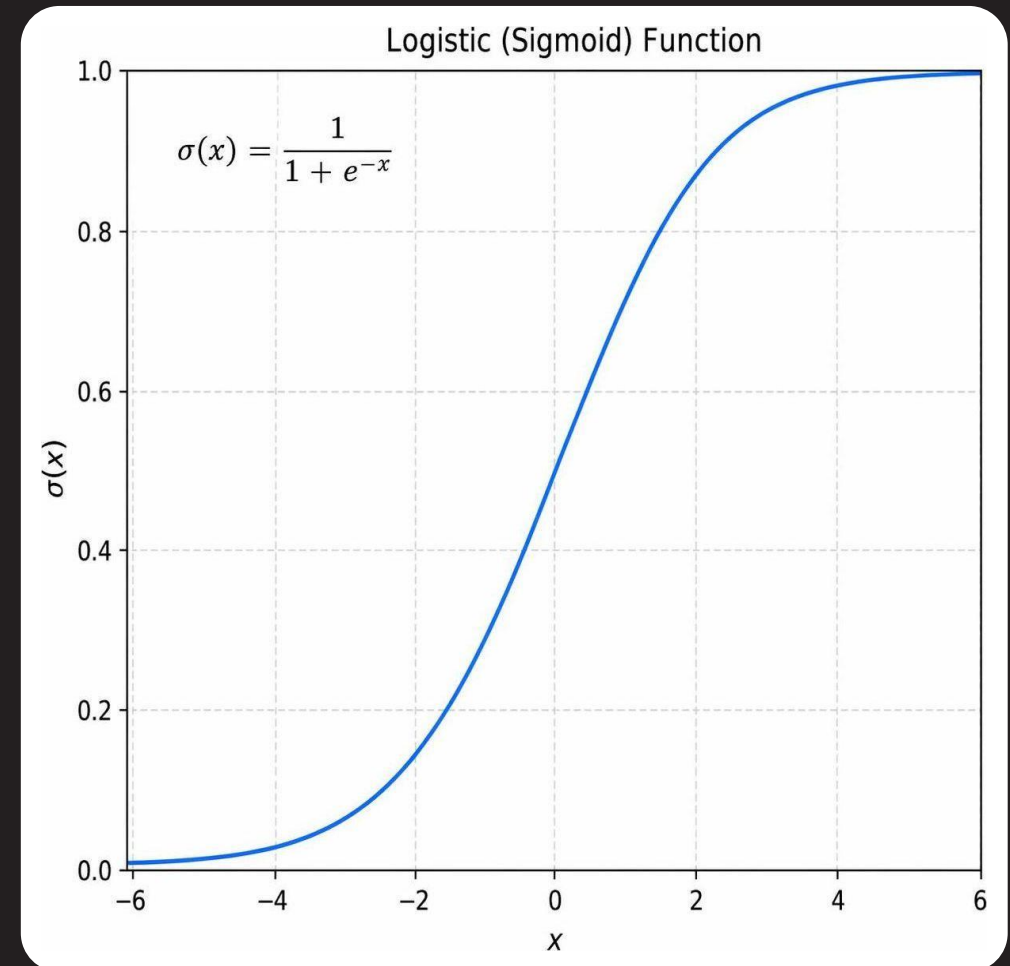
Regressão Logística – Intuição

- **Objetivo:** prever uma classe (sim/não, spam/não-spam) em vez de um valor contínuo.
- **Ideia central:** modelar a probabilidade de pertencer a uma classe, não o valor exato.
- **Diferença para a linear:** a saída é limitada ao intervalo $[0, 1]$ — uma probabilidade.
- **Decisão:** aplica-se um limiar (geralmente 0,5) sobre a probabilidade para escolher a classe.
- **Exemplos:** aprovar ou negar crédito, diagnosticar doença, classificar e-mails.



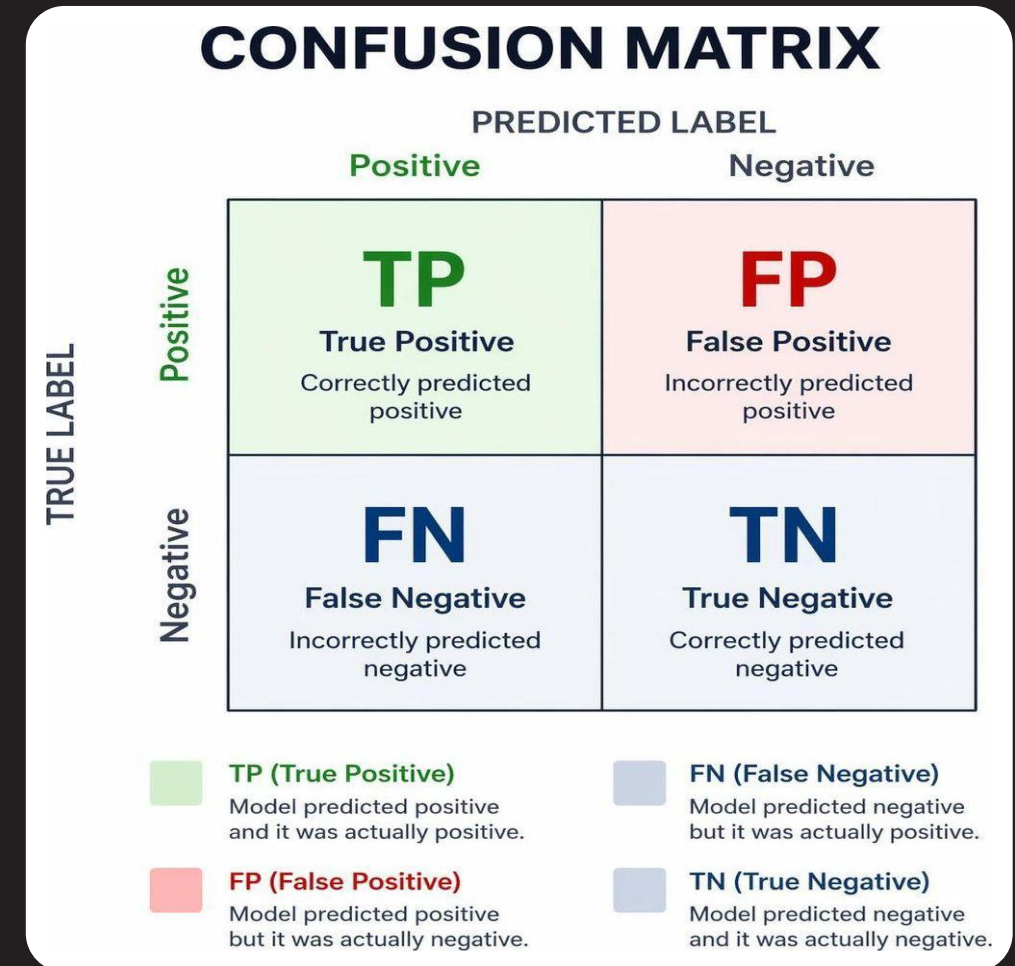
Função Sigmoide e Equação

- **Equação:** $P(Y=1|X) = 1 / (1 + e^{-(\beta_0 + \beta_1 \cdot X)})$.
- **Sigmoide:** transforma qualquer número real em um valor entre 0 e 1.
- **Coeficientes β :** indicam como cada variável aumenta ou diminui a chance da classe positiva.
- **Log-odds:** a relação $\ln(p/(1-p))$ é linear nas variáveis — daí o nome “linear” no log.
- **Treino:** os β são estimados maximizando a verossimilhança dos dados observados.



Avaliação: Matriz de Confusão e Métricas

- **Matriz de confusão:** resume acertos e erros em VP, VN, FP e FN.
- **Acurácia:** $(VP + VN) / \text{total}$ — boa quando as classes são equilibradas.
- **Precisão e Recall:** precisão = $VP / (VP + FP)$; recall = $VP / (VP + FN)$ — críticos em classes desbalanceadas.
- **F1-score:** média harmônica entre precisão e recall, útil quando ambas importam.
- **AUC-ROC:** mede a qualidade do ranqueamento de probabilidades — independente do limiar.



Implementação em Python (scikit-learn)

- **Importação:** `from sklearn.linear_model import LogisticRegression`.
- **Treino:** `modelo = LogisticRegression(); modelo.fit(X_train, y_train)`.
- **Predição:** `modelo.predict(X_test)` retorna a classe; `modelo.predict_proba(X_test)` retorna a probabilidade.
- **Avaliação:** use `classification_report` e `confusion_matrix` do `sklearn.metrics`.
- **Dicas:** padronize as features e ajuste o parâmetro `C` para controlar regularização.

```
1 # Logistic Regression with scikit-learn
2 import numpy as np
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
8
9 # Load dataset
10 # Example: using breast cancer dataset from scikit-learn
11 from sklearn.datasets import load_breast_cancer
12 data = load_breast_cancer(as_frame=True)
13 X = data.frame.drop(columns='target')
14 y = data.frame['target']
15
16 # Train-test split
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42, stratify=y
19 )
20
21 # Feature scaling
22 scaler = StandardScaler()
23 X_train_scaled = scaler.fit_transform(X_train)
24 X_test_scaled = scaler.transform(X_test)
25
26 # Train logistic regression model
27 model = LogisticRegression(max_iter=1000, solver='lbfgs')
28 model.fit(X_train_scaled, y_train)
29
30 # Predictions
31 y_pred = model.predict(X_test_scaled)
32 y_proba = model.predict_proba(X_test_scaled)[:, 1]
33
34 # Evaluation
35 acc = accuracy_score(y_test, y_pred)
36 report = classification_report(y_test, y_pred)
37 cm = confusion_matrix(y_test, y_pred)
38
39 # Results
40 print(f"Accuracy: {acc:.4f}")
41 print("\nClassification Report:\n")
42 print(report)
43 print("Confusion Matrix:\n", cm)
```

SUPPORT VECTOR MACHINE SVM

Find the hyperplane that best separates the classes with maximum margin

CLASS 1

CLASS 2

Support Vector Machine (SVM)

Encontrando a fronteira de margem máxima entre classes

HYPERPLANE

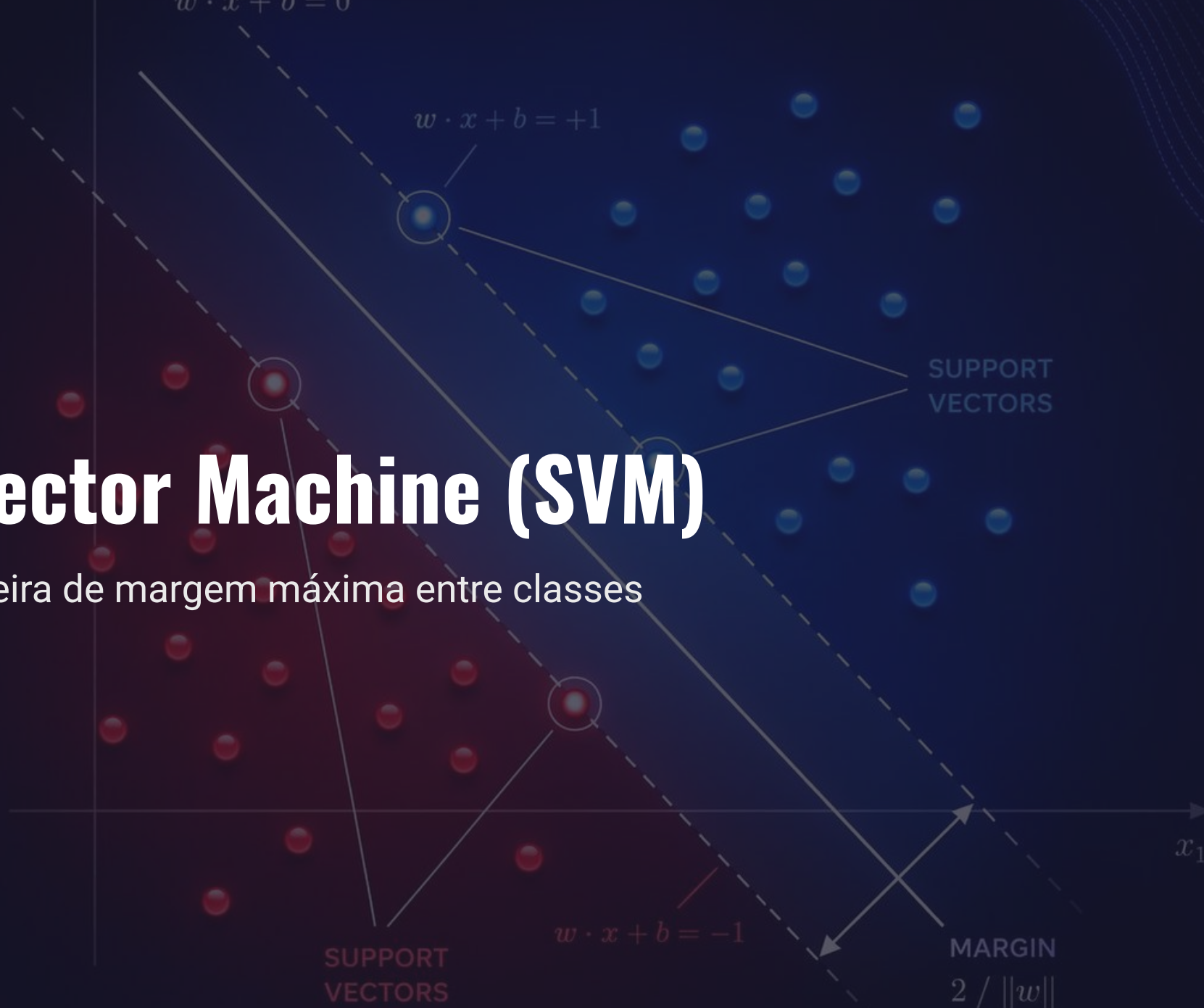
$$w \cdot x + b = 0$$

MARGIN

$$2 / \|w\|$$

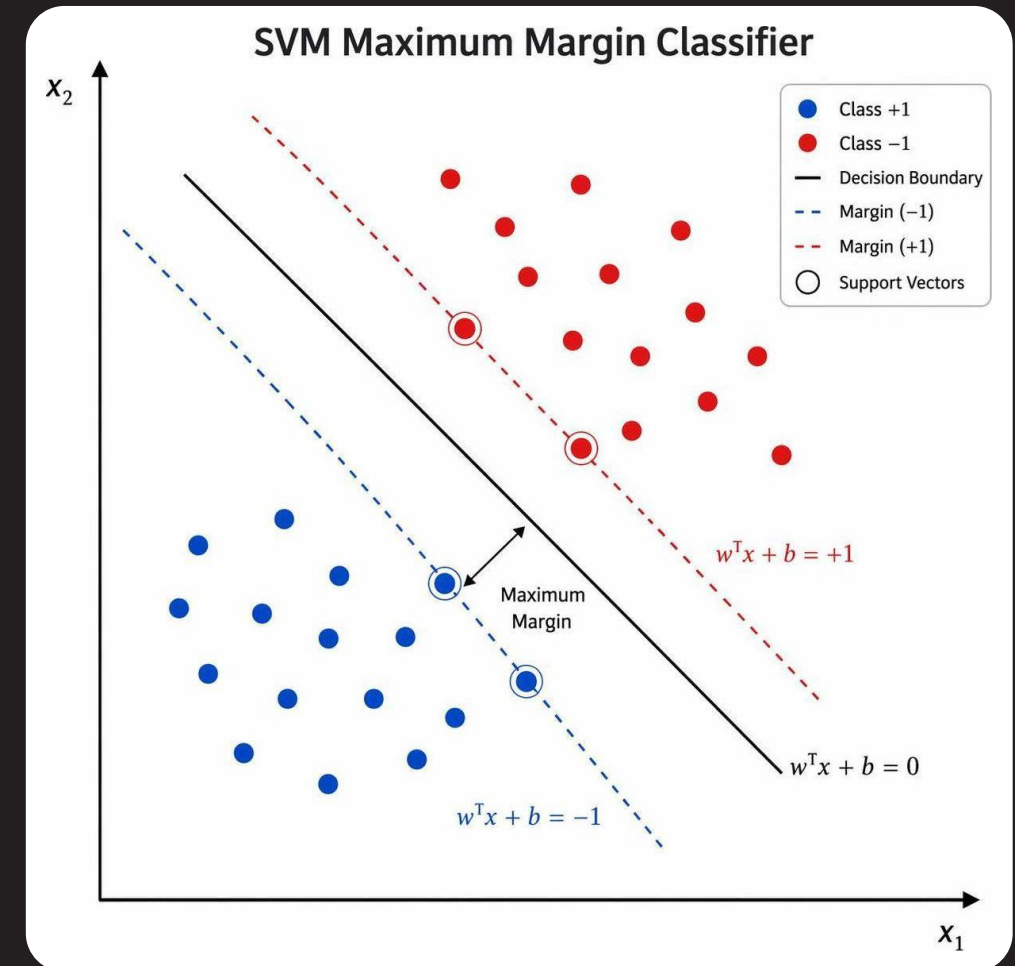
DECISION FUNCTION

$$f(x) = \text{sign}(w \cdot x + b)$$



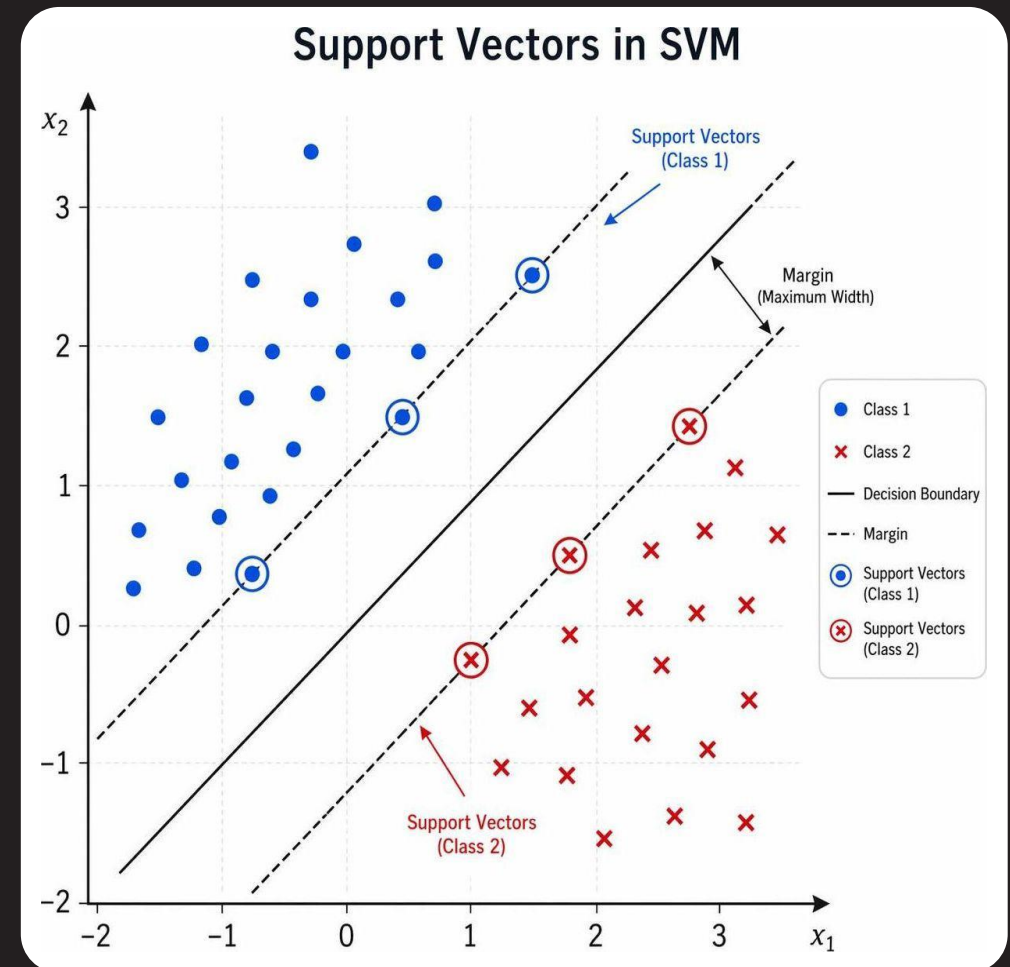
SVM – Intuição

- **Objetivo:** encontrar o hiperplano que melhor separa duas classes.
- **Margem máxima:** a melhor fronteira é a que fica mais distante dos pontos das duas classes.
- **Vetores de suporte:** apenas os pontos mais próximos do hiperplano definem a fronteira.
- **Robustez:** funciona muito bem em alta dimensionalidade e com poucos exemplos.
- **Usos:** classificação de texto, reconhecimento de imagens e bioinformática.



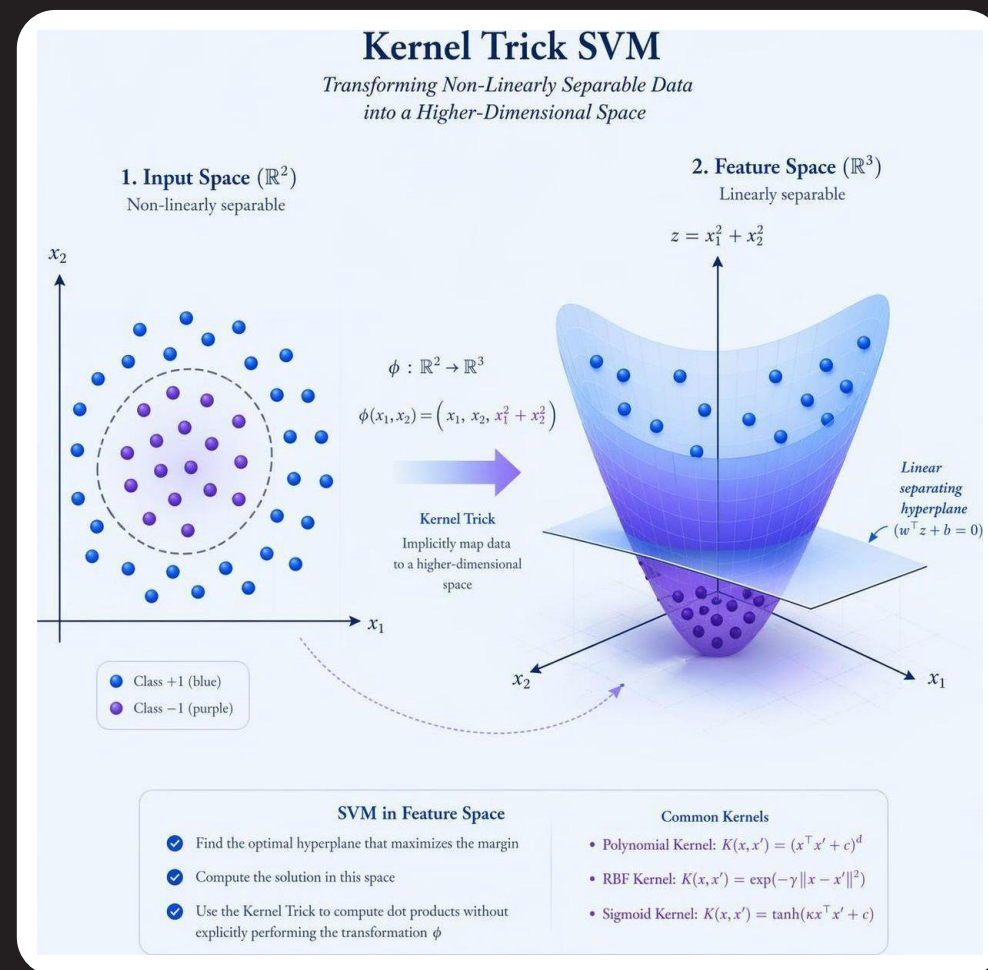
Hiperplano e Vetores de Suporte

- **Hiperplano:** superfície de decisão definida por $w \cdot x + b = 0$.
- **Margem:** distância do hiperplano aos vetores de suporte — deve ser maximizada.
- **Hard margin:** exige separação perfeita; só funciona em dados linearmente separáveis.
- **Soft margin (C):** permite alguns erros; C controla a troca entre margem larga e poucas violações.
- **Otimização:** problema convexo resolvido por programação quadrática — solução única e global.



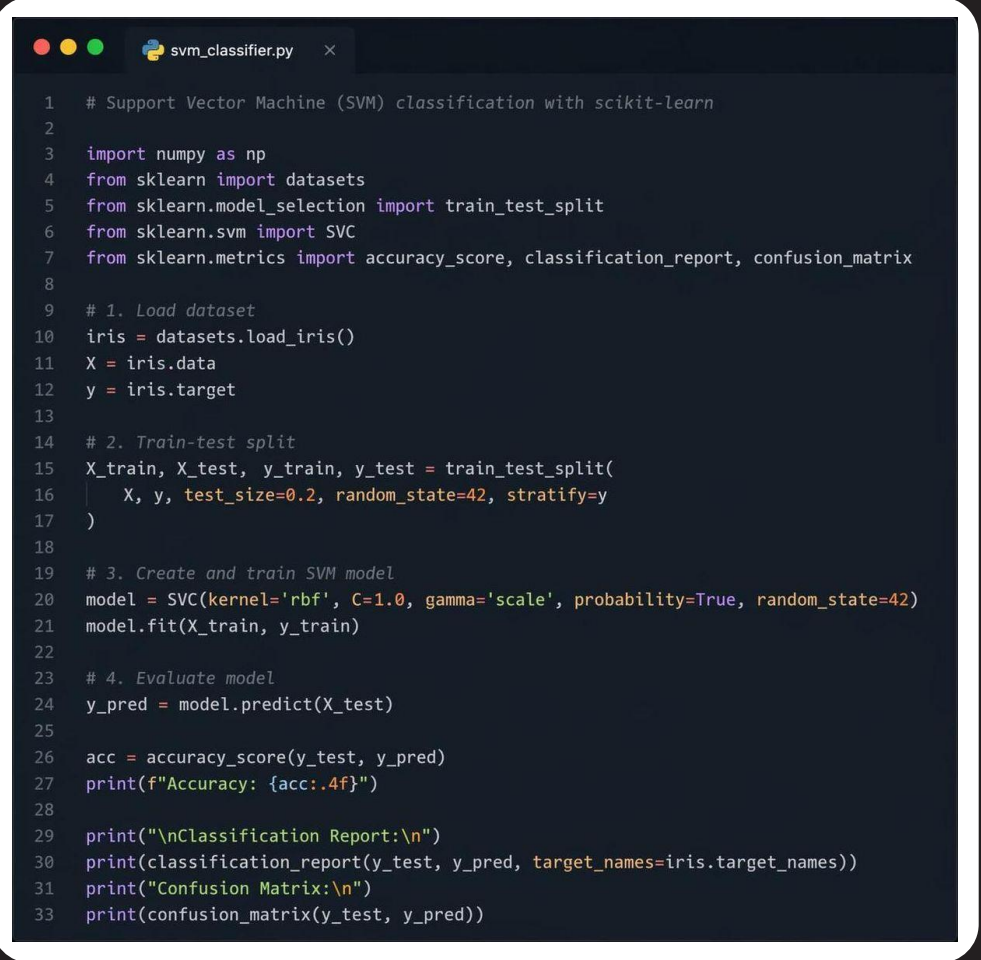
Kernel Trick

- **Problema:** nem todos os dados são linearmente separáveis no espaço original.
- **Ideia:** mapear os dados para um espaço de maior dimensão onde a separação seja possível.
- **Truque:** o kernel calcula esse produto interno sem mapear explicitamente — eficiência.
- **Kernels comuns:** linear, polinomial, RBF (gaussiano) e sigmoide.
- **Hiperparâmetros:** C (regularização) e γ (alcance do kernel RBF) são ajustados por validação cruzada.



Implementação em Python (scikit-learn)

- **Importação:** `from sklearn.svm import SVC`.
- **Treino:** `modelo = SVC(kernel="rbf", C=1.0, gamma="scale"); modelo.fit(X_train, y_train)`.
- **Predição:** `modelo.predict(X_test)` retorna a classe prevista.
- **Pré-processamento:** padronize as features — SVM é sensível à escala.
- **Avaliação:** `classification_report` e `GridSearchCV` para ajustar C, gamma e kernel.

A screenshot of a code editor window titled 'svm_classifier.py'. The code is a Python script for SVM classification using scikit-learn. It includes comments for each step: loading the dataset, splitting it into training and testing sets, creating and training the SVM model, and evaluating the model. The script uses the iris dataset, splits it with a test size of 0.2, and prints the accuracy, classification report, and confusion matrix.

```
1 # Support Vector Machine (SVM) classification with scikit-learn
2
3 import numpy as np
4 from sklearn import datasets
5 from sklearn.model_selection import train_test_split
6 from sklearn.svm import SVC
7 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
8
9 # 1. Load dataset
10 iris = datasets.load_iris()
11 X = iris.data
12 y = iris.target
13
14 # 2. Train-test split
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, random_state=42, stratify=y
17 )
18
19 # 3. Create and train SVM model
20 model = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
21 model.fit(X_train, y_train)
22
23 # 4. Evaluate model
24 y_pred = model.predict(X_test)
25
26 acc = accuracy_score(y_test, y_pred)
27 print(f"Accuracy: {acc:.4f}")
28
29 print("\nClassification Report:\n")
30 print(classification_report(y_test, y_pred, target_names=iris.target_names))
31 print("Confusion Matrix:\n")
32 print(confusion_matrix(y_test, y_pred))
```

ROOT NODE
Entire dataset

DECISION NODE
Feature-based split

LEAF NODE
Final prediction

ROOT NODE
All Data

```
if condition_A:  
    return class_A  
else:  
    return class_B  
else:  
    return class_C
```

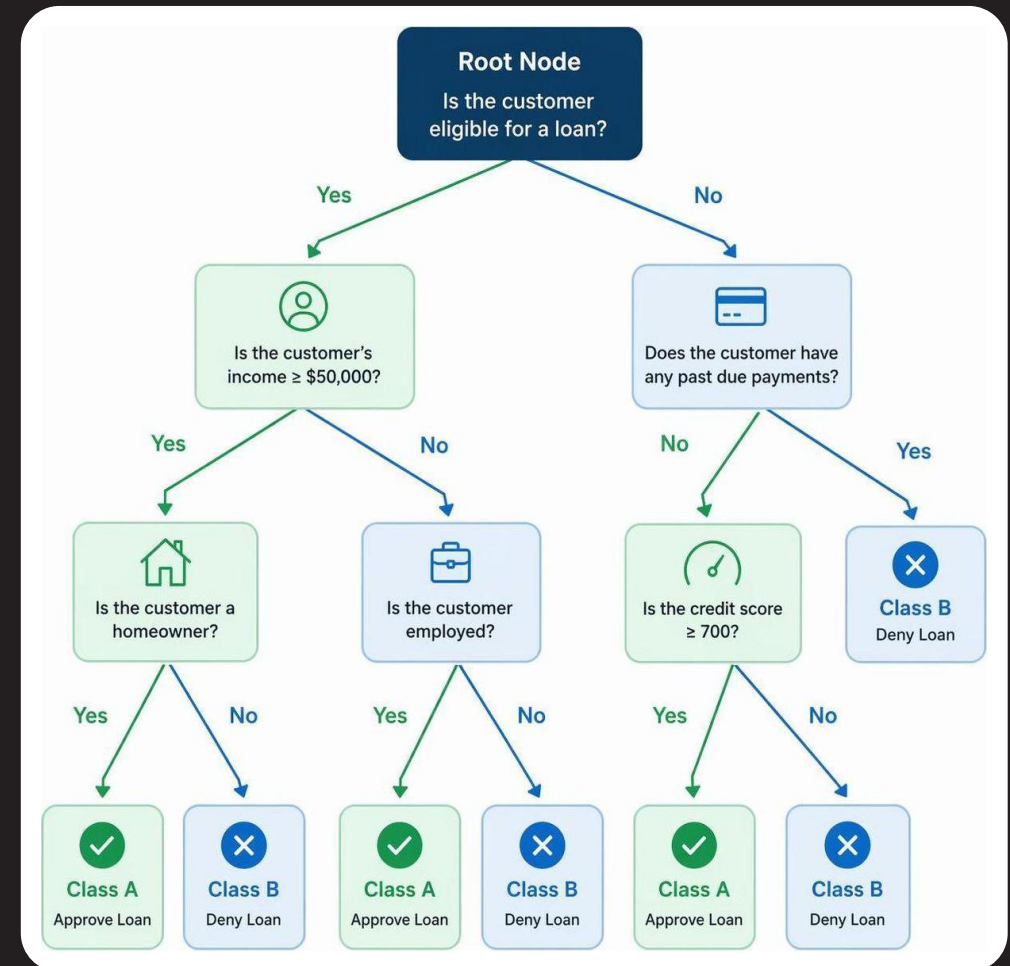
Árvores de Decisão

Decisões em cascata: simples, interpretáveis e poderosas



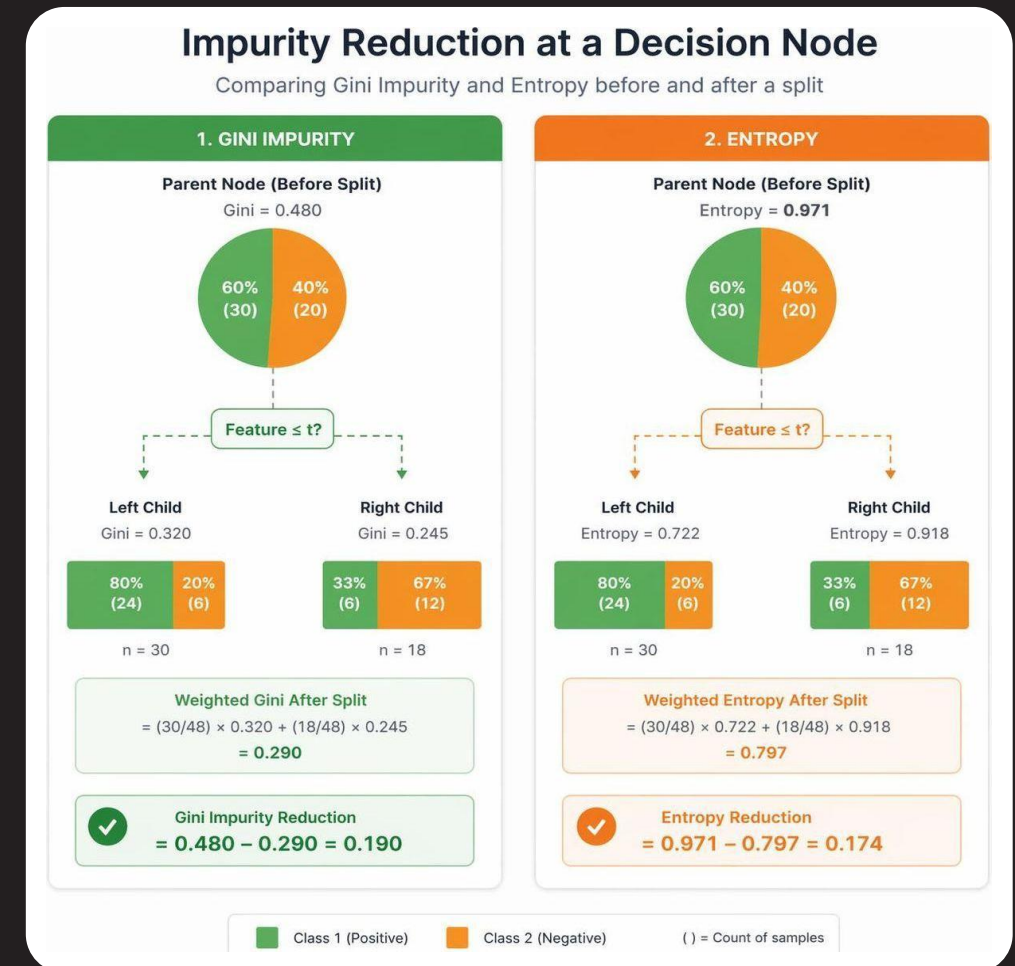
Árvores de Decisão – Intuição

- **Ideia central:** fazer perguntas sucessivas sobre os atributos para chegar à decisão final.
- **Estrutura:** nó raiz, nós internos (divisões) e folhas (classes ou valores previstos).
- **Interpretabilidade:** fácil de visualizar e explicar para pessoas não técnicas.
- **Funciona para:** classificação e regressão, com features numéricas ou categóricas.
- **Exemplos:** aprovação de crédito, diagnóstico médico, segmentação de clientes.



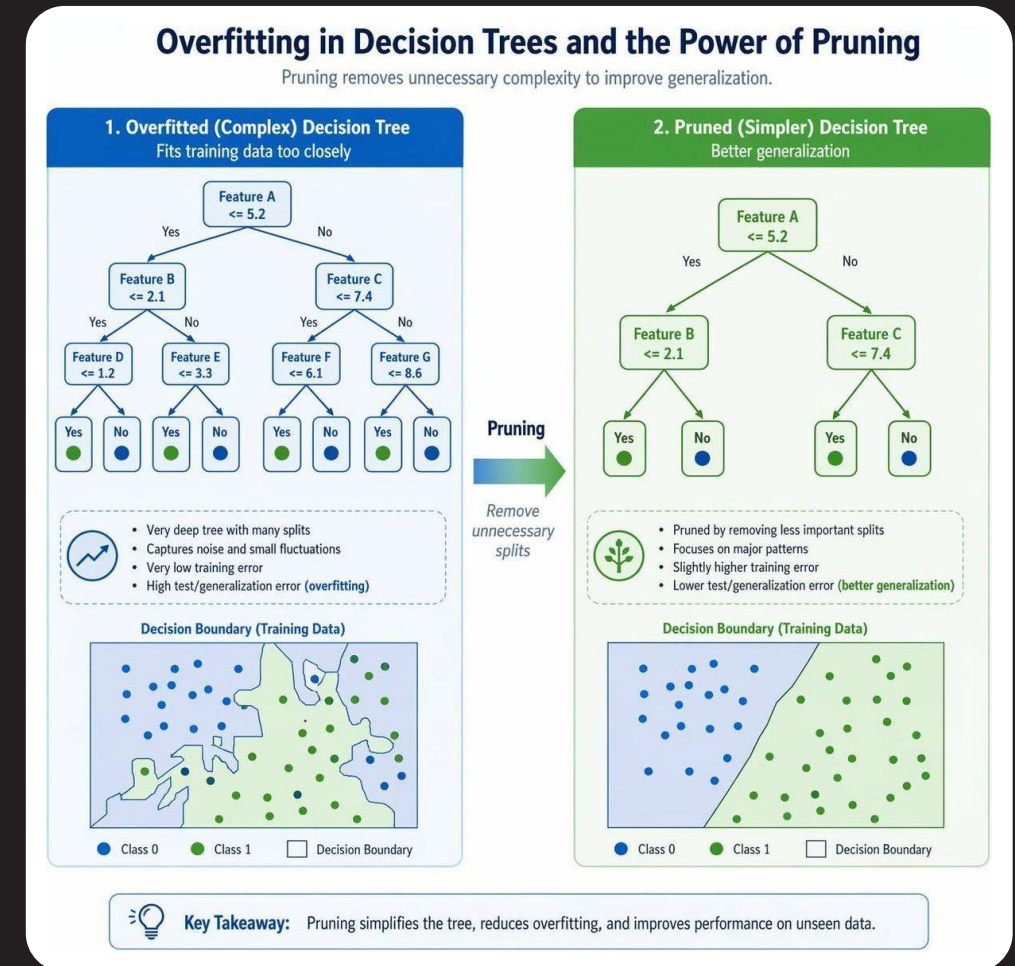
Como a Árvore Decide as Divisões

- **Critério:** a cada nó, busca-se a divisão que mais reduz a impureza.
- **Gini:** mede a probabilidade de classificar errado um exemplo aleatório do nó.
- **Entropia:** mede a desordem da distribuição das classes — usada no ganho de informação.
- **Regressão:** minimiza a variância (MSE) dos valores em cada nó.
- **Greedy:** a cada passo escolhe a melhor divisão local — não garante a árvore ótima global.



Overfitting e Poda

- **Problema:** árvores muito profundas decoram o treino e generalizam mal.
- **Profundidade máxima:** limitar `max_depth` controla a complexidade e o overfitting.
- **Mínimo por nó:** exigir `min_samples_split` ou `min_samples_leaf` evita divisões irrelevantes.
- **Poda:** técnicas pós-treino (cost-complexity pruning) reduzem a árvore mantendo desempenho.
- **Conjunto:** Random Forest e Gradient Boosting combinam várias árvores para reduzir variância.



Implementação em Python (scikit-learn)

- **Importação:** from sklearn.tree import DecisionTreeClassifier.
- **Treino:** modelo = DecisionTreeClassifier(max_depth=5); modelo.fit(X_train, y_train).
- **Predição:** modelo.predict(X_test) retorna a classe; predict_proba estima probabilidades.
- **Visualização:** use plot_tree ou export_graphviz para inspecionar a árvore treinada.
- **Avaliação:** classification_report, feature_importances_ e validação cruzada do sklearn.

```
decision_tree_example.py x
1 # Decision Tree example using scikit-learn
2 import numpy as np
3 from sklearn.tree import DecisionTreeClassifier, plot_tree
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, classification_report
6
7 # Sample dataset: Iris
8 from sklearn.datasets import load_iris
9
10 # Load data
11 data = load_iris()
12 X = data.data
13 y = data.target
14 feature_names = data.feature_names
15 target_names = data.target_names
16
17 # Train-test split
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42, stratify=y
20 )
21
22 # Initialize and train the Decision Tree model
23 clf = DecisionTreeClassifier(random_state=42, criterion='gini', max_depth=3)
24 clf.fit(X_train, y_train)
25
26 # Predictions
27 y_pred = clf.predict(X_test)
28
29 # Evaluation
30 acc = accuracy_score(y_test, y_pred)
31 report = classification_report(y_test, y_pred, target_names=target_names)
32 print(f"Accuracy: {acc:.4f}\n")
33 print(report)
34
35 # Visualize the tree
36 import matplotlib.pyplot as plt
37
38 plt.figure(figsize=(12, 6))
39 plot_tree(
40     clf,
41     feature_names=feature_names,
42     class_names=target_names,
43     filled=True,
44     rounded=True,
45     impurity=True
46 )
47 plt.title("Decision Tree (Iris Dataset)")
48 plt.show()
```

MACHINE LEARNING REGRESSION

MODELING CONTINUOUS OUTCOMES

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

- DATA POINTS
- PREDICTED TREND
- CONFIDENCE INTERVAL

Aprendizado de Máquina – Regressão

Algoritmos para prever valores numéricos contínuos

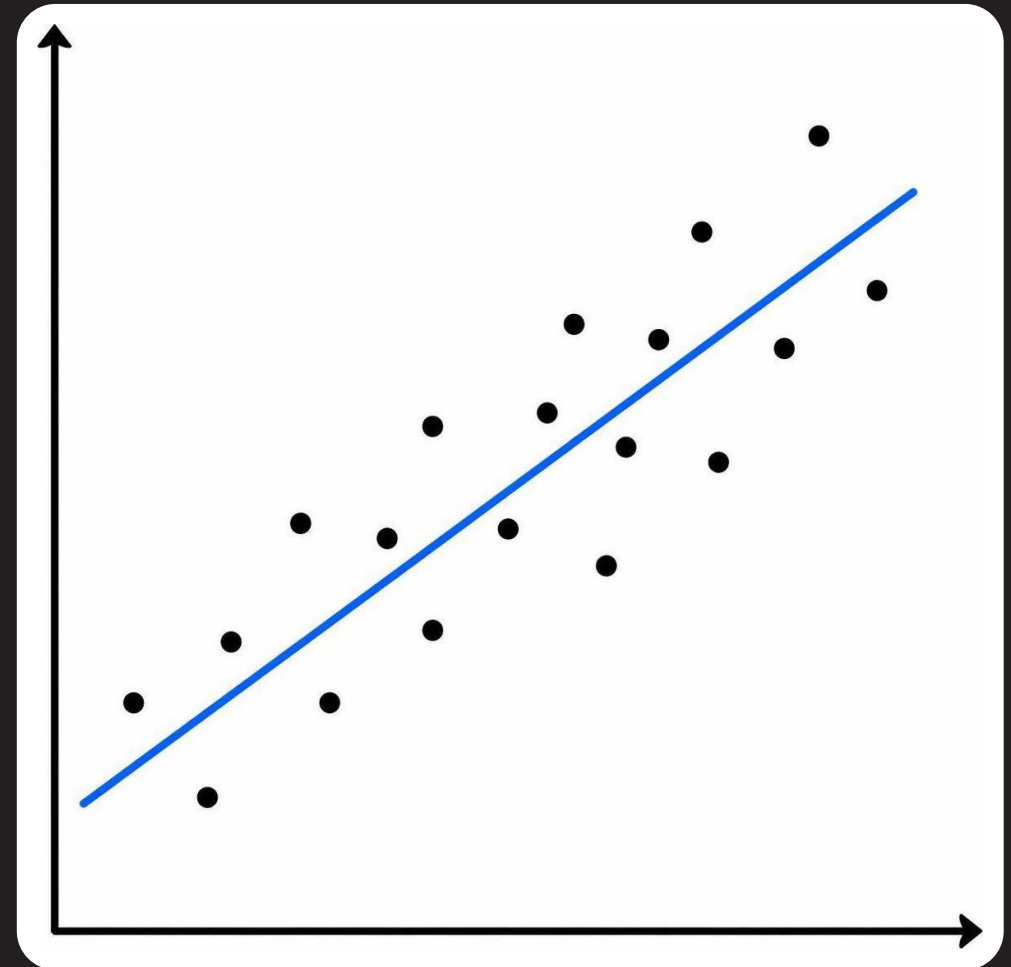


Regressão Linear

Modelando relações entre variáveis a partir de dados

Regressão Linear – Intuição

- **Objetivo:** prever um valor numérico contínuo (Y) a partir de uma ou mais variáveis explicativas (X);
- Ideia central: encontrar a “melhor reta” que descreve a tendência dos dados;
- **Equação:** $Y = \beta_0 + \beta_1 \cdot X + \varepsilon$;
- β_0 é o intercepto (onde a reta cruza Y) e β_1 é a inclinação (quanto Y varia para cada unidade de X);
- ε representa o erro: tudo o que a reta não consegue explicar.



Exemplo Guiado: Horas de Estudo vs Nota

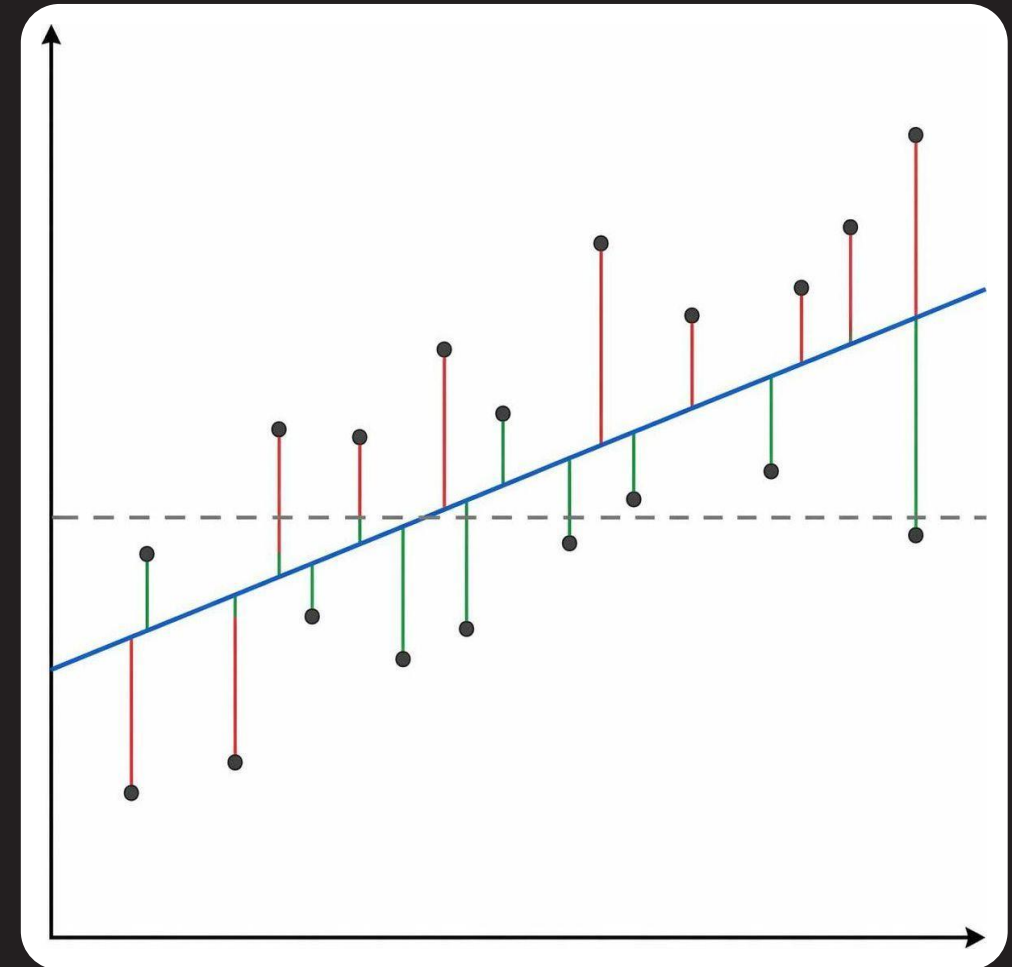
- **Dataset:** 5 alunos com (x = horas, y = nota) — (1, 2), (2, 4), (3, 5), (4, 4), (5, 5);
- **Objetivo:** achar a reta $\hat{y} = \beta_0 + \beta_1 \cdot x$ que melhor prevê a nota a partir das horas;
- **Médias:** $\bar{x} = (1+2+3+4+5)/5 = 3$ e $\bar{y} = (2+4+5+4+5)/5 = 4$;
- Essas médias ancoram a reta — ela sempre passa pelo ponto $(\bar{x}, \bar{y})$;
- Nos próximos slides, vamos calcular β_1 , β_0 e o R^2 com esses dados.

Passo a Passo: Calculando β_0 e β_1

- **Desvios:** $(x_i - \bar{x}) = (-2, -1, 0, 1, 2)$ e $(y_i - \bar{y}) = (-2, 0, 1, 0, 1)$;
- **Numerador:** $\sum (x_i - \bar{x})(y_i - \bar{y}) = 4 + 0 + 0 + 0 + 2 = 6$;
- **Denominador:** $\sum (x_i - \bar{x})^2 = 4 + 1 + 0 + 1 + 4 = 10$;
- $\beta_1 = 6/10 = 0,6 \rightarrow$ cada hora extra de estudo soma 0,6 ponto à nota;
- $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x} = 4 - 0,6 \cdot 3 = 2,2 \rightarrow$ reta final: $\hat{y} = 2,2 + 0,6 \cdot x$.

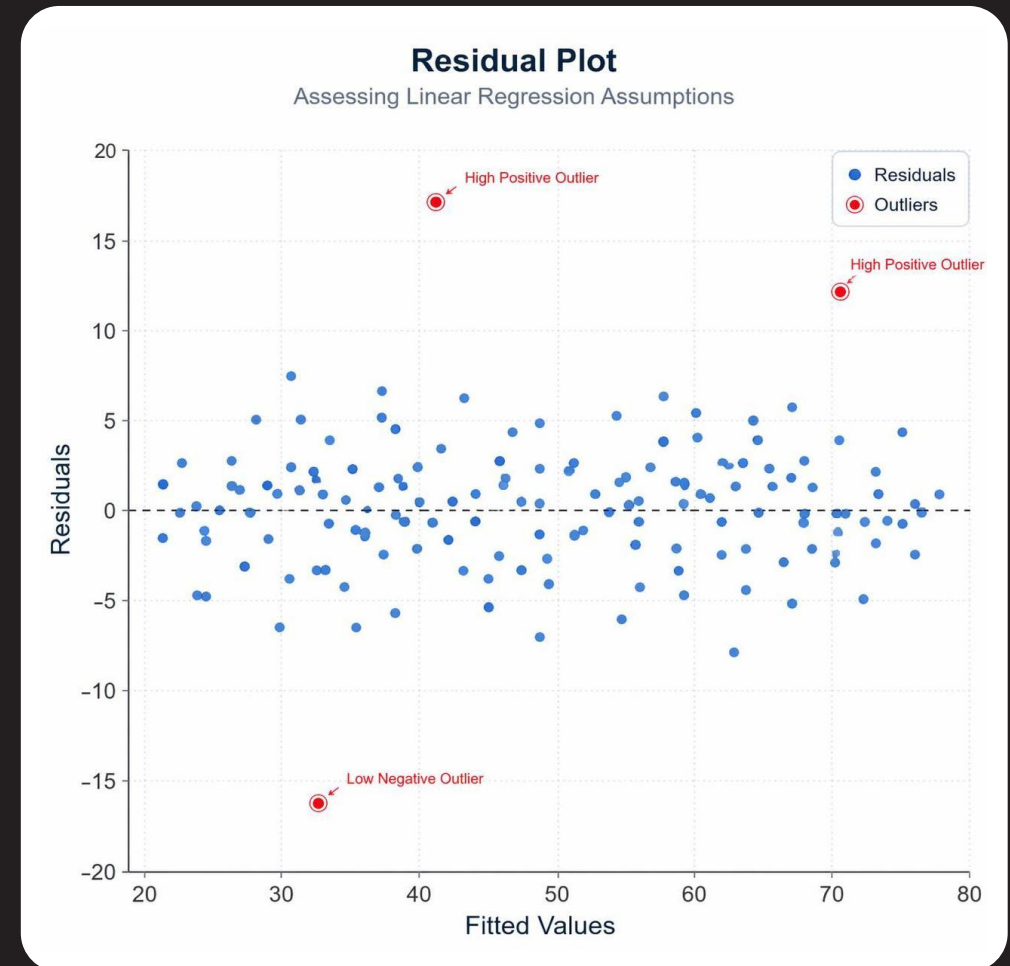
R^2 – Conceito e Fórmula

- **R^2 (coeficiente de determinação):** fração da variação de Y explicada pelo modelo, com valor entre 0 e 1.
- **Fórmula:** $R^2 = 1 - \text{SQR} / \text{SQT}$.
- **Componentes:** $\text{SQR} = \sum (y_i - \hat{y}_i)^2$ mede o erro do modelo; $\text{SQT} = \sum (y_i - \bar{y})^2$ mede a variação total de Y.
- **Leitura prática:** $R^2 = 0,75$ significa que 75% da variabilidade de Y é capturada pelo modelo.



R² – Cuidados na Prática

- **R² alto ≠ modelo correto:** sempre analise os resíduos antes de aceitar o ajuste.
- **Resíduos e outliers:** padrões nos resíduos indicam viés; outliers podem inflar ou distorcer o R².
- **R² ajustado:** em regressão múltipla, prefira o R² ajustado, que penaliza variáveis irrelevantes.
- **Validação:** compare o R² em treino e teste para detectar overfitting.
- **Boa prática:** combine R² com métricas como MAE, RMSE e gráficos diagnósticos.



Avaliando o Exemplo: Calculando o R^2

- **Previsões:** $\hat{y} = 2,2 + 0,6 \cdot x \rightarrow (2,8; 3,4; 4,0; 4,6; 5,2);$
- **Resíduos ($y_i - \hat{y}_i$):** $(-0,8; 0,6; 1,0; -0,6; -0,2);$
- **SQR:** $\sum (y_i - \hat{y}_i)^2 = 0,64 + 0,36 + 1,00 + 0,36 + 0,04 = 2,4;$
- **SQT:** $\sum (y_i - \bar{y})^2 = 4 + 0 + 1 + 0 + 1 = 6;$
- **$R^2 = 1 - 2,4/6 = 0,60$** $\rightarrow$ o modelo explica 60% da variação das notas.

Implementação em Python (scikit-learn)

- Reproduzindo o exemplo das notas com poucas linhas de código:

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2, 4, 5, 4, 5])
model = LinearRegression().fit(X, y)
print(model.intercept_, model.coef_[0])    # 2.2  0.6
print(model.score(X, y))                   # R² = 0.60
```

Aplicações Reais e Pegadinhas

- **Exemplos:** preço de imóveis, vendas vs. média, IMC vs. pressão, horas de estudo vs. nota;
- **Pressupostos:** linearidade, independência, homocedasticidade e resíduos normais;
- **Pegadinhas:** correlação $\neq$ causalidade, multicolinearidade e outliers que distorcem a reta;
- **Na prática:** scikit-learn (LinearRegression) ou statsmodels (OLS);
- **A seguir:** regressão múltipla, Ridge/Lasso e modelos não-lineares.

MACHINE LEARNING REGRESSION

MODELING CONTINUOUS OUTCOMES

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

- DATA POINTS
- PREDICTED TREND
- CONFIDENCE INTERVAL

Aprendizado de Máquina – Não Supervisionado

Descobrendo padrões e estruturas escondidas nos dados — sem rótulos



O que é Clustering?

- **Definição:** agrupar dados não rotulados em conjuntos (clusters) onde pontos do mesmo grupo são mais semelhantes entre si do que com pontos de outros grupos.
- **Não supervisionado:** não usa rótulos prévios — o algoritmo descobre a estrutura natural dos dados.
- **Aplicações:** segmentação de clientes, análise de cesta de compras, segmentação de imagens, detecção de anomalias e redes sociais.
- **Famílias principais:** baseadas em centróides (K-Means), em densidade (DBSCAN) e hierárquicas (aglomerativa/divisiva).
- **Desafio central:** definir o que é “semelhante” (métrica de distância) e quantos grupos existem nos dados.

MACHINE LEARNING REGRESSION

MODELING CONTINUOUS OUTCOMES

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

- DATA POINTS
- PREDICTED TREND
- CONFIDENCE INTERVAL

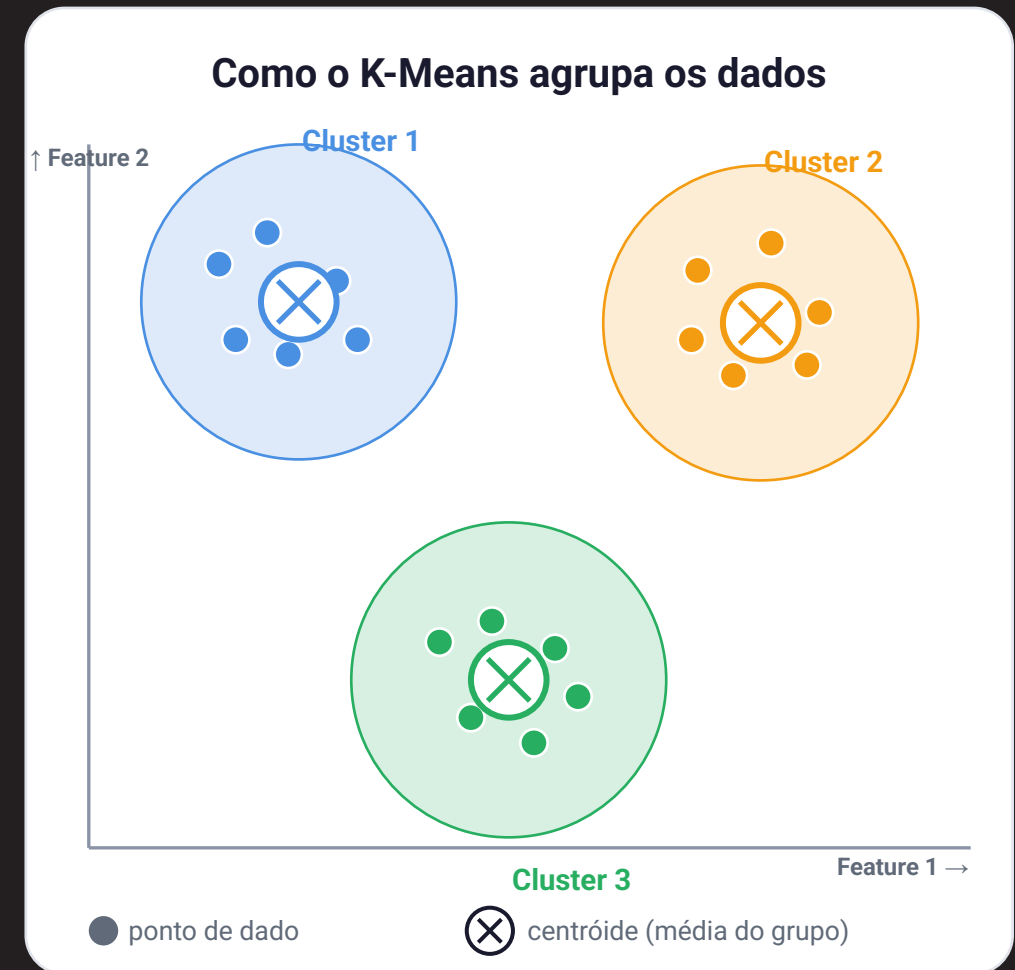
K-Means Clustering

O algoritmo de clustering mais clássico — particionando dados em K grupos



K-Means – Intuição

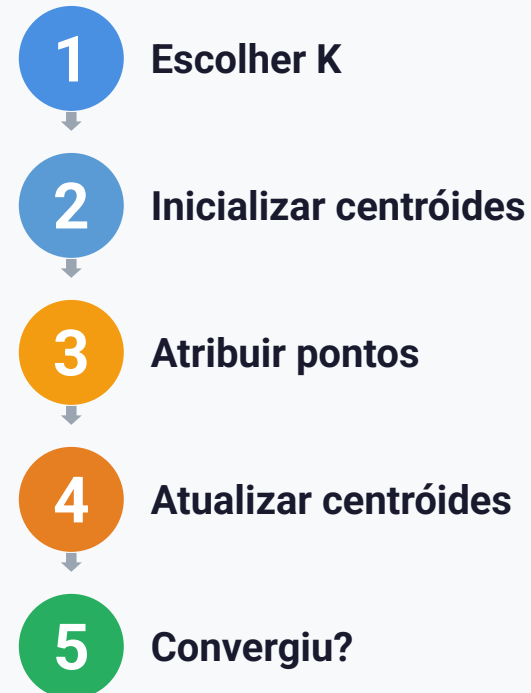
- **Ideia central:** particionar os dados em K grupos, onde cada ponto pertence ao cluster com a média (centróide) mais próxima.
- **Centróide:** ponto que representa o "centro" de um grupo — média das coordenadas dos seus membros.
- **Métrica:** usa distância euclidiana por padrão; assume clusters aproximadamente esféricos e de tamanho similar.
- **Analogia:** imagine plantar K bandeirinhas em um mapa e puxá-las iterativamente para o centro dos pontos mais próximos.
- **Não supervisionado:** não precisa de rótulos — só dos dados e do valor de K escolhido antecipadamente.



Algoritmo Passo a Passo

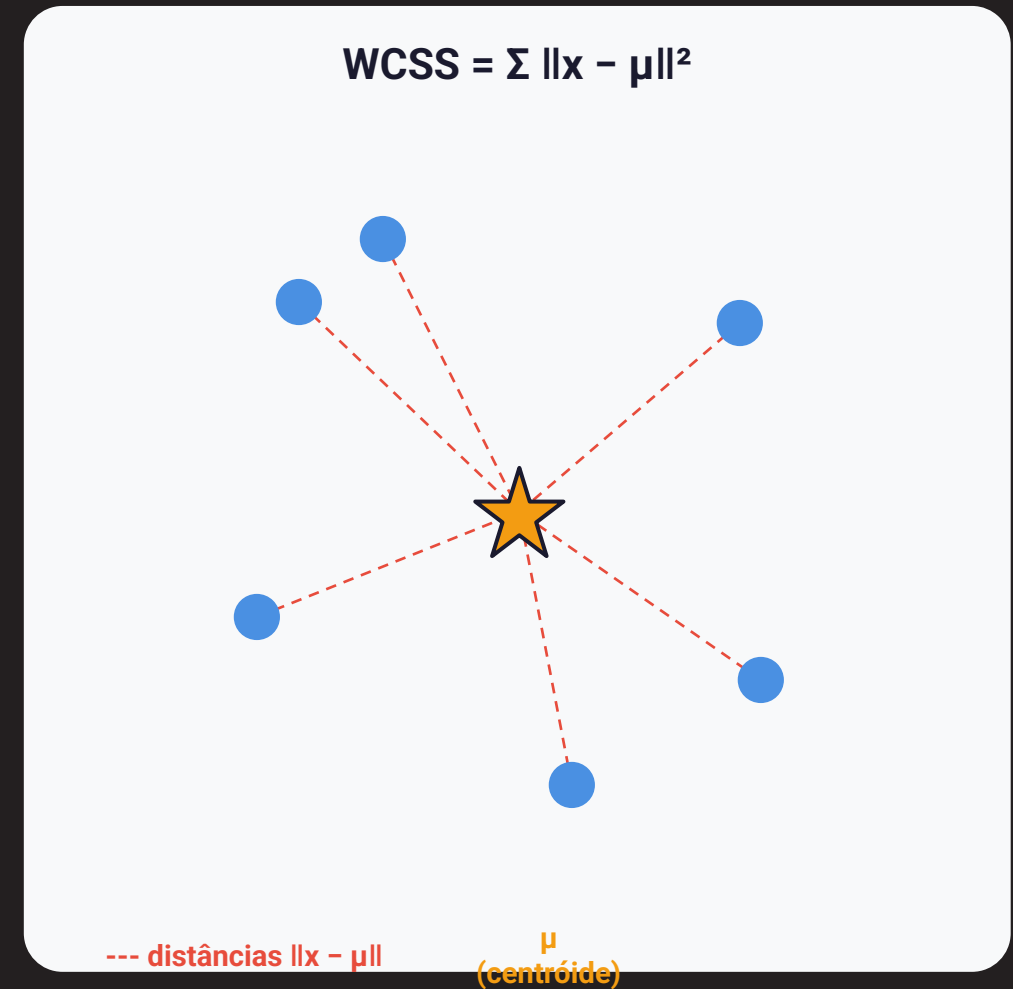
- **1. Escolher K:** definir o número de clusters desejado.
- **2. Inicializar centróides:** posicionar K centróides aleatoriamente (ou via K-Means++ para melhor convergência).
- **3. Atribuição:** associar cada ponto ao centróide mais próximo, formando K clusters.
- **4. Atualização:** recalcular cada centróide como a média dos pontos do seu cluster.
- **5. Repetir:** iterar 3 e 4 até a convergência — quando os centróides param de se mover (ou atingir max_iter).

Fluxo iterativo do K-Means



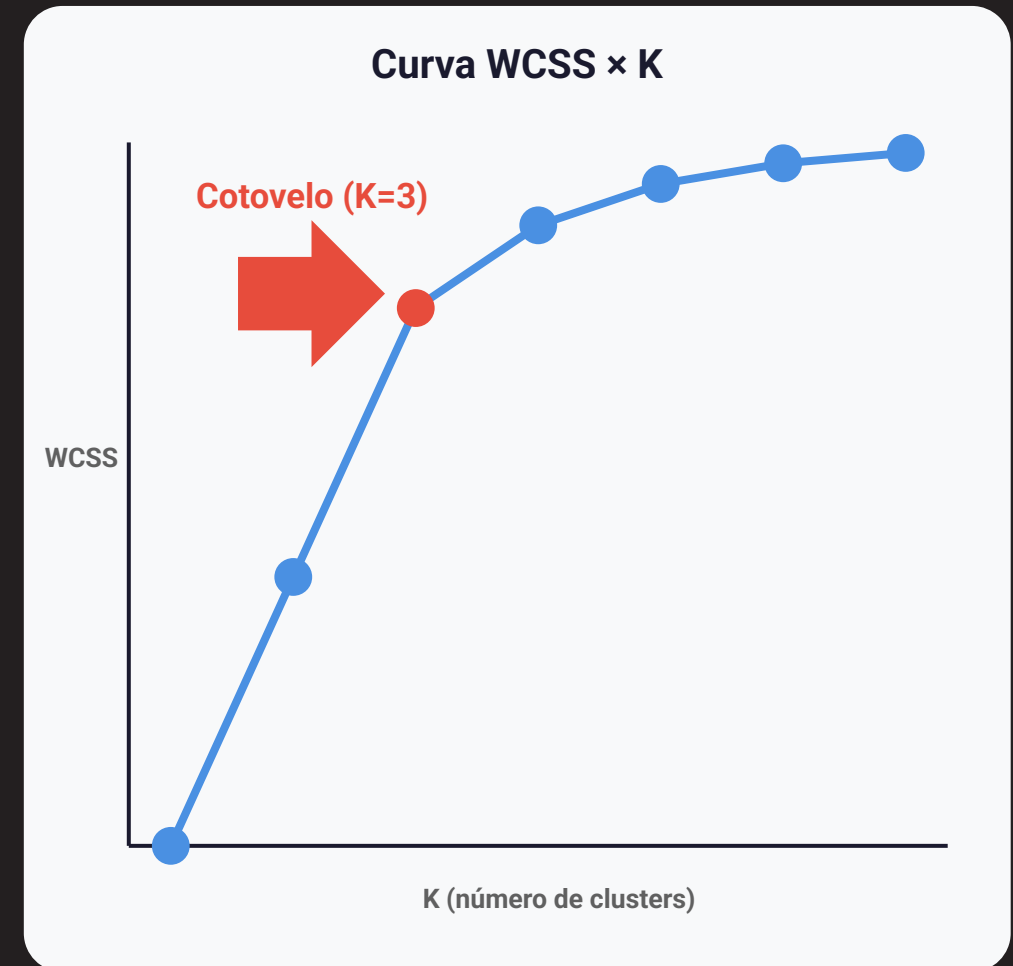
Função Objetivo – WCSS

- **WCSS:** Within-Cluster Sum of Squares — soma dos quadrados das distâncias de cada ponto ao centróide do seu cluster.
- **Fórmula:** $WCSS = \sum_i \sum_{\{x \in C_i\}} \|x - \mu_i\|^2$, onde μ_i é o centróide do cluster i .
- **Objetivo:** o K-Means minimiza o WCSS — clusters mais compactos = menor WCSS.
- **Inércia:** termo equivalente usado no scikit-learn (`kmeans.inertia_`).
- **Convergência:** o algoritmo é guloso e garante um mínimo local — diferentes inicializações podem gerar resultados distintos.



Escolha do K – Método do Cotovelo

- **Problema:** K-Means exige definir K antes — mas qual valor é o ideal?
- **Ideia:** executar o K-Means para vários K (ex.: 1 a 10) e calcular o WCSS de cada um.
- **Gráfico:** plotar K (eixo x) versus WCSS (eixo y) — a curva sempre decresce conforme K aumenta.
- **Cotovelo:** identificar o ponto em que a queda do WCSS desacelera bruscamente — esse "cotovelo" sugere o K ótimo.
- **Limitação:** o cotovelo nem sempre é claro — em dados ruidosos a curva é suave e a leitura fica subjetiva.



Escolha do K – Silhouette Score

- **Definição:** mede o quão parecido um ponto é com seu próprio cluster comparado aos outros clusters.
- **Fórmula:** $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$, com valor entre -1 e 1.
- **Interpretação:** valores próximos de 1 indicam clusters bem separados; próximos de 0 indicam sobreposição; negativos indicam atribuição errada.
- **Escolha do K:** rodar para vários K e escolher o que maximiza a silhueta média.
- **Comparação:** costuma ser mais robusta que o cotovelo quando a curva do WCSS não tem um ponto de inflexão claro.

Escala da Silhueta



$$s(i) = (b - a) / \max(a, b)$$

a = distância média ao próprio cluster
b = distância média ao cluster vizinho mais próximo
Maior s ⇒ melhor separação

Vantagens, Desvantagens e Comparação

- **Vantagens:** simples, rápido, escalável para grandes datasets e fácil de interpretar.
- **Desvantagens:** exige K prévio, sensível a outliers e à escala das features, assume clusters esféricos de tamanho similar.
- **Inicialização:** usar K-Means++ (padrão no scikit-learn) reduz o impacto da escolha inicial dos centróides.
- **Pré-processamento:** padronize as features (StandardScaler) — o algoritmo é baseado em distância.
- **Vs. DBSCAN:** DBSCAN encontra clusters de forma arbitrária e detecta ruído, mas é mais sensível aos hiperparâmetros (eps, minPts).
- **Vs. Hierárquico:** clustering hierárquico gera um dendrograma e não exige K prévio, mas tem custo $O(n^2)$ ou maior.

Comparação de Algoritmos

K-Means

Centróides • exige K • rápido • clusters esféricos

DBSCAN

Densidade • detecta ruído • sem K prévio • formas arbitrárias

Hierárquico

Dendrograma • sem K prévio • custo $O(n^2)$ • visual intuitivo

Implementação em Python e Aplicações

- **Importação:** `from sklearn.cluster import KMeans`.
- **Treino:** `modelo = KMeans(n_clusters=3, init="k-means++", n_init=10, random_state=42); modelo.fit(X)`.
- **Resultados:** `modelo.labels_` (cluster de cada ponto), `modelo.cluster_centers_` (centróides) e `modelo.inertia_` (WCSS).
- **Avaliação:** `silhouette_score(X, labels)` e visualização com PCA/t-SNE para dados de alta dimensão.
- **Aplicações reais:** segmentação de clientes em marketing, agrupamento de regimes de mercado em trading, compressão de imagens, detecção de anomalias e organização de documentos.

```
# Python (scikit-learn)
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3,
            init='k-means++',
            random_state=42)
km.fit(X)
labels = km.labels_
centers = km.cluster_centers_
```

Aplicações reais



Clientes



Imagens



Mercado



Anomalias

Copyright © 2026
Slides do Prof. Felipe Souza Amaral - FIAP

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).